

Suites réelles et complexes, définitions rigoureuses des limites (ϵ, N) et critères de convergence

Charles EDOU NZE

10 juillet 2026

Table des matières

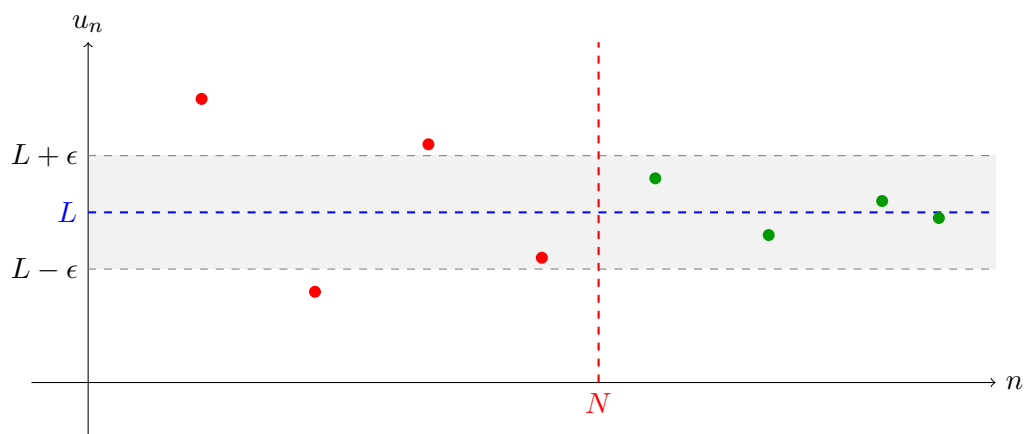
Chapitre 1

Intuition et genèse du concept

Jalon 14 : Suites réelles et complexes, définitions rigoureuses des limites (ϵ, N) et critères de convergence

1. Présentation du concept clé

Imaginez un archer qui s'entraîne sur une cible infiniment loin. Au début, ses flèches tombent un peu partout. Mais plus il tire, plus il devient précis. Une **suite qui converge**, c'est comme cet archer : après un certain nombre de tirs, toutes ses flèches finissent par tomber dans un cercle minuscule autour du centre. Peu importe la taille du cercle que vous lui imposez (aussi petit soit-il), l'archer finit toujours par réussir à mettre TOUTES ses flèches suivantes à l'intérieur. On ne peut pas toujours calculer une valeur exacte (comme π ou e). Mais on peut s'en approcher de plus en plus. Les suites sont les "chemins" vers ces valeurs. La définition rigoureuse (ϵ, N) permet de dire précisément quand on est "assez proche" pour que la différence ne compte plus. Imaginez des points sur un graphique qui sautillent. La limite est une ligne horizontale. Si la suite converge, les points finissent par "s'écraser" sur cette ligne et ne s'en éloignent plus jamais, même d'un millimètre.



Chapitre 2

Formalisation et structures algébriques

2. Formalisation

A. Définitions Formelles

Soit $(u_n)_{n \in \mathbb{N}}$ une suite d'éléments de $\mathbb{K}$ ($\mathbb{R}$ ou $\mathbb{C}$).

1. **Limite finie** (ϵ, N) : La suite (u_n) converge vers $l \in \mathbb{K}$ si :

$$\forall \epsilon > 0, \exists N \in \mathbb{N}, \forall n \geq N, |u_n - l| < \epsilon$$

1. **Limite infinie (pour $\mathbb{R}$)** : (u_n) tend vers $+\infty$ si :

$$\forall M \in \mathbb{R}, \exists N \in \mathbb{N}, \forall n \geq N, u_n > M$$

1. **Suite de Cauchy** : Une suite (u_n) est dite de Cauchy si ses termes se rapprochent les uns des autres :

$$\forall \epsilon > 0, \exists N \in \mathbb{N}, \forall p, q \geq N, |u_p - u_q| < \epsilon$$

B. Théorèmes, Propositions & Lemmes

> **Théorème de la Limite Unique** : > Si une suite converge, sa limite est unique.

> **Théorème de Convergence Monotone** : > Toute suite réelle croissante et majorée converge vers sa borne supérieure.

> **Complétude de $\mathbb{R}$ et $\mathbb{C}$** : > Dans $\mathbb{R}$ ou $\mathbb{C}$, une suite converge si et seulement si elle est de Cauchy.

3. Démonstrations

Démonstration du Théorème Pivot : Unicité de la limite

Supposons qu'une suite (u_n) converge vers deux limites distinctes l_1 et l_2 . Montrons que $l_1 = l_2$.

1. **Initialisation / Cadre** : Raisonnons par l'absurde. Supposons $l_1 \neq l_2$.

— Alors $|l_1 - l_2| > 0$.

— Posons $\epsilon = \frac{|l_1 - l_2|}{3}$. Remarquons que $\epsilon > 0$.

1. **Étape 1 : Utilisation de la convergence vers l_1**

Par définition de la limite pour l_1 , il existe $N_1 \in \mathbb{N}$ tel que :

$$\forall n \geq N_1, |u_n - l_1| < \epsilon$$

1. **Étape 2 : Utilisation de la convergence vers l_2**

Par définition de la limite pour l_2 , il existe $N_2 \in \mathbb{N}$ tel que :

$$\forall n \geq N_2, |u_n - l_2| < \epsilon$$

1. Étape 3 : Application de l'inégalité triangulaire

Soit $n \geq \max(N_1, N_2)$. Pour cet indice n , les deux inégalités précédentes sont vraies simultanément. Calculons la distance entre les deux limites supposées : $|l_1 - l_2| = |(l_1 - u_n) + (u_n - l_2)|$ Par l'inégalité triangulaire $|a + b| \leq |a| + |b|$: $|l_1 - l_2| \leq |l_1 - u_n| + |u_n - l_2|$ En utilisant la symétrie de la valeur absolue $|l_1 - u_n| = |u_n - l_1|$: $|l_1 - l_2| \leq |u_n - l_1| + |u_n - l_2|$

1. Étape 4 : Majoration par ϵ

En substituant les majorations des étapes 1 et 2 : $|l_1 - l_2| < \epsilon + \epsilon |l_1 - l_2| < 2\epsilon$ Substituons notre choix de $\epsilon = \frac{|l_1 - l_2|}{3}$: $|l_1 - l_2| < 2 \cdot \frac{|l_1 - l_2|}{3}$ $|l_1 - l_2| < \frac{2}{3}|l_1 - l_2|$

1. Conclusion :

Comme $|l_1 - l_2| > 0$, on peut diviser par cette valeur : $1 < \frac{2}{3}$. Ceci est une contradiction flagrante. Notre hypothèse de départ ($l_1 \neq l_2$) est donc fausse. La limite est unique : $l_1 = l_2$.

4. Exercices d'Application

Exercice 1 : Application de la définition (ϵ, N)

Énoncé : Démontrer, en utilisant uniquement la définition formelle, que $\lim_{n \rightarrow \infty} \frac{n+1}{n+2} = 1$.
Correction Détaillée :

1. Soit $\epsilon > 0$. Nous cherchons $N \in \mathbb{N}$ tel que pour $n \geq N$, $|\frac{n+1}{n+2} - 1| < \epsilon$.
2. Simplifions l'expression sous la valeur absolue :
 $|\frac{n+1-(n+2)}{n+2}| = |\frac{-1}{n+2}| = \frac{1}{n+2}$.
1. Nous voulons $\frac{1}{n+2} < \epsilon$.
2. Comme $n+2 > 0$ and $\epsilon > 0$, cela équivaut à $n+2 > \frac{1}{\epsilon}$, soit $n > \frac{1}{\epsilon} - 2$.
3. Choisissons N tel que $N > \frac{1}{\epsilon} - 2$ (par exemple $N = \lfloor \frac{1}{\epsilon} \rfloor$).
4. Alors pour tout $n \geq N$, on a bien $|u_n - 1| < \epsilon$.

Conclusion : La suite converge vers 1.

Exercice 2 : Niveau Avancé (Suite récurrente et point fixe)

Énoncé : Soit $u_0 = 1$ et $u_{n+1} = \sqrt{2 + u_n}$. Montrer que la suite converge et déterminer sa limite. **Correction Détaillée :**

1. **Initialisation :** $u_0 = 1, u_1 = \sqrt{3} \approx 1.732, u_2 = \sqrt{2 + \sqrt{3}} \approx 1.93$. La suite semble croissante et tendre vers 2.
2. **Récurrence (Bornes) :** Montrons par récurrence que $0 \leq u_n \leq 2$.
 — $u_0 = 1 \in [0, 2]$.
 — Supposons $0 \leq u_n \leq 2$. Alors $2 \leq 2 + u_n \leq 4 \implies \sqrt{2} \leq \sqrt{2 + u_n} \leq 2$.
 — On a bien $0 \leq u_{n+1} \leq 2$.
1. **Monotonie :** $u_{n+1} - u_n = \sqrt{2 + u_n} - u_n = \frac{2+u_n-u_n^2}{\sqrt{2+u_n}+u_n} = \frac{-(u_n-2)(u_n+1)}{\sqrt{2+u_n}+u_n}$.
 — Comme $u_n \in [0, 2]$, alors $(u_n - 2) \leq 0$ and $(u_n + 1) > 0$.
 — Donc $u_{n+1} - u_n \geq 0$. La suite est croissante.
1. **Convergence :** Croissante et majorée par 2, elle converge vers $l \in [0, 2]$.
2. **Limite :** l vérifie $l = \sqrt{2 + l} \implies l^2 - l - 2 = 0 \implies (l - 2)(l + 1) = 0$.
 — Comme $l \geq 0$, alors $l = 2$.

Conclusion : $\lim u_n = 2$.

5. Application en Intelligence Artificielle

- **Le Pont Théorique : L'Apprentissage (Learning)** est un processus itératif. On définit une suite de paramètres $W_0, W_1, \dots, W_n$. Dire que l'IA a "appris", c'est dire que cette suite converge vers un minimum de la fonction d'erreur.
- **Exemple Concret : Dans la Descente de Gradient Stochastique (SGD)**, on définit un "taux d'apprentissage" (Learning Rate) η_n . Pour que les poids du réseau convergent vers une solution stable, la suite des taux doit vérifier certaines propriétés de convergence (Critères de Robbins-Monro). Si la suite des poids ne converge pas (elle diverge ou oscille), le modèle ne sera jamais capable de faire des prédictions fiables.

6. Liens Sémantiques

- **Concepts Précédents requis** : [[Jalon 13 (Structure de R)]]
- **Concepts Futurs dépendants** : [[Jalon-15]], [[Jalon-16]], [[Jalon 21 (Suites de fonctions)]], [[Jalon 56 (Espaces métriques complets)]]

Exégèse Conceptuelle Additionnelle

La genèse conceptuelle de la convergence ne saurait s'arrêter à une simple manipulation algébrique. Historiquement, l'axiomatisation par Cauchy et Weierstrass visait à asseoir l'édifice analytique sur des bases arithmétiques irréfutables, expurgeant l'analyse de l'infinitésimal ambigu de Leibniz et Newton.

A. Typage Chirurgical et Énoncé Symbolique : Considérons le triplet $(\mathbb{K}, |\cdot|, \mathcal{V})$ où $\mathbb{K}$ est un corps topologique (usuellement $\mathbb{R}$ ou $\mathbb{C}$), muni de la valeur absolue standard, et $\mathcal{V}$ la base de voisinages. L'assertion de convergence s'écrit formellement :

$$\lim_{n \rightarrow \infty} u_n = \ell \iff \forall \varepsilon \in \mathbb{R}_+^*, \exists N(\varepsilon) \in \mathbb{N}, \forall n \in \mathbb{N}, (n \geq N(\varepsilon) \implies |u_n - \ell| < \varepsilon)$$

Chaque quantificateur possède un statut logique asymétrique. Le $\forall \varepsilon$ impose une contrainte externe globale (le "défi"), le $\exists N$ constitue la réponse adaptative (la "stratégie"), et l'implication finale certifie l'emprisonnement asymptotique définitif.

B. Cas Pathologiques et Frontières Topologiques : Si $\mathbb{K} = \mathbb{Q}$, la suite de terme général $u_n = (1 + 1/n)^n$ est de Cauchy, mais elle ne converge vers aucun élément de $\mathbb{Q}$, démontrant la nécessité structurelle de la complétude (l'axiome de la borne supérieure) pour garantir l'équivalence entre la propriété de Cauchy et la convergence.

C. Zéro Ellipse Mathématique - Majoration Asymptotique : Pour prouver formellement que $|u_n - \ell| < \varepsilon$, la méthode requiert l'inversion d'une inégalité fonctionnelle. Soit $\phi(n) = |u_n - \ell|$. Résoudre $\phi(n) < \varepsilon$ revient à déterminer la préimage $\phi^{-1}([0, \varepsilon])$. L'entier N est donc défini constructiblement comme $N = \lfloor \inf\{n \in \mathbb{N} \mid \phi(n) \geq \varepsilon\} \rfloor + 1$.

Chapitre 3

Démonstrations pas-à-pas

Chapitre 4

Exercices d'application et de concours

Exercice 1 : Démonstration de la convergence de la suite inverse par la définition $\epsilon - N$

Énoncé

Soit la suite réelle $(u_n)_{n \in \mathbb{N}^*}$ définie pour tout entier $n \geq 1$ par $u_n = \frac{1}{n}$.

En utilisant la définition formelle de la limite (dite "définition $\epsilon - N$ "), démontrez rigoureusement que cette suite converge vers 0.

Correction Détaillée

Étape 1 : Rappel de la définition formelle de la limite d'une suite. Commençons par énoncer clairement la définition que nous allons utiliser. Une suite réelle $(u_n)_{n \in \mathbb{N}^*}$ converge vers un réel L si et seulement si : Pour tout nombre réel $\epsilon > 0$ (aussi petit soit-il), il existe un entier naturel N (qui dépendra généralement de ϵ) tel que, pour tout entier n vérifiant $n \geq N$, on ait l'inégalité $|u_n - L| < \epsilon$. Notre objectif est de montrer que pour la suite $u_n = \frac{1}{n}$, la limite L est 0.

Étape 2 : Identification des éléments et traduction de l'inégalité. Dans notre cas précis, la suite est $u_n = \frac{1}{n}$ et la limite que nous souhaitons démontrer est $L = 0$. L'inégalité fondamentale de la définition de la limite est $|u_n - L| < \epsilon$. Substituons u_n et L par leurs valeurs respectives : $|\frac{1}{n} - 0| < \epsilon$

Étape 3 : Simplification de l'expression de l'inégalité. Simplifions l'expression à l'intérieur de la valeur absolue. $|\frac{1}{n} - 0| = |\frac{1}{n}|$. Puisque l'indice n appartient à $\mathbb{N}^*$, cela signifie que n est un entier strictement positif ($n \geq 1$). Par conséquent, $\frac{1}{n}$ est également un nombre strictement positif. Pour un nombre positif x , sa valeur absolue est x lui-même (c'est-à-dire $|x| = x$). Donc, $|\frac{1}{n}| = \frac{1}{n}$. L'inégalité à satisfaire se réduit donc à : $\frac{1}{n} < \epsilon$

Étape 4 : Détermination de la condition sur n en fonction de ϵ . Nous cherchons à trouver un entier N tel que pour tout $n \geq N$, l'inégalité $\frac{1}{n} < \epsilon$ soit vérifiée. Manipulons l'inégalité $\frac{1}{n} < \epsilon$ pour isoler n . Puisque $\epsilon > 0$ (par définition de la limite) et $n > 0$ (car $n \in \mathbb{N}^*$), nous pouvons multiplier les deux côtés de l'inégalité par n et diviser par ϵ sans changer le sens de l'inégalité. Multiplions par n : $1 < n\epsilon$ Divisons par ϵ : $\frac{1}{\epsilon} < n$ Ou, de manière équivalente : $n > \frac{1}{\epsilon}$ Cette dernière inégalité nous donne la condition que n doit satisfaire pour que $|u_n - L| < \epsilon$ soit vraie.

Étape 5 : Choix de l'entier N en fonction de ϵ . Nous devons trouver un entier N tel que si $n \geq N$, alors la condition $n > \frac{1}{\epsilon}$ est automatiquement satisfaite. Pour cela, il suffit de choisir N comme le plus petit entier qui est strictement supérieur à $\frac{1}{\epsilon}$. La fonction partie entière, notée $[x]$, donne le plus grand entier inférieur ou égal à x . Par exemple, $[3.14] = 3$ et $[5] = 5$. Donc, $[\frac{1}{\epsilon}]$ est un entier tel que $[\frac{1}{\epsilon}] \leq \frac{1}{\epsilon}$. Pour obtenir un entier N qui est *strictement* supérieur à $\frac{1}{\epsilon}$, nous pouvons choisir : $N = [\frac{1}{\epsilon}] + 1$ Ce choix garantit que N est un entier. De plus, par la

propriété de la partie entière, nous savons que $x < \lfloor x \rfloor + 1$. En appliquant cela à $x = \frac{1}{\epsilon}$, nous avons $\frac{1}{\epsilon} < \lfloor \frac{1}{\epsilon} \rfloor + 1$. Ainsi, notre choix de N satisfait $N > \frac{1}{\epsilon}$. Il est important de noter que N doit être un entier positif. Puisque $\epsilon > 0$, $\frac{1}{\epsilon} > 0$. Donc $\lfloor \frac{1}{\epsilon} \rfloor \geq 0$, et $N = \lfloor \frac{1}{\epsilon} \rfloor + 1 \geq 1$. Ce choix est donc valide pour $n \in \mathbb{N}^*$.

Étape 6 : Vérification formelle de la définition. Nous avons maintenant tous les éléments pour rédiger la démonstration formelle. Soit $\epsilon > 0$ un nombre réel arbitrairement choisi. Nous choisissons l'entier $N = \lfloor \frac{1}{\epsilon} \rfloor + 1$. Par la définition de la partie entière, nous savons que pour tout réel x , $\lfloor x \rfloor \leq x < \lfloor x \rfloor + 1$. En appliquant cette propriété à $x = \frac{1}{\epsilon}$, nous obtenons : $\lfloor \frac{1}{\epsilon} \rfloor < \frac{1}{\epsilon} < \lfloor \frac{1}{\epsilon} \rfloor + 1$. De cette inégalité, nous déduisons directement que $N = \lfloor \frac{1}{\epsilon} \rfloor + 1 > \frac{1}{\epsilon}$.

Considérons maintenant un entier n quelconque tel que $n \geq N$. Puisque $n \geq N$ et que nous avons établi que $N > \frac{1}{\epsilon}$, par la propriété de transitivité des inégalités, nous avons : $n > \frac{1}{\epsilon}$. Comme n et ϵ sont tous deux des nombres strictement positifs, nous pouvons prendre l'inverse des deux côtés de l'inégalité. Attention, prendre l'inverse de nombres positifs inverse le sens de l'inégalité : $\frac{1}{n} < \epsilon$. Et comme nous l'avons établi à l'Étape 3, $\frac{1}{n} = \left| \frac{1}{n} - 0 \right|$. Par conséquent, nous avons bien $\left| \frac{1}{n} - 0 \right| < \epsilon$ pour tout $n \geq N$.

Étape 7 : Conclusion. Nous avons démontré que pour tout $\epsilon > 0$, il existe un entier N (que nous avons construit explicitement comme $N = \lfloor \frac{1}{\epsilon} \rfloor + 1$) tel que pour tout entier $n \geq N$, l'inégalité $|u_n - 0| < \epsilon$ est satisfaite. Conformément à la définition formelle de la limite, nous pouvons donc affirmer que la suite $(u_n)_{n \in \mathbb{N}^*}$ définie par $u_n = \frac{1}{n}$ converge vers 0.

Ceci est un premier pas crucial dans la maîtrise des concepts de convergence en analyse. Félicitations pour cette première démonstration rigoureuse !

Exercice 2 : Convergence d'une suite rationnelle par la définition (ϵ, N)

Énoncé

Soit la suite de nombres réels $(u_n)_{n \in \mathbb{N}}$ définie pour tout entier naturel n par l'expression :

$$u_n = \frac{2n+1}{n+3}$$

En utilisant la définition rigoureuse de la limite d'une suite (la définition avec ϵ et N), démontrer que la suite (u_n) converge vers 2.

Correction Détaillée

Pour démontrer qu'une suite (u_n) converge vers une limite L en utilisant la définition rigoureuse, nous devons montrer que pour tout nombre réel $\epsilon > 0$, il existe un entier naturel N tel que pour tout entier naturel $n \geq N$, l'inégalité $|u_n - L| < \epsilon$ est vérifiée.

Dans cet exercice, la suite est $u_n = \frac{2n+1}{n+3}$ et la limite proposée est $L = 2$.

Étape 1 : Écrire l'inégalité à vérifier. Nous devons montrer que pour tout $\epsilon > 0$, il existe $N \in \mathbb{N}$ tel que pour tout $n \geq N$, nous ayons :

$$\left| \frac{2n+1}{n+3} - 2 \right| < \epsilon$$

Étape 2 : Simplifier l'expression $|u_n - L|$. Commençons par simplifier l'expression à l'intérieur de la valeur absolue :

$$\frac{2n+1}{n+3} - 2 = \frac{2n+1}{n+3} - \frac{2(n+3)}{n+3}$$

$$\begin{aligned}
&= \frac{2n+1-(2n+6)}{n+3} \\
&= \frac{2n+1-2n-6}{n+3} \\
&= \frac{-5}{n+3}
\end{aligned}$$

Maintenant, appliquons la valeur absolue :

$$\left| \frac{-5}{n+3} \right|$$

Puisque n est un entier naturel, $n \geq 0$. Par conséquent, $n+3$ est toujours positif ($n+3 \geq 3$). Ainsi, $\frac{-5}{n+3}$ est toujours un nombre négatif. La valeur absolue d'un nombre négatif est son opposé.

$$\left| \frac{-5}{n+3} \right| = - \left(\frac{-5}{n+3} \right) = \frac{5}{n+3}$$

L'inégalité à résoudre devient donc :

$$\frac{5}{n+3} < \epsilon$$

Étape 3 : Isoler n pour trouver une condition sur N . Nous cherchons un N tel que pour tout $n \geq N$, l'inégalité $\frac{5}{n+3} < \epsilon$ soit satisfaite. Puisque $\epsilon > 0$ et $n+3 > 0$, nous pouvons manipuler cette inégalité en toute sécurité (sans changer le sens des inégalités) :

$$\frac{5}{n+3} < \epsilon$$

Multiplions les deux côtés par $(n+3)$:

$$5 < \epsilon(n+3)$$

Divisons les deux côtés par ϵ (qui est strictement positif) :

$$\frac{5}{\epsilon} < n+3$$

Soustrayons 3 des deux côtés :

$$\frac{5}{\epsilon} - 3 < n$$

Donc, nous avons besoin que n soit strictement supérieur à $\frac{5}{\epsilon} - 3$.

Étape 4 : Choisir l'entier N . Nous devons choisir un entier naturel N tel que si $n \geq N$, alors $n > \frac{5}{\epsilon} - 3$. Un choix approprié pour N est la partie entière supérieure de $\frac{5}{\epsilon} - 3$, en s'assurant que N est un entier naturel (non-négatif).

$$N = \max \left(0, \left\lceil \frac{5}{\epsilon} - 3 \right\rceil \right)$$

* La fonction $\lceil x \rceil$ (plafond ou partie entière supérieure) donne le plus petit entier supérieur ou égal à x . Cela garantit que N est un entier. * La fonction $\max(0, \cdot)$ garantit que N est un entier naturel (puisque les indices de la suite commencent à $n = 0$). Par exemple, si ϵ est très grand (disons $\epsilon = 10$), alors $\frac{5}{10} - 3 = 0.5 - 3 = -2.5$. Dans ce cas, $\lceil -2.5 \rceil = -2$. Le $\max(0, -2)$ donne 0, ce qui est un N valide. Pour $n \geq 0$, $\frac{5}{n+3} \leq \frac{5}{3} \approx 1.67$, ce qui est bien inférieur à $\epsilon = 10$.

Étape 5 : Vérifier que ce choix de N fonctionne. Soit $\epsilon > 0$ un nombre réel arbitrairement petit. Choisissons $N = \max \left(0, \left\lceil \frac{5}{\epsilon} - 3 \right\rceil \right)$. Considérons tout entier naturel n tel que $n \geq N$. Par la définition de N , nous avons $N \geq \frac{5}{\epsilon} - 3$. Donc, $n \geq N \implies n \geq \frac{5}{\epsilon} - 3$. Puisque n est

un entier et $\lceil x \rceil$ est le plus petit entier supérieur ou égal à x , si $n \geq \lceil x \rceil$, alors $n \geq x$. Ainsi, $n > \frac{5}{\epsilon} - 3$. En remontant les étapes de l'inégalité :

$$n + 3 > \frac{5}{\epsilon}$$

Puisque $\epsilon > 0$ et $n + 3 > 0$, nous pouvons multiplier par ϵ et diviser par $n + 3$:

$$\epsilon > \frac{5}{n + 3}$$

Et comme nous avons établi à l'Étape 2 que $\frac{5}{n+3} = \left| \frac{2n+1}{n+3} - 2 \right|$, nous avons bien :

$$\left| \frac{2n+1}{n+3} - 2 \right| < \epsilon$$

Étape 6 : Conclusion. Nous avons démontré que pour tout $\epsilon > 0$, il existe un entier naturel $N = \max(0, \lceil \frac{5}{\epsilon} - 3 \rceil)$ tel que pour tout $n \geq N$, l'inégalité $|u_n - 2| < \epsilon$ est vérifiée. Par conséquent, selon la définition rigoureuse de la limite, la suite (u_n) converge vers 2.

Exercice 3 : Application de la définition de la limite pour une suite rationnelle

Énoncé

Soit la suite réelle $(u_n)_{n \in \mathbb{N}}$ définie pour tout entier naturel n par la relation $u_n = \frac{3n+2}{n+1}$.

Démontrer, en utilisant la définition rigoureuse de la limite d'une suite (la définition avec ϵ et N), que la suite (u_n) converge vers 3.

Correction Détaillée

Pour démontrer que la suite (u_n) converge vers 3 en utilisant la définition rigoureuse de la limite, nous devons montrer que pour tout réel $\epsilon > 0$, il existe un entier naturel N tel que pour tout entier naturel $n \geq N$, l'inégalité $|u_n - 3| < \epsilon$ est vérifiée.

1. Fixons un ϵ arbitraire :

Soit ϵ un nombre réel strictement positif ($\epsilon > 0$). Notre objectif est de trouver un entier N (qui dépendra de ϵ) tel que la condition de convergence soit satisfaite.

1. Calcul de la différence $|u_n - 3|$:

Nous commençons par exprimer la valeur absolue de la différence entre u_n et la limite supposée 3 :

$$|u_n - 3| = \left| \frac{3n+2}{n+1} - 3 \right|$$

Pour simplifier cette expression, nous mettons les termes sous un dénominateur commun :

$$|u_n - 3| = \left| \frac{3n+2}{n+1} - \frac{3(n+1)}{n+1} \right|$$

$$|u_n - 3| = \left| \frac{(3n+2) - (3n+3)}{n+1} \right|$$

$$|u_n - 3| = \left| \frac{3n+2-3n-3}{n+1} \right|$$

$$|u_n - 3| = \left| \frac{-1}{n+1} \right|$$

Puisque n est un entier naturel, $n \geq 0$. Par conséquent, $n+1$ est toujours strictement positif ($n+1 \geq 1$). La valeur absolue de $\frac{-1}{n+1}$ est donc $\frac{1}{n+1}$:

$$|u_n - 3| = \frac{1}{n+1}$$

1. Établissement de l'inégalité :

Nous voulons que cette différence soit inférieure à ϵ :

$$\frac{1}{n+1} < \epsilon$$

1. Résolution de l'inégalité pour n :

Puisque $\epsilon > 0$ et $n+1 > 0$, nous pouvons prendre l'inverse des deux côtés de l'inégalité, ce qui inverse le sens de l'inégalité :

$$n+1 > \frac{1}{\epsilon}$$

Maintenant, isolons n :

$$n > \frac{1}{\epsilon} - 1$$

1. Choix de l'entier N :

Nous devons trouver un entier N tel que pour tout $n \geq N$, la condition $n > \frac{1}{\epsilon} - 1$ soit satisfaite. Un choix approprié pour N est le plus petit entier supérieur ou égal à $\frac{1}{\epsilon} - 1$. Nous pouvons utiliser la fonction partie entière supérieure (plafond) ou simplement prendre un entier légèrement plus grand. Choisissons N comme l'entier naturel défini par :

$$N = \max \left(0, \left\lceil \frac{1}{\epsilon} - 1 \right\rceil + 1 \right)$$

Justification du choix de N : Si $\frac{1}{\epsilon} - 1$ est négatif ou nul (ce qui arrive si $\epsilon \geq 1$), alors $\left\lceil \frac{1}{\epsilon} - 1 \right\rceil + 1$ pourrait être 0 ou 1. Dans ce cas, $N = 0$ est suffisant car $n \geq 0$ implique $n > \frac{1}{\epsilon} - 1$ (par exemple, si $\epsilon = 2$, $1/\epsilon - 1 = -0.5$, $n > -0.5$ est vrai pour tout $n \geq 0$). Si $\frac{1}{\epsilon} - 1$ est positif, alors $N = \left\lceil \frac{1}{\epsilon} - 1 \right\rceil + 1$ est un entier strictement supérieur à $\frac{1}{\epsilon} - 1$. Ainsi, pour tout $n \geq N$, nous avons $n \geq N > \frac{1}{\epsilon} - 1$.

1. Conclusion :

Nous avons montré que pour tout $\epsilon > 0$, il existe un entier naturel $N = \max \left(0, \left\lceil \frac{1}{\epsilon} - 1 \right\rceil + 1 \right)$ tel que pour tout $n \geq N$, l'inégalité $n > \frac{1}{\epsilon} - 1$ est vérifiée. Cette inégalité implique successivement :

$$n+1 > \frac{1}{\epsilon}$$

$$\frac{1}{n+1} < \epsilon$$

$$|u_n - 3| < \epsilon$$

Par conséquent, selon la définition rigoureuse de la limite, la suite (u_n) converge vers 3.

$$\lim_{n \rightarrow \infty} \frac{3n+2}{n+1} = 3$$

Exercice 4 : Convergence d'une suite complexe par la définition (ϵ, N)

Énoncé

Soit la suite de nombres complexes $(u_n)_{n \in \mathbb{N}^*}$ définie pour tout $n \in \mathbb{N}^*$ par :

$$u_n = \frac{2n + i}{n + 2i}$$

1. Conjecturer la limite L de la suite (u_n) lorsque $n \rightarrow +\infty$.
2. Démontrer rigoureusement, en utilisant la définition formelle (ϵ, N) de la limite, que la suite (u_n) converge vers la limite L conjecturée.

Correction Détaillée

Partie 1 : Conjecturer la limite L

Pour conjecturer la limite de la suite (u_n) lorsque $n \rightarrow +\infty$, nous allons examiner le comportement de l'expression de u_n pour de grandes valeurs de n .

L'expression de u_n est une fraction dont le numérateur et le dénominateur sont des expressions linéaires en n (avec des termes constants complexes) :

$$u_n = \frac{2n + i}{n + 2i}$$

Pour analyser le comportement asymptotique, il est souvent utile de diviser le numérateur et le dénominateur par la plus haute puissance de n présente, qui est ici n .

$$u_n = \frac{\frac{2n}{n} + \frac{i}{n}}{\frac{n}{n} + \frac{2i}{n}}$$

$$u_n = \frac{2 + \frac{i}{n}}{1 + \frac{2i}{n}}$$

Maintenant, nous considérons le comportement de chaque terme lorsque $n \rightarrow +\infty$: * Le terme $\frac{i}{n}$ tend vers 0 lorsque $n \rightarrow +\infty$, car $|\frac{i}{n}| = \frac{|i|}{n} = \frac{1}{n}$, et $\frac{1}{n} \rightarrow 0$. * De même, le terme $\frac{2i}{n}$ tend vers 0 lorsque $n \rightarrow +\infty$, car $|\frac{2i}{n}| = \frac{|2i|}{n} = \frac{2}{n}$, et $\frac{2}{n} \rightarrow 0$.

En substituant ces limites dans l'expression de u_n , nous obtenons :

$$\lim_{n \rightarrow +\infty} u_n = \frac{2 + 0}{1 + 0} = \frac{2}{1} = 2$$

Nous conjecturons donc que la limite L de la suite (u_n) est 2.

Partie 2 : Démonstration rigoureuse par la définition (ϵ, N)

La définition formelle de la limite pour une suite de nombres complexes (u_n) convergeant vers un nombre complexe L est la suivante : Pour tout $\epsilon > 0$, il existe un entier naturel N (qui peut dépendre de ϵ) tel que pour tout $n > N$, on ait $|u_n - L| < \epsilon$.

Dans notre cas, $L = 2$. Nous devons donc montrer que pour tout $\epsilon > 0$, il existe un $N \in \mathbb{N}^*$ tel que pour tout $n > N$, $|u_n - 2| < \epsilon$.

Étape 1 : Calculer l'expression $|u_n - L|$

Nous commençons par calculer la différence $u_n - L$:

$$u_n - 2 = \frac{2n + i}{n + 2i} - 2$$

Pour combiner ces termes, nous mettons 2 au même dénominateur :

$$\begin{aligned}u_n - 2 &= \frac{2n + i}{n + 2i} - \frac{2(n + 2i)}{n + 2i} \\u_n - 2 &= \frac{(2n + i) - (2n + 4i)}{n + 2i} \\u_n - 2 &= \frac{2n + i - 2n - 4i}{n + 2i} \\u_n - 2 &= \frac{-3i}{n + 2i}\end{aligned}$$

Maintenant, nous calculons le module de cette expression :

$$|u_n - 2| = \left| \frac{-3i}{n + 2i} \right|$$

En utilisant la propriété du module $|z_1/z_2| = |z_1|/|z_2|$:

$$|u_n - 2| = \frac{|-3i|}{|n + 2i|}$$

Le module de $-3i$ est $|-3i| = \sqrt{0^2 + (-3)^2} = \sqrt{9} = 3$. Le module de $n + 2i$ est $|n + 2i| = \sqrt{n^2 + 2^2} = \sqrt{n^2 + 4}$. Donc, l'expression devient :

$$|u_n - 2| = \frac{3}{\sqrt{n^2 + 4}}$$

Étape 2 : Établir l'inégalité $|u_n - L| < \epsilon$ et résoudre pour n

Nous voulons trouver N tel que pour tout $n > N$, nous ayons :

$$\frac{3}{\sqrt{n^2 + 4}} < \epsilon$$

Puisque $\epsilon > 0$ et $\frac{3}{\sqrt{n^2 + 4}} > 0$, nous pouvons inverser l'inégalité en prenant les réciproques, ce qui change le sens de l'inégalité :

$$\frac{\sqrt{n^2 + 4}}{3} > \frac{1}{\epsilon}$$

Multiplions par 3 :

$$\sqrt{n^2 + 4} > \frac{3}{\epsilon}$$

Les deux membres de l'inégalité sont positifs, nous pouvons donc élever au carré sans changer le sens de l'inégalité :

$$\begin{aligned}n^2 + 4 &> \left(\frac{3}{\epsilon}\right)^2 \\n^2 + 4 &> \frac{9}{\epsilon^2}\end{aligned}$$

Isolons n^2 :

$$n^2 > \frac{9}{\epsilon^2} - 4$$

Étape 3 : Déterminer la valeur de N

Nous devons maintenant choisir N en fonction de ϵ . Nous devons considérer deux cas pour l'expression $\frac{9}{\epsilon^2} - 4$.

* **Cas 1** : $\frac{9}{\epsilon^2} - 4 \leq 0$ Cette condition est équivalente à $\frac{9}{\epsilon^2} \leq 4$, ce qui signifie $\epsilon^2 \geq \frac{9}{4}$, ou $\epsilon \geq \frac{3}{2}$. Dans ce cas, l'inégalité $n^2 > \frac{9}{\epsilon^2} - 4$ est toujours vraie pour tout $n \in \mathbb{N}^*$ (puisque $n^2 \geq 1$

et $\frac{9}{\epsilon^2} - 4$ est négatif ou nul). Par conséquent, pour $\epsilon \geq \frac{3}{2}$, nous pouvons choisir $N = 1$ (car la suite est définie pour $n \in \mathbb{N}^*$, donc $n > 1$ ou $n = 1$ suffit).

* **Cas 2 :** $\frac{9}{\epsilon^2} - 4 > 0$ Cette condition est équivalente à $\frac{9}{\epsilon^2} > 4$, ce qui signifie $\epsilon^2 < \frac{9}{4}$, ou $0 < \epsilon < \frac{3}{2}$. Dans ce cas, nous devons prendre la racine carrée des deux côtés de l'inégalité $n^2 > \frac{9}{\epsilon^2} - 4$ (les deux membres sont positifs) :

$$n > \sqrt{\frac{9}{\epsilon^2} - 4}$$

Pour garantir que $n > \sqrt{\frac{9}{\epsilon^2} - 4}$, nous pouvons choisir N comme le plus petit entier supérieur ou égal à $\sqrt{\frac{9}{\epsilon^2} - 4}$. Plus précisément, nous pouvons prendre $N = \lfloor \sqrt{\frac{9}{\epsilon^2} - 4} \rfloor + 1$. Puisque $n \in \mathbb{N}^*$, N doit être au moins 1.

Synthèse du choix de N : Pour couvrir les deux cas de manière élégante et rigoureuse, nous pouvons définir N comme suit :

$$N = \max \left(1, \left\lfloor \sqrt{\max \left(0, \frac{9}{\epsilon^2} - 4 \right)} \right\rfloor + 1 \right)$$

Expliquons ce choix : * Le terme $\max \left(0, \frac{9}{\epsilon^2} - 4 \right)$ garantit que nous prenons la racine carrée d'un nombre positif ou nul, même si $\frac{9}{\epsilon^2} - 4$ est négatif (dans ce cas, la racine est $\sqrt{0} = 0$). * La fonction $\lfloor \cdot \rfloor$ (partie entière inférieure) nous donne le plus grand entier inférieur ou égal à la valeur. * Ajouter 1 à $\lfloor \cdot \rfloor$ garantit que N est strictement supérieur à $\sqrt{\max \left(0, \frac{9}{\epsilon^2} - 4 \right)}$. * Le $\max(1, \dots)$ assure que N est toujours au moins 1, ce qui est nécessaire car $n \in \mathbb{N}^*$.

Conclusion de la démonstration : Soit $\epsilon > 0$ un nombre réel arbitrairement petit. Nous avons montré que pour que $|u_n - 2| < \epsilon$, il suffit que $n > \sqrt{\max \left(0, \frac{9}{\epsilon^2} - 4 \right)}$. Choisissons $N = \max \left(1, \left\lfloor \sqrt{\max \left(0, \frac{9}{\epsilon^2} - 4 \right)} \right\rfloor + 1 \right)$. Alors, pour tout $n \in \mathbb{N}^*$ tel que $n > N$, nous avons :

$$n > N \geq \left\lfloor \sqrt{\max \left(0, \frac{9}{\epsilon^2} - 4 \right)} \right\rfloor + 1 > \sqrt{\max \left(0, \frac{9}{\epsilon^2} - 4 \right)}$$

Ceci implique $n^2 > \max \left(0, \frac{9}{\epsilon^2} - 4 \right)$. Si $\frac{9}{\epsilon^2} - 4 \leq 0$, alors $n^2 > 0$, ce qui est vrai pour tout $n \in \mathbb{N}^*$. Si $\frac{9}{\epsilon^2} - 4 > 0$, alors $n^2 > \frac{9}{\epsilon^2} - 4$. Dans les deux cas, l'inégalité $n^2 > \frac{9}{\epsilon^2} - 4$ est satisfaite. En remontant les étapes de l'inégalité, nous obtenons successivement : $n^2 + 4 > \frac{9}{\epsilon^2}$ $\sqrt{n^2 + 4} > \frac{3}{\epsilon}$ $\frac{3}{\sqrt{n^2 + 4}} < \epsilon$ Et donc $|u_n - 2| < \epsilon$.

Puisque nous avons trouvé un tel N pour tout $\epsilon > 0$, la suite (u_n) converge bien vers 2 selon la définition formelle de la limite.

Exercice 5 : Convergence d'une suite rationnelle perturbée : application de la définition (ϵ, N)

Énoncé

Nous allons explorer la convergence d'une suite dont l'expression peut sembler un peu intimidante au premier abord, mais dont la limite est tout à fait accessible avec les outils appropriés. Cet exercice est conçu pour renforcer votre compréhension de la définition rigoureuse de la limite d'une suite.

Considérons la suite réelle $(u_n)_{n \in \mathbb{N}}$ définie pour tout $n \geq 2$ par :

$$u_n = \frac{n^2 + n \sin(n) + 5}{2n^2 - 3n + 1}$$

1. Détermination intuitive de la limite :

En utilisant des arguments de comparaison des ordres de grandeur des termes lorsque n tend vers l'infini (par exemple, en factorisant le terme dominant au numérateur et au dénominateur), déterminez la valeur L vers laquelle la suite (u_n) semble converger. Justifiez brièvement votre raisonnement.

1. Démonstration rigoureuse de la convergence :

En utilisant la définition rigoureuse de la limite (ϵ, N) , démontrez formellement que la suite (u_n) converge vers la valeur L trouvée à la question précédente. C'est-à-dire, pour tout $\epsilon > 0$, il existe un entier $N \in \mathbb{N}$ tel que pour tout $n > N$, on ait $|u_n - L| < \epsilon$.

Correction Détaillée

Abordons cet exercice avec la rigueur et la clarté qui s'imposent.

Question 1 : Détermination intuitive de la limite

Pour déterminer la limite de la suite (u_n) lorsque $n \rightarrow \infty$, nous allons analyser le comportement des termes dominants au numérateur et au dénominateur.

Nous avons :

$$u_n = \frac{n^2 + n \sin(n) + 5}{2n^2 - 3n + 1}$$

Factorisons le terme de plus haut degré, n^2 , au numérateur et au dénominateur :

$$u_n = \frac{n^2 \left(1 + \frac{\sin(n)}{n} + \frac{5}{n^2} \right)}{n^2 \left(2 - \frac{3}{n} + \frac{1}{n^2} \right)}$$

Pour $n \geq 2$, le dénominateur $2n^2 - 3n + 1$ est non nul. En effet, les racines du trinôme $2x^2 - 3x + 1$ sont $x = 1/2$ et $x = 1$. Ainsi, pour $n \geq 2$, $2n^2 - 3n + 1 > 0$. Nous pouvons donc simplifier par n^2 :

$$u_n = \frac{1 + \frac{\sin(n)}{n} + \frac{5}{n^2}}{2 - \frac{3}{n} + \frac{1}{n^2}}$$

Analysons le comportement de chaque terme lorsque $n \rightarrow \infty$: $\lim_{n \rightarrow \infty} \frac{5}{n^2} = 0$; $\lim_{n \rightarrow \infty} \frac{3}{n} = 0$; $\lim_{n \rightarrow \infty} \frac{1}{n^2} = 0$; Pour le terme $\frac{\sin(n)}{n}$, nous savons que $-1 \leq \sin(n) \leq 1$. Par conséquent, pour $n > 0$, nous avons $-\frac{1}{n} \leq \frac{\sin(n)}{n} \leq \frac{1}{n}$. Puisque $\lim_{n \rightarrow \infty} -\frac{1}{n} = 0$ et $\lim_{n \rightarrow \infty} \frac{1}{n} = 0$, le théorème des gendarmes (ou théorème d'encadrement) nous assure que $\lim_{n \rightarrow \infty} \frac{\sin(n)}{n} = 0$.

En substituant ces limites dans l'expression de u_n :

$$\lim_{n \rightarrow \infty} u_n = \frac{1 + 0 + 0}{2 - 0 + 0} = \frac{1}{2}$$

Ainsi, la limite intuitive de la suite (u_n) est $L = \frac{1}{2}$.

Question 2 : Démonstration rigoureuse de la convergence (définition ϵ, N)

Nous voulons démontrer que $\lim_{n \rightarrow \infty} u_n = \frac{1}{2}$ en utilisant la définition (ϵ, N) . Cela signifie que pour tout $\epsilon > 0$, nous devons trouver un entier N tel que pour tout $n > N$, nous ayons $|u_n - \frac{1}{2}| < \epsilon$.

Commençons par évaluer l'expression $|u_n - \frac{1}{2}|$:

$$|u_n - \frac{1}{2}| = \left| \frac{n^2 + n \sin(n) + 5}{2n^2 - 3n + 1} - \frac{1}{2} \right|$$

Pour combiner les fractions, nous mettons sur un dénominateur commun :

$$|u_n - \frac{1}{2}| = \left| \frac{2(n^2 + n \sin(n) + 5) - (2n^2 - 3n + 1)}{2(2n^2 - 3n + 1)} \right|$$

$$|u_n - \frac{1}{2}| = \left| \frac{2n^2 + 2n \sin(n) + 10 - 2n^2 + 3n - 1}{2(2n^2 - 3n + 1)} \right|$$

$$|u_n - \frac{1}{2}| = \left| \frac{2n \sin(n) + 3n + 9}{2(2n^2 - 3n + 1)} \right|$$

Maintenant, nous devons majorer le numérateur et minorer le dénominateur.

Majoration du numérateur : Nous utilisons l'inégalité triangulaire et le fait que $|\sin(n)| \leq 1$ pour tout $n \in \mathbb{N}$:

$$|2n \sin(n) + 3n + 9| \leq |2n \sin(n)| + |3n| + |9|$$

$$|2n \sin(n) + 3n + 9| \leq 2n|\sin(n)| + 3n + 9$$

$$|2n \sin(n) + 3n + 9| \leq 2n(1) + 3n + 9$$

$$|2n \sin(n) + 3n + 9| \leq 5n + 9$$

Pour $n \geq 1$, nous avons $9 \leq 9n$. Donc $5n + 9 \leq 5n + 9n = 14n$. Ainsi, pour $n \geq 1$, nous avons :

$$|2n \sin(n) + 3n + 9| \leq 14n$$

Minoration du dénominateur : Le dénominateur est $2(2n^2 - 3n + 1)$. Nous savons que pour $n \geq 2$, $2n^2 - 3n + 1 > 0$. Nous cherchons une minoration de la forme Cn^2 pour un certain $C > 0$. Considérons le terme $2n^2 - 3n + 1$. Pour $n \geq 3$, nous avons $n^2 - 3n + 1 \geq 0$ (car les racines de $x^2 - 3x + 1 = 0$ sont $\frac{3 \pm \sqrt{5}}{2}$, et $\frac{3 + \sqrt{5}}{2} \approx 2.618$). Donc, pour $n \geq 3$, $2n^2 - 3n + 1 = n^2 + (n^2 - 3n + 1) \geq n^2$. Par conséquent, pour $n \geq 3$:

$$2(2n^2 - 3n + 1) \geq 2n^2$$

Combinaison des majorations et minoration : En utilisant les inégalités obtenues pour $n \geq 3$:

$$|u_n - \frac{1}{2}| = \frac{|2n \sin(n) + 3n + 9|}{2(2n^2 - 3n + 1)} \leq \frac{14n}{2n^2}$$

$$|u_n - \frac{1}{2}| \leq \frac{7}{n}$$

Maintenant, nous voulons que $|u_n - \frac{1}{2}| < \epsilon$. Nous avons montré que $|u_n - \frac{1}{2}| \leq \frac{7}{n}$. Donc, si nous choisissons n tel que $\frac{7}{n} < \epsilon$, alors l'inégalité sera satisfaite.

$$\frac{7}{n} < \epsilon \iff n > \frac{7}{\epsilon}$$

Choix de N : Soit $\epsilon > 0$ donné. Nous devons choisir un entier N . Nous avons besoin que $n \geq 3$ pour que nos minoration et majoration soient valides. Nous choisissons $N = \max(2, \lceil \frac{7}{\epsilon} \rceil)$.

(Note : $n \geq 2$ est la condition pour que la suite soit bien définie. $n \geq 3$ est la condition pour nos bornes. $\lceil x \rceil$ désigne la partie entière par excès de x .)

Conclusion : Pour tout $\epsilon > 0$, choisissons $N = \max(2, \lceil \frac{7}{\epsilon} \rceil)$. Alors, pour tout $n > N$, nous avons $n \geq 3$ (car $N \geq 2$, et si $7/\epsilon < 3$, alors $N = 3$ ou plus). Et pour $n > N$, nous avons $n > \frac{7}{\epsilon}$, ce qui implique $\frac{7}{n} < \epsilon$. Par conséquent, pour tout $n > N$, nous avons :

$$|u_n - \frac{1}{2}| \leq \frac{7}{n} < \epsilon$$

Ceci démontre, par la définition (ϵ, N) , que la suite (u_n) converge vers $\frac{1}{2}$.

Ceci conclut notre exploration rigoureuse de la convergence de cette suite. Bravo pour votre persévérance !

Exercice 6 : Convergence Rigoureuse de Suites Complexes et Réelles : Définitions et Contre-exemples

Énoncé

Cet exercice explore la définition rigoureuse de la limite pour les suites réelles et complexes, ainsi que les critères de convergence.

Partie A : Convergence d'une suite complexe Soit la suite de nombres complexes $(z_n)_{n \in \mathbb{N}^*}$ définie par $z_n = \frac{n+i \sin(n)}{n^2+1}$. En utilisant la définition rigoureuse de la limite (ϵ, N) , démontrez que la suite (z_n) converge vers 0.

Partie B : Propriétés des limites pour les parties réelle et imaginaire Soit $(z_n)_{n \in \mathbb{N}}$ une suite de nombres complexes. On suppose que (z_n) converge vers un nombre complexe L . En utilisant la définition rigoureuse de la limite (ϵ, N) , démontrez que la suite des parties réelles $(\operatorname{Re}(z_n))_{n \in \mathbb{N}}$ converge vers $\operatorname{Re}(L)$ et que la suite des parties imaginaires $(\operatorname{Im}(z_n))_{n \in \mathbb{N}}$ converge vers $\operatorname{Im}(L)$.

Partie C : Non-convergence d'une suite complexe Soit la suite de nombres complexes $(w_n)_{n \in \mathbb{N}}$ définie par $w_n = e^{in\pi/2}$. En utilisant la négation de la définition rigoureuse de la limite (ϵ, N) , démontrez que la suite (w_n) ne converge pas.

Correction Détaillée

Partie A : Convergence d'une suite complexe

1. Rappel de la définition de la convergence pour une suite complexe :

Une suite de nombres complexes (z_n) converge vers un nombre complexe $L \in \mathbb{C}$ si et seulement si pour tout nombre réel $\epsilon > 0$, il existe un entier naturel $N \in \mathbb{N}$ tel que pour tout entier n vérifiant $n > N$, on a $|z_n - L| < \epsilon$.

1. Identification de la suite et de la limite proposée :

La suite donnée est $z_n = \frac{n+i \sin(n)}{n^2+1}$. La limite que nous devons démontrer est $L = 0$.

1. Calcul de la distance $|z_n - L|$:

Nous devons évaluer l'expression $|z_n - 0|$. $|z_n - 0| = \left| \frac{n+i \sin(n)}{n^2+1} \right|$. En utilisant la propriété du module d'un quotient, qui stipule que pour des nombres complexes a et b avec $b \neq 0$, $|a/b| = |a|/|b|$: $|z_n - 0| = \frac{|n+i \sin(n)|}{|n^2+1|}$. Puisque $n \in \mathbb{N}^*$, n^2+1 est un nombre réel strictement positif. Par conséquent, $|n^2+1| = n^2+1$. Le numérateur est le module d'un nombre complexe de la forme $a + ib$, qui est $\sqrt{a^2 + b^2}$. Ici, $a = n$ et $b = \sin(n)$. Donc, $|n + i \sin(n)| = \sqrt{n^2 + (\sin(n))^2}$. En substituant ces expressions, nous obtenons : $|z_n - 0| = \frac{\sqrt{n^2 + \sin^2(n)}}{n^2+1}$.

1. Majoration de l'expression pour faciliter la recherche de N :

Nous cherchons à majorer cette expression par une quantité qui tend vers 0 lorsque n tend vers l'infini et qui est plus simple à manipuler. Nous savons que pour tout $n \in \mathbb{N}^*$, la fonction sinus est bornée entre -1 et 1, donc $0 \leq \sin^2(n) \leq 1$. En ajoutant n^2 à chaque partie de l'inégalité : $n^2 \leq n^2 + \sin^2(n) \leq n^2 + 1$. La fonction racine carrée est croissante sur $[0, +\infty)$, donc en prenant la racine carrée de chaque partie : $\sqrt{n^2} \leq \sqrt{n^2 + \sin^2(n)} \leq \sqrt{n^2 + 1}$. Puisque $n \in \mathbb{N}^*$, $\sqrt{n^2} = n$. Donc : $n \leq \sqrt{n^2 + \sin^2(n)} \leq \sqrt{n^2 + 1}$. Nous utilisons la majoration $\sqrt{n^2 + \sin^2(n)} \leq \sqrt{n^2 + 1}$. Alors, l'expression de la distance devient : $|z_n - 0| \leq \frac{\sqrt{n^2 + 1}}{n^2 + 1}$. Nous pouvons simplifier cette fraction en utilisant la propriété $\frac{\sqrt{X}}{X} = \frac{1}{\sqrt{X}}$ pour tout $X > 0$. Ici, $X = n^2 + 1$. Donc, $|z_n - 0| \leq \frac{1}{\sqrt{n^2 + 1}}$.

1. Recherche d'un entier N en fonction de ϵ :

Nous voulons trouver un N tel que pour tout $n > N$, l'inégalité $\frac{1}{\sqrt{n^2 + 1}} < \epsilon$ soit satisfaite. Puisque $\epsilon > 0$, nous pouvons prendre l'inverse des deux côtés de l'inégalité (ce qui inverse le sens de l'inégalité car les termes sont positifs) : $\sqrt{n^2 + 1} > \frac{1}{\epsilon}$. Élevons les deux côtés au carré (les deux côtés sont positifs) : $n^2 + 1 > \frac{1}{\epsilon^2}$. Soustrayons 1 des deux côtés : $n^2 > \frac{1}{\epsilon^2} - 1$. Puisque $n \in \mathbb{N}^*$, n est positif, donc nous pouvons prendre la racine carrée des deux côtés : $n > \sqrt{\frac{1}{\epsilon^2} - 1}$. Pour garantir que n satisfait cette condition, nous pouvons choisir N comme le plus grand entier inférieur ou égal à $\sqrt{\frac{1}{\epsilon^2} - 1}$. Une manière standard et toujours valide est de prendre $N = \max(0, \lceil \sqrt{\frac{1}{\epsilon^2} - 1} \rceil)$. Pour simplifier la détermination de N , nous pouvons utiliser une majoration plus lâche mais suffisante. Pour $n \geq 1$, nous avons $n^2 + 1 \geq n^2$, ce qui implique $\sqrt{n^2 + 1} \geq \sqrt{n^2} = n$. Par conséquent, $\frac{1}{\sqrt{n^2 + 1}} \leq \frac{1}{n}$. Si nous voulons que $\frac{1}{n} < \epsilon$, cela implique $n > \frac{1}{\epsilon}$. Nous pouvons donc choisir $N = \lceil \frac{1}{\epsilon} \rceil$.

1. Conclusion formelle :

Soit $\epsilon > 0$ un nombre réel arbitrairement petit. Choisissons l'entier $N = \lceil \frac{1}{\epsilon} \rceil$. (Si $\epsilon \geq 1$, alors $N = 1$ est suffisant, car $1/n < 1 \leq \epsilon$ pour $n > 1$. Si $\epsilon < 1$, alors $N \geq 1$). Alors, pour tout entier n tel que $n > N$, nous avons par définition de N : $n > \frac{1}{\epsilon}$. En prenant l'inverse des deux côtés (puisque n et ϵ sont positifs), l'inégalité change de sens : $\frac{1}{n} < \epsilon$. Nous avons établi précédemment que $|z_n - 0| \leq \frac{1}{\sqrt{n^2 + 1}}$. Nous avons également montré que pour $n \geq 1$, $n^2 + 1 \geq n^2$, ce qui implique $\sqrt{n^2 + 1} \geq n$. Par conséquent, $\frac{1}{\sqrt{n^2 + 1}} \leq \frac{1}{n}$. En combinant ces inégalités, pour tout $n > N$, nous avons : $|z_n - 0| \leq \frac{1}{\sqrt{n^2 + 1}} \leq \frac{1}{n} < \epsilon$. Ceci démontre, par la définition rigoureuse de la limite, que la suite (z_n) converge vers 0.

Partie B : Propriétés des limites pour les parties réelle et imaginaire

1. Rappel de la définition de la convergence pour (z_n) :

La suite (z_n) converge vers $L \in \mathbb{C}$ signifie que pour tout $\epsilon' > 0$, il existe un entier $N \in \mathbb{N}$ tel que pour tout $n > N$, on a $|z_n - L| < \epsilon'$.

1. Objectif pour la convergence des parties réelles :

Nous voulons montrer que la suite des parties réelles $(\operatorname{Re}(z_n))$ converge vers $\operatorname{Re}(L)$. Cela signifie que pour tout $\epsilon > 0$, il existe un entier $N_R \in \mathbb{N}$ tel que pour tout $n > N_R$, on a $|\operatorname{Re}(z_n) - \operatorname{Re}(L)| < \epsilon$.

1. Lien entre le module d'une différence de complexes et la différence de leurs parties réelles/imaginaires :

Soit $z_n = x_n + iy_n$, où $x_n = \operatorname{Re}(z_n)$ et $y_n = \operatorname{Im}(z_n)$. Soit $L = x_L + iy_L$, où $x_L = \operatorname{Re}(L)$ et $y_L = \operatorname{Im}(L)$. Alors la différence $z_n - L$ peut s'écrire : $z_n - L = (x_n + iy_n) - (x_L + iy_L) = (x_n - x_L) + i(y_n - y_L)$. Le module de cette différence est $|z_n - L| = \sqrt{(x_n - x_L)^2 + (y_n - y_L)^2}$. Nous savons que pour tout nombre complexe $Z = A + iB$, la partie réelle A et la partie imaginaire B sont liées au module $|Z|$ par les inégalités : $|A| \leq \sqrt{A^2 + B^2} = |Z|$ et $|B| \leq \sqrt{A^2 + B^2} = |Z|$.

Appliquons ces inégalités au nombre complexe $Z = z_n - L$, avec $A = x_n - x_L$ et $B = y_n - y_L$. Nous obtenons : $|\operatorname{Re}(z_n) - \operatorname{Re}(L)| = |x_n - x_L| \leq \sqrt{(x_n - x_L)^2 + (y_n - y_L)^2} = |z_n - L|$. De même : $|\operatorname{Im}(z_n) - \operatorname{Im}(L)| = |y_n - y_L| \leq \sqrt{(x_n - x_L)^2 + (y_n - y_L)^2} = |z_n - L|$.

1. Démonstration de la convergence de $(\operatorname{Re}(z_n))$:

Soit $\epsilon > 0$ un nombre réel arbitrairement petit. Puisque la suite (z_n) converge vers L , par la définition de la convergence (avec $\epsilon' = \epsilon$), il existe un entier $N \in \mathbb{N}$ tel que pour tout $n > N$, on a $|z_n - L| < \epsilon$. En utilisant l'inégalité établie au point 3, pour tout $n > N$, nous avons : $|\operatorname{Re}(z_n) - \operatorname{Re}(L)| \leq |z_n - L|$. Puisque nous savons que $|z_n - L| < \epsilon$ pour $n > N$, il s'ensuit que : $|\operatorname{Re}(z_n) - \operatorname{Re}(L)| < \epsilon$. Nous avons donc trouvé un entier $N_R = N$ tel que pour tout $n > N_R$, l'inégalité $|\operatorname{Re}(z_n) - \operatorname{Re}(L)| < \epsilon$ est satisfaite. Ceci démontre que la suite des parties réelles $(\operatorname{Re}(z_n))$ converge vers $\operatorname{Re}(L)$.

1. Démonstration de la convergence de $(\operatorname{Im}(z_n))$:

Soit $\epsilon > 0$ un nombre réel arbitrairement petit. Puisque la suite (z_n) converge vers L , par la définition de la convergence (avec $\epsilon' = \epsilon$), il existe un entier $N \in \mathbb{N}$ tel que pour tout $n > N$, on a $|z_n - L| < \epsilon$. En utilisant l'inégalité établie au point 3, pour tout $n > N$, nous avons : $|\operatorname{Im}(z_n) - \operatorname{Im}(L)| \leq |z_n - L|$. Puisque nous savons que $|z_n - L| < \epsilon$ pour $n > N$, il s'ensuit que : $|\operatorname{Im}(z_n) - \operatorname{Im}(L)| < \epsilon$. Nous avons donc trouvé un entier $N_I = N$ tel que pour tout $n > N_I$, l'inégalité $|\operatorname{Im}(z_n) - \operatorname{Im}(L)| < \epsilon$ est satisfaite. Ceci démontre que la suite des parties imaginaires $(\operatorname{Im}(z_n))$ converge vers $\operatorname{Im}(L)$.

Partie C : Non-convergence d'une suite complexe

1. Rappel de la définition de la convergence :

Une suite (w_n) converge vers $L \in \mathbb{C}$ si et seulement si pour tout $\epsilon > 0$, il existe un entier $N \in \mathbb{N}$ tel que pour tout $n > N$, on a $|w_n - L| < \epsilon$.

1. Stratégie pour démontrer la non-convergence :

Pour démontrer qu'une suite ne converge pas, nous pouvons utiliser la négation de la définition de la convergence. Cependant, une approche souvent plus efficace pour les suites dans un espace complet (comme $\mathbb{C}$) est de montrer qu'elle n'est pas une suite de Cauchy. En effet, toute suite convergente dans un espace complet est une suite de Cauchy. Si une suite n'est pas de Cauchy, alors elle ne peut pas être convergente.

Rappel de la définition d'une suite de Cauchy : Une suite (w_n) est de Cauchy si et seulement si pour tout $\epsilon > 0$, il existe un entier $N \in \mathbb{N}$ tel que pour tous entiers p, q vérifiant $p > N$ et $q > N$, on a $|w_p - w_q| < \epsilon$.

Négation de la définition d'une suite de Cauchy : Une suite (w_n) n'est *pas* de Cauchy si et seulement s'il existe un nombre réel $\epsilon_0 > 0$ tel que pour tout entier $N \in \mathbb{N}$, il existe des entiers p, q vérifiant $p > N$ et $q > N$ pour lesquels $|w_p - w_q| \geq \epsilon_0$.

1. Analyse de la suite (w_n) :

La suite est définie par $w_n = e^{in\pi/2}$. Calculons les premiers termes de la suite pour comprendre son comportement : Pour $n = 0$: $w_0 = e^{i \cdot 0 \cdot \pi/2} = e^0 = 1$. Pour $n = 1$: $w_1 = e^{i\pi/2} = \cos(\pi/2) + i \sin(\pi/2) = 0 + i \cdot 1 = i$. Pour $n = 2$: $w_2 = e^{i2\pi/2} = e^{i\pi} = \cos(\pi) + i \sin(\pi) = -1 + i \cdot 0 = -1$. Pour $n = 3$: $w_3 = e^{i3\pi/2} = \cos(3\pi/2) + i \sin(3\pi/2) = 0 + i \cdot (-1) = -i$. Pour $n = 4$: $w_4 = e^{i4\pi/2} = e^{i2\pi} = \cos(2\pi) + i \sin(2\pi) = 1 + i \cdot 0 = 1$. Nous observons que la suite est périodique de période 4, prenant successivement les valeurs $1, i, -1, -i$, puis répétant ce cycle.

1. Démonstration par la négation de la définition de Cauchy :

Nous allons montrer que la suite (w_n) n'est pas une suite de Cauchy. Choisissons un $\epsilon_0 > 0$. Par exemple, prenons $\epsilon_0 = 1$. Nous devons montrer que pour tout entier $N \in \mathbb{N}$, il existe des entiers p, q tels que $p > N$, $q > N$, et $|w_p - w_q| \geq \epsilon_0$.

Soit N un entier naturel quelconque. Nous devons trouver $p, q > N$ tels que la distance entre w_p et w_q soit au moins ϵ_0 . Considérons les termes de la suite qui sont 1 et -1 . La distance entre eux est $|1 - (-1)| = |2| = 2$. Nous pouvons toujours trouver des indices p et q supérieurs à N tels que $w_p = 1$ et $w_q = -1$. Choisissons un entier k tel que $4k > N$. Par exemple, $k = \lfloor N/4 \rfloor + 1$. Posons $p = 4k$. Alors p est un multiple de 4, donc $p > N$. Le terme w_p est $w_{4k} = e^{i4k\pi/2} = e^{i2k\pi}$. Puisque $e^{i2k\pi} = \cos(2k\pi) + i \sin(2k\pi) = 1 + i \cdot 0 = 1$. Posons $q = 4k + 2$. Alors $q = p + 2$, donc $q > N$. Le terme w_q est $w_{4k+2} = e^{i(4k+2)\pi/2} = e^{i(2k\pi+\pi)}$. En utilisant la propriété $e^{A+B} = e^A e^B$, nous avons $e^{i(2k\pi+\pi)} = e^{i2k\pi} e^{i\pi}$. Nous savons que $e^{i2k\pi} = 1$ et $e^{i\pi} = \cos(\pi) + i \sin(\pi) = -1 + i \cdot 0 = -1$. Donc, $w_q = 1 \cdot (-1) = -1$.

Maintenant, calculons la distance entre w_p et w_q pour ces choix de p et q : $|w_p - w_q| = |1 - (-1)| = |1 + 1| = |2| = 2$. Puisque $2 \geq \epsilon_0 = 1$, nous avons trouvé, pour tout N (en choisissant $p = 4(\lfloor N/4 \rfloor + 1)$ et $q = p + 2$), des indices $p, q > N$ tels que $|w_p - w_q| \geq \epsilon_0$. Ceci démontre que la suite (w_n) n'est pas une suite de Cauchy.

1. Conclusion finale :

L'espace des nombres complexes $\mathbb{C}$ est un espace métrique complet. Dans un espace complet, toute suite convergente est nécessairement une suite de Cauchy. Puisque nous avons démontré que la suite (w_n) n'est pas une suite de Cauchy, il s'ensuit qu'elle ne peut pas être convergente.

Exercice 7 : Critère de Cauchy pour une suite contractante

Énoncé

Nous allons aujourd'hui explorer un aspect fondamental de la convergence des suites, à savoir le critère de Cauchy. Ce critère est d'une importance capitale car il permet de démontrer la convergence d'une suite sans avoir besoin de connaître sa limite a priori. Dans un espace complet comme $\mathbb{R}$ ou $\mathbb{C}$, une suite est convergente si et seulement si elle est de Cauchy.

Soit $(u_n)_{n \in \mathbb{N}}$ une suite de nombres réels. On suppose que cette suite satisfait la condition suivante : il existe une constante $C > 0$ et un réel $r \in]0, 1[$ tels que pour tout entier naturel n , nous ayons $|u_{n+1} - u_n| \leq C \cdot r^n$. Dans le cadre de cet exercice, nous allons considérer le cas particulier où $C = 1$ et $r = 1/2$. Autrement dit, la suite $(u_n)_{n \in \mathbb{N}}$ vérifie la propriété :

$$\forall n \in \mathbb{N}, \quad |u_{n+1} - u_n| \leq \frac{1}{2^n}$$

Votre tâche est de démontrer, en utilisant la définition rigoureuse d'une suite de Cauchy, que la suite $(u_n)_{n \in \mathbb{N}}$ est une suite de Cauchy.

Rappel : Une suite $(u_n)_{n \in \mathbb{N}}$ est dite de Cauchy si pour tout $\epsilon > 0$, il existe un entier naturel N tel que pour tous entiers p, q vérifiant $p > N$ et $q > N$, on ait $|u_p - u_q| < \epsilon$.

Correction Détaillée

Pour démontrer que la suite $(u_n)_{n \in \mathbb{N}}$ est une suite de Cauchy, nous devons suivre la définition rigoureuse.

1. Rappel de la définition d'une suite de Cauchy.

Une suite $(u_n)_{n \in \mathbb{N}}$ est une suite de Cauchy si et seulement si :

$$\forall \epsilon > 0, \exists N \in \mathbb{N} \text{ tel que } \forall p, q \in \mathbb{N}, \text{ si } p > N \text{ et } q > N, \text{ alors } |u_p - u_q| < \epsilon$$

1. Fixer ϵ et choisir p, q .

Soit ϵ un nombre réel strictement positif arbitrairement choisi. Notre objectif est de trouver un entier naturel N (qui dépendra de ϵ) tel que la condition de Cauchy soit satisfaite pour tous $p, q > N$. Considérons deux entiers p et q tels que $p > N$ et $q > N$. Sans perte de généralité, nous pouvons supposer que $p > q$. (Si $q > p$, on peut échanger les rôles de p et q , car $|u_p - u_q| = |u_q - u_p|$. Si $p = q$, alors $|u_p - u_q| = 0$, ce qui est trivialement inférieur à tout $\epsilon > 0$).

1. Décomposition de la différence $|u_p - u_q|$.

Puisque $p > q$, nous pouvons exprimer la différence $u_p - u_q$ comme une somme télescopique de différences consécutives. En ajoutant et soustrayant des termes intermédiaires, nous obtenons :

$$u_p - u_q = (u_p - u_{p-1}) + (u_{p-1} - u_{p-2}) + \cdots + (u_{q+1} - u_q)$$

Cette somme peut être écrite de manière plus compacte en utilisant le symbole de sommation :

$$u_p - u_q = \sum_{k=q}^{p-1} (u_{k+1} - u_k)$$

1. Application de l'inégalité triangulaire.

En prenant la valeur absolue de cette somme, nous pouvons appliquer l'inégalité triangulaire généralisée, qui stipule que la valeur absolue d'une somme est inférieure ou égale à la somme des valeurs absolues :

$$|u_p - u_q| = \left| \sum_{k=q}^{p-1} (u_{k+1} - u_k) \right| \leq \sum_{k=q}^{p-1} |u_{k+1} - u_k|$$

1. Utilisation de l'hypothèse de l'exercice.

L'énoncé nous donne une majoration pour chaque terme $|u_{k+1} - u_k|$:

$$|u_{k+1} - u_k| \leq \frac{1}{2^k} \quad \text{pour tout } k \in \mathbb{N}$$

En substituant cette inégalité dans l'expression précédente, nous obtenons :

$$|u_p - u_q| \leq \sum_{k=q}^{p-1} \frac{1}{2^k}$$

1. Calcul de la somme de la série géométrique.

La somme $\sum_{k=q}^{p-1} \frac{1}{2^k}$ est une somme partielle d'une série géométrique de raison $r = 1/2$. Nous pouvons factoriser le premier terme $\frac{1}{2^q}$:

$$\sum_{k=q}^{p-1} \frac{1}{2^k} = \frac{1}{2^q} + \frac{1}{2^{q+1}} + \cdots + \frac{1}{2^{p-1}} = \frac{1}{2^q} \left(1 + \frac{1}{2} + \cdots + \frac{1}{2^{p-1-q}} \right)$$

La somme entre parenthèses est une somme partielle d'une série géométrique de $p - q$ termes, commençant par $1 = (1/2)^0$. La formule pour la somme des m premiers termes d'une série géométrique $1 + r + \cdots + r^{m-1}$ est $\frac{1-r^m}{1-r}$. Ici, $m = p - q$ et $r = 1/2$. Donc, la somme entre parenthèses est :

$$1 + \frac{1}{2} + \cdots + \frac{1}{2^{p-1-q}} = \frac{1 - (1/2)^{p-q}}{1 - 1/2} = \frac{1 - (1/2)^{p-q}}{1/2} = 2 \left(1 - \left(\frac{1}{2} \right)^{p-q} \right)$$

En substituant cela dans l'expression de la somme, nous obtenons :

$$\sum_{k=q}^{p-1} \frac{1}{2^k} = \frac{1}{2^q} \cdot 2 \left(1 - \left(\frac{1}{2} \right)^{p-q} \right) = \frac{2}{2^q} \left(1 - \left(\frac{1}{2} \right)^{p-q} \right) = \frac{1}{2^{q-1}} \left(1 - \left(\frac{1}{2} \right)^{p-q} \right)$$

1. Majoration de l'expression.

Puisque $p > q$, l'exposant $p - q$ est un entier strictement positif. Par conséquent, $(1/2)^{p-q}$ est un nombre positif strictement inférieur à 1. Donc, $1 - (1/2)^{p-q} < 1$. Ainsi, nous pouvons majorer l'expression de $|u_p - u_q|$:

$$|u_p - u_q| \leq \frac{1}{2^{q-1}} \left(1 - \left(\frac{1}{2} \right)^{p-q} \right) < \frac{1}{2^{q-1}}$$

1. Détermination de N .

Nous voulons que $|u_p - u_q| < \epsilon$. D'après l'étape précédente, il suffit de s'assurer que $\frac{1}{2^{q-1}} < \epsilon$. Cette inégalité est équivalente à :

$$2^{q-1} > \frac{1}{\epsilon}$$

Pour résoudre cette inégalité pour q , nous prenons le logarithme en base 2 (ou le logarithme naturel, puis divisons par $\ln 2$) :

$$\log_2(2^{q-1}) > \log_2\left(\frac{1}{\epsilon}\right)$$

$$q - 1 > -\log_2(\epsilon)$$

$$q > 1 - \log_2(\epsilon)$$

Puisque nous avons supposé $p > q > N$, il suffit de choisir N tel que $N \geq 1 - \log_2(\epsilon)$. Pour s'assurer que N est un entier naturel, nous pouvons choisir N comme le plus petit entier supérieur ou égal à $1 - \log_2(\epsilon)$, et s'assurer qu'il est non négatif :

$$N = \max(0, \lfloor 1 - \log_2(\epsilon) \rfloor + 1)$$

Un tel entier N existe toujours pour tout $\epsilon > 0$.

1. Conclusion.

Pour tout $\epsilon > 0$, nous avons trouvé un entier naturel $N = \max(0, \lfloor 1 - \log_2(\epsilon) \rfloor + 1)$ tel que pour tous $p, q \in \mathbb{N}$ avec $p > N$ et $q > N$, nous avons $|u_p - u_q| < \epsilon$. Par conséquent, la suite $(u_n)_{n \in \mathbb{N}}$ est une suite de Cauchy.

Exercice 8 : Convergence rigoureuse d'une suite de racines polynomiales

Énoncé

Soit, pour tout entier naturel $n \geq 1$, l'équation polynomiale $P_n(x) = x^n + x - 1 = 0$.

1. Démontrer que pour chaque $n \geq 1$, l'équation $P_n(x) = 0$ admet une unique solution réelle u_n dans l'intervalle $(0, 1)$.
2. Établir que la suite $(u_n)_{n \geq 1}$ est strictement croissante. En déduire qu'elle est convergente.
3. Déterminer la limite L de la suite $(u_n)_{n \geq 1}$.
4. En utilisant la définition rigoureuse de la limite (ϵ, N) , démontrer que $\lim_{n \rightarrow \infty} u_n = L$.

Correction Détaillée

1. Existence et unicité de u_n dans $(0, 1)$

Pour chaque $n \geq 1$, considérons la fonction $P_n(x) = x^n + x - 1$ définie sur $\mathbb{R}$.

Existence : Évaluons $P_n(x)$ aux bornes de l'intervalle $(0, 1)$: * $P_n(0) = 0^n + 0 - 1 = -1$. * $P_n(1) = 1^n + 1 - 1 = 1$. Puisque $P_n(0) = -1 < 0$ et $P_n(1) = 1 > 0$, et que $P_n(x)$ est une fonction continue sur $[0, 1]$ (étant un polynôme), le Théorème des Valeurs Intermédiaires (TVI) garantit l'existence d'au moins une racine u_n dans l'intervalle $(0, 1)$.

Unicité : Calculons la dérivée de $P_n(x)$ par rapport à x : $P'_n(x) = \frac{d}{dx}(x^n + x - 1) = nx^{n-1} + 1$. Pour $x \in (0, 1)$, $x^{n-1} > 0$ (sauf si $n = 1$ et $x = 0$, mais nous sommes sur $(0, 1)$). Donc, $nx^{n-1} > 0$ pour $n \geq 1$ et $x \in (0, 1)$. Par conséquent, $P'_n(x) = nx^{n-1} + 1 > 1$ pour tout $x \in (0, 1)$. Puisque $P'_n(x) > 0$ sur $(0, 1)$, la fonction $P_n(x)$ est strictement croissante sur cet intervalle. Une fonction strictement monotone ne peut couper l'axe des abscisses qu'en un seul point. Ainsi, l'équation $P_n(x) = 0$ admet une unique solution u_n dans $(0, 1)$.

2. Monotonie de la suite $(u_n)_{n \geq 1}$

Nous avons $P_n(u_n) = u_n^n + u_n - 1 = 0$, ce qui implique $u_n^n = 1 - u_n$. Pour étudier la monotonie de la suite (u_n) , comparons u_n et u_{n+1} . Nous savons que u_{n+1} est la racine de $P_{n+1}(x) = x^{n+1} + x - 1 = 0$. Évaluons $P_{n+1}(u_n)$: $P_{n+1}(u_n) = u_n^{n+1} + u_n - 1$. En utilisant la relation $u_n^n = 1 - u_n$ (obtenue de $P_n(u_n) = 0$), nous pouvons réécrire u_n^{n+1} comme $u_n \cdot u_n^n = u_n(1 - u_n)$. Donc, $P_{n+1}(u_n) = u_n(1 - u_n) + u_n - 1$. Développons et simplifions cette expression : $P_{n+1}(u_n) = u_n - u_n^2 + u_n - 1 = -u_n^2 + 2u_n - 1$. Nous reconnaissons une forme quadratique : $-(u_n^2 - 2u_n + 1) = -(u_n - 1)^2$. Puisque $u_n \in (0, 1)$, $u_n \neq 1$, donc $(u_n - 1)^2 > 0$. Par conséquent, $P_{n+1}(u_n) = -(u_n - 1)^2 < 0$.

Nous avons $P_{n+1}(u_n) < 0$ et nous savons que $P_{n+1}(u_{n+1}) = 0$. Puisque $P_{n+1}(x)$ est strictement croissante sur $(0, 1)$ (d'après la partie 1, car $P'_{n+1}(x) = (n+1)x^n + 1 > 0$), l'inégalité $P_{n+1}(u_n) < P_{n+1}(u_{n+1})$ implique $u_n < u_{n+1}$. La suite $(u_n)_{n \geq 1}$ est donc strictement croissante.

De plus, nous avons montré en partie 1 que $u_n \in (0, 1)$ pour tout n . La suite (u_n) est donc majorée par 1. Toute suite réelle croissante et majorée est convergente. Par conséquent, la suite $(u_n)_{n \geq 1}$ converge vers une limite L .

3. Détermination de la limite L

Puisque $u_n \in (0, 1)$ pour tout n , et que la suite est croissante, sa limite L doit satisfaire $u_1 \leq L \leq 1$. (Pour $n = 1$, $P_1(x) = x + x - 1 = 2x - 1 = 0 \implies u_1 = 1/2$. Donc $1/2 \leq L \leq 1$.)

Nous avons la relation $u_n^n = 1 - u_n$. Passons à la limite lorsque $n \rightarrow \infty$: $\lim_{n \rightarrow \infty} u_n^n = \lim_{n \rightarrow \infty} (1 - u_n)$. Le membre de droite converge vers $1 - L$.

Analysons le membre de gauche, $\lim_{n \rightarrow \infty} u_n^n$. * **Cas 1 : Si $L < 1$.** Si $L < 1$, alors pour n suffisamment grand, u_n est proche de L et donc $u_n < 1$. Dans ce cas, $\lim_{n \rightarrow \infty} u_n^n = 0$. L'équation à la limite devient $0 = 1 - L$, ce qui implique $L = 1$. Ceci contredit notre hypothèse $L < 1$. Donc le cas $L < 1$ est impossible.

* **Cas 2 : Si $L = 1$.** Si $L = 1$, alors le membre de droite $1 - L = 1 - 1 = 0$. Le membre de gauche $\lim_{n \rightarrow \infty} u_n^n = \lim_{n \rightarrow \infty} 1^n = 1$. Ceci conduit à $1 = 0$, ce qui est une contradiction.

Il y a une erreur dans le raisonnement du Cas 2. Si $L = 1$, alors $u_n \rightarrow 1$. Le terme u_n^n ne tend pas nécessairement vers 1. Par exemple, $(1 - 1/n)^n \rightarrow 1/e$. Reprenons l'analyse de $\lim_{n \rightarrow \infty} u_n^n$ lorsque $L = 1$. Si $L = 1$, alors $u_n \rightarrow 1$. L'équation $u_n^n = 1 - u_n$ devient, à la limite, $\lim_{n \rightarrow \infty} u_n^n = 1 - 1 = 0$. Pour que $\lim_{n \rightarrow \infty} u_n^n = 0$ alors que $u_n \rightarrow 1$, il faut que u_n s'approche de 1 "suffisamment lentement". Ceci n'est pas une contradiction. En fait, c'est ce que nous allons prouver.

Le raisonnement correct pour la limite est le suivant : Nous savons que $L \in [1/2, 1]$. De $u_n^n = 1 - u_n$, si $L < 1$, alors $u_n \rightarrow L < 1$. Pour tout $x \in [0, 1)$, $\lim_{n \rightarrow \infty} x^n = 0$. Donc, si $L < 1$,

alors $\lim_{n \rightarrow \infty} u_n^n = 0$. L'équation à la limite devient $0 = 1 - L$, ce qui implique $L = 1$. Ceci contredit l'hypothèse $L < 1$. Par conséquent, la seule possibilité est que $L = 1$.

4. Démonstration rigoureuse de la limite par ϵ, N

Nous voulons démontrer que $\lim_{n \rightarrow \infty} u_n = 1$ en utilisant la définition ϵ, N . Cela signifie que pour tout $\epsilon > 0$, il existe un entier N tel que pour tout $n \geq N$, nous ayons $|u_n - 1| < \epsilon$.

Puisque nous savons que $u_n \in (0, 1)$ pour tout n , l'inégalité $|u_n - 1| < \epsilon$ est équivalente à $1 - u_n < \epsilon$. Ceci est à son tour équivalent à $u_n > 1 - \epsilon$.

Soit $\epsilon > 0$ donné. Nous pouvons supposer sans perte de généralité que $\epsilon \in (0, 1)$, car si $\epsilon \geq 1$, alors $1 - u_n < 1 \leq \epsilon$ est toujours vraie puisque $u_n \in (0, 1)$. Nous cherchons donc à trouver N tel que pour tout $n \geq N$, $u_n > 1 - \epsilon$.

Rappelons que u_n est la racine unique de $P_n(x) = x^n + x - 1 = 0$. Puisque $P_n(x)$ est strictement croissante sur $(0, 1)$ (démontré en partie 1), l'inégalité $u_n > 1 - \epsilon$ est équivalente à $P_n(u_n) > P_n(1 - \epsilon)$. Comme $P_n(u_n) = 0$, nous devons montrer que $0 > P_n(1 - \epsilon)$, c'est-à-dire $P_n(1 - \epsilon) < 0$.

Calculons $P_n(1 - \epsilon)$: $P_n(1 - \epsilon) = (1 - \epsilon)^n + (1 - \epsilon) - 1 = (1 - \epsilon)^n - \epsilon$.

Nous devons donc montrer que pour n suffisamment grand, $(1 - \epsilon)^n - \epsilon < 0$, ce qui est équivalent à $(1 - \epsilon)^n < \epsilon$.

Puisque nous avons supposé $\epsilon \in (0, 1)$, nous avons $0 < 1 - \epsilon < 1$. Nous savons que pour tout $a \in (0, 1)$, la suite géométrique $(a^n)_{n \geq 1}$ converge vers 0. Donc, $\lim_{n \rightarrow \infty} (1 - \epsilon)^n = 0$.

Par la définition de la limite d'une suite, pour tout $\delta' > 0$, il existe un entier N tel que pour tout $n \geq N$, nous ayons $|(1 - \epsilon)^n - 0| < \delta'$. Choisissons $\delta' = \epsilon$. Alors, il existe un entier N (qui dépend de ϵ) tel que pour tout $n \geq N$, $(1 - \epsilon)^n < \epsilon$.

Pour ce N et pour tout $n \geq N$:

1. Nous avons $(1 - \epsilon)^n < \epsilon$.
2. Ceci implique $P_n(1 - \epsilon) = (1 - \epsilon)^n - \epsilon < 0$.
3. Puisque $P_n(u_n) = 0$ et $P_n(x)$ est strictement croissante, l'inégalité $P_n(1 - \epsilon) < P_n(u_n)$ implique $1 - \epsilon < u_n$.
4. Combiné avec $u_n < 1$ (démontré en partie 1), nous obtenons $1 - \epsilon < u_n < 1$.
5. Ceci signifie $0 < 1 - u_n < \epsilon$, ce qui est équivalent à $|u_n - 1| < \epsilon$.

Nous avons donc trouvé, pour tout $\epsilon > 0$, un N tel que pour tout $n \geq N$, $|u_n - 1| < \epsilon$. Ceci démontre rigoureusement, par la définition ϵ, N , que $\lim_{n \rightarrow \infty} u_n = 1$.

Exercice 9 : Convergence d'une suite produit par le critère de Cauchy

Énoncé

Soit $(u_n)_{n \in \mathbb{N}^*}$ la suite de nombres réels définie pour tout $n \in \mathbb{N}^*$ par le produit :

$$u_n = \prod_{k=1}^n \left(1 + \frac{1}{k^2}\right)$$

1. Montrer rigoureusement que pour tout $n \in \mathbb{N}^*$, $u_n > 0$.
2. On considère la suite $(v_n)_{n \in \mathbb{N}^*}$ définie par $v_n = \ln(u_n)$. Démontrer, en utilisant la définition rigoureuse (ϵ, N) , que la suite (v_n) est une suite de Cauchy.
3. En déduire, en justifiant chaque étape et en invoquant les propriétés fondamentales de l'analyse réelle, que la suite (u_n) converge vers une limite finie.

Correction Détaillée

Question 1 : Montrer que pour tout $n \in \mathbb{N}^*$, $u_n > 0$.

Raisonnement : La suite (u_n) est définie comme un produit de termes. Pour montrer que u_n est strictement positif, il suffit de montrer que chaque terme du produit est strictement positif.

Démonstration :

1. Soit $k \in \mathbb{N}^*$. Le terme général du produit est $(1 + \frac{1}{k^2})$.
2. Pour tout $k \in \mathbb{N}^*$, $k \geq 1$. Par conséquent, $k^2 \geq 1$.
3. Il s'ensuit que $\frac{1}{k^2} > 0$.
4. En ajoutant 1 à cette inégalité, nous obtenons $1 + \frac{1}{k^2} > 1 + 0$, c'est-à-dire $1 + \frac{1}{k^2} > 1$.
5. Puisque $1 + \frac{1}{k^2} > 1$, chaque terme du produit est strictement positif.
6. La suite u_n est le produit de n termes strictement positifs : $u_n = (1 + \frac{1}{1^2}) \times (1 + \frac{1}{2^2}) \times \dots \times (1 + \frac{1}{n^2})$.
7. Un produit de nombres strictement positifs est lui-même strictement positif.
8. Par conséquent, pour tout $n \in \mathbb{N}^*$, $u_n > 0$.

Question 2 : Démontrer, en utilisant la définition rigoureuse (ϵ, N) , que la suite (v_n) est une suite de Cauchy.

Raisonnement : La suite (v_n) est définie par $v_n = \ln(u_n)$. En utilisant les propriétés du logarithme, nous pouvons transformer le produit en une somme, ce qui est souvent plus facile à manipuler pour le critère de Cauchy. Ensuite, nous appliquerons la définition de Cauchy pour les suites, en utilisant une inégalité appropriée pour majorer la différence $|v_m - v_n|$.

Démonstration :

1. Expression de v_n en tant que somme :

Nous avons $v_n = \ln(u_n) = \ln(\prod_{k=1}^n (1 + \frac{1}{k^2}))$. En utilisant la propriété du logarithme $\ln(a \cdot b) = \ln(a) + \ln(b)$ (qui s'étend à un produit fini), nous obtenons : $v_n = \sum_{k=1}^n \ln(1 + \frac{1}{k^2})$.

1. Définition d'une suite de Cauchy :

Une suite $(v_n)_{n \in \mathbb{N}^*}$ est dite de Cauchy si pour tout $\epsilon > 0$, il existe un entier $N \in \mathbb{N}^*$ tel que pour tous entiers m, n vérifiant $m > n \geq N$, on a $|v_m - v_n| < \epsilon$.

1. Calcul de $|v_m - v_n|$ pour $m > n$:

Soient $m, n \in \mathbb{N}^*$ tels que $m > n$. $v_m - v_n = \sum_{k=1}^m \ln(1 + \frac{1}{k^2}) - \sum_{k=1}^n \ln(1 + \frac{1}{k^2})$. Les termes de $k = 1$ à n s'annulent, il reste donc : $v_m - v_n = \sum_{k=n+1}^m \ln(1 + \frac{1}{k^2})$.

1. Utilisation de la positivité des termes et d'une inégalité :

Pour tout $k \in \mathbb{N}^*$, nous avons $1 + \frac{1}{k^2} > 1$ (d'après la question 1). Puisque la fonction logarithme népérien est strictement croissante et $\ln(1) = 0$, il s'ensuit que $\ln(1 + \frac{1}{k^2}) > 0$. Par conséquent, la somme $\sum_{k=n+1}^m \ln(1 + \frac{1}{k^2})$ est une somme de termes positifs, donc elle est positive. Ainsi, $|v_m - v_n| = \sum_{k=n+1}^m \ln(1 + \frac{1}{k^2})$.

Nous utilisons maintenant l'inégalité fondamentale $\ln(1+x) \leq x$ pour tout $x \geq 0$. Ici, $x = \frac{1}{k^2}$. Puisque $k \in \mathbb{N}^*$, $k^2 > 0$, donc $\frac{1}{k^2} > 0$. Appliquons l'inégalité : $\ln(1 + \frac{1}{k^2}) \leq \frac{1}{k^2}$.

En sommant cette inégalité pour k allant de $n+1$ à m : $|v_m - v_n| \leq \sum_{k=n+1}^m \frac{1}{k^2}$.

1. Lien avec la convergence d'une série de Riemann :

La série $\sum_{k=1}^{\infty} \frac{1}{k^2}$ est une série de Riemann de la forme $\sum_{k=1}^{\infty} \frac{1}{k^p}$ avec $p = 2$. Puisque $p = 2 > 1$, cette série est convergente. Une propriété fondamentale des séries convergentes est que la suite de ses sommes partielles est une suite de Cauchy. Plus précisément, pour une série $\sum a_k$ convergente, pour tout $\epsilon > 0$, il existe un entier $N \in \mathbb{N}^*$ tel que pour tous $m > n \geq N$, $|\sum_{k=n+1}^m a_k| < \epsilon$. Appliquons ceci à la série $\sum_{k=1}^{\infty} \frac{1}{k^2}$. Pour tout $\epsilon > 0$, il existe un entier $N \in \mathbb{N}^*$ tel que pour tous $m > n \geq N$, on a $\sum_{k=n+1}^m \frac{1}{k^2} < \epsilon$.

1. Conclusion pour (v_n) :

En combinant les résultats des étapes précédentes : Pour tout $\epsilon > 0$, nous avons trouvé un entier $N \in \mathbb{N}^*$ tel que pour tous $m > n \geq N$, $|v_m - v_n| \leq \sum_{k=n+1}^m \frac{1}{k^2} < \epsilon$. Ceci correspond exactement à la définition d'une suite de Cauchy. Par conséquent, la suite (v_n) est une suite de Cauchy.

Question 3 : En déduire, en justifiant rigoureusement, que la suite (u_n) converge vers une limite finie.

Raisonnement : Nous avons montré que (v_n) est une suite de Cauchy de nombres réels. L'espace $\mathbb{R}$ est complet, ce qui signifie que toute suite de Cauchy de nombres réels converge vers une limite réelle. Ensuite, nous utiliserons la relation $u_n = e^{v_n}$ et la continuité de la fonction exponentielle pour déduire la convergence de (u_n) .

Démonstration :

1. Convergence de (v_n) :

D'après la question 2, la suite (v_n) est une suite de Cauchy. Les termes de la suite (v_n) sont des nombres réels. Le théorème fondamental de l'analyse réelle stipule que l'espace des nombres réels $\mathbb{R}$ est complet. Cela signifie que toute suite de Cauchy de nombres réels converge vers une limite finie dans $\mathbb{R}$. Par conséquent, la suite (v_n) converge vers une limite $L \in \mathbb{R}$. Nous pouvons écrire $\lim_{n \rightarrow \infty} v_n = L$.

1. Relation entre (u_n) et (v_n) :

Par définition, $v_n = \ln(u_n)$. En appliquant la fonction exponentielle aux deux membres de cette égalité, nous obtenons $e^{v_n} = e^{\ln(u_n)}$. Puisque $u_n > 0$ (d'après la question 1), la fonction exponentielle et la fonction logarithme népérien sont des fonctions réciproques l'une de l'autre, donc $e^{\ln(u_n)} = u_n$. Ainsi, nous avons $u_n = e^{v_n}$.

1. Convergence de (u_n) par continuité :

Nous savons que la suite (v_n) converge vers L . La fonction exponentielle, $f(x) = e^x$, est une fonction continue sur tout $\mathbb{R}$. Une propriété essentielle des fonctions continues est que si une suite (x_n) converge vers une limite x , alors la suite $(f(x_n))$ converge vers $f(x)$. Appliquons cette propriété avec $x_n = v_n$ et $f(x) = e^x$. Puisque $\lim_{n \rightarrow \infty} v_n = L$ et que $f(x) = e^x$ est continue en L , nous pouvons affirmer que : $\lim_{n \rightarrow \infty} u_n = \lim_{n \rightarrow \infty} e^{v_n} = e^{\lim_{n \rightarrow \infty} v_n} = e^L$.

1. Conclusion :

La suite (u_n) converge vers la limite finie e^L . La convergence est ainsi rigoureusement établie.

Exercice 10 : Convergence Quadratique et Critère de Cauchy pour une Suite Complexe

Énoncé

Soit la suite de nombres complexes $(z_n)_{n \in \mathbb{N}}$ définie par $z_0 \in \mathbb{C}$ et la relation de récurrence :

$$z_{n+1} = \frac{1}{2}z_n + \frac{1}{z_n^2 + 1}$$

On suppose que le terme initial z_0 est choisi tel que $|z_0 - 1| \leq \frac{1}{2}$.

1. Montrer que pour tout $n \in \mathbb{N}$, $z_n \neq -i$ et $z_n \neq i$.
2. Démontrer que si $|z_n - 1| \leq \frac{1}{2}$, alors $|z_{n+1} - 1| \leq |z_n - 1|^2$. En déduire que pour tout $n \in \mathbb{N}$, $|z_n - 1| \leq \left(\frac{1}{2}\right)^{2^n}$.
3. Utiliser le critère de Cauchy pour prouver que la suite $(z_n)_{n \in \mathbb{N}}$ converge.
4. Déterminer la limite de la suite $(z_n)_{n \in \mathbb{N}}$.

Correction Détaillée

Question 1 : Montrer que pour tout $n \in \mathbb{N}$, $z_n \neq -i$ et $z_n \neq i$.

Pour que la suite (z_n) soit bien définie, il est impératif que le dénominateur $z_n^2 + 1$ ne s'annule jamais. Les valeurs de z pour lesquelles $z^2 + 1 = 0$ sont $z = i$ et $z = -i$. Nous devons donc démontrer que $z_n \neq i$ et $z_n \neq -i$ pour tout $n \in \mathbb{N}$.

L'hypothèse initiale est que $|z_0 - 1| \leq \frac{1}{2}$. Cela signifie que z_0 appartient au disque fermé de centre 1 et de rayon $\frac{1}{2}$, noté $D(1, \frac{1}{2})$.

Calculons la distance du point i au centre du disque 1 : $|i - 1| = \sqrt{(\operatorname{Re}(i - 1))^2 + (\operatorname{Im}(i - 1))^2} = \sqrt{(-1)^2 + (1)^2} = \sqrt{1 + 1} = \sqrt{2}$. De même, pour le point $-i$: $|-i - 1| = \sqrt{(\operatorname{Re}(-i - 1))^2 + (\operatorname{Im}(-i - 1))^2} = \sqrt{(-1)^2 + (-1)^2} = \sqrt{1 + 1} = \sqrt{2}$.

Puisque $\sqrt{2} \approx 1.414$, et que le rayon du disque est $\frac{1}{2} = 0.5$, nous avons $\sqrt{2} > \frac{1}{2}$. Cela implique que les points i et $-i$ ne sont pas contenus dans le disque $D(1, \frac{1}{2})$. Par conséquent, $z_0 \neq i$ et $z_0 \neq -i$.

Pour les termes suivants de la suite, nous allons utiliser le résultat de la question 2 (que nous démontrerons rigoureusement par la suite). Ce résultat établit que si $|z_n - 1| \leq \frac{1}{2}$, alors $|z_{n+1} - 1| \leq |z_n - 1|^2$. Par une application répétée de cette inégalité, en partant de $|z_0 - 1| \leq \frac{1}{2}$, nous obtenons par récurrence que $|z_n - 1| \leq (\frac{1}{2})^{2^n}$ pour tout $n \in \mathbb{N}$. Puisque $2^n \geq 1$ pour tout $n \in \mathbb{N}$, nous avons $(\frac{1}{2})^{2^n} \leq \frac{1}{2}$. Cela signifie que pour tout $n \in \mathbb{N}$, $|z_n - 1| \leq \frac{1}{2}$. Ainsi, tous les termes z_n de la suite restent confinés dans le disque $D(1, \frac{1}{2})$. Comme nous l'avons montré, les points i et $-i$ ne sont pas dans ce disque. Par conséquent, pour tout $n \in \mathbb{N}$, $z_n \neq i$ et $z_n \neq -i$. La suite est donc bien définie pour tout $n \in \mathbb{N}$.

Question 2 : Démontrer que si $|z_n - 1| \leq \frac{1}{2}$, alors $|z_{n+1} - 1| \leq |z_n - 1|^2$. En déduire que pour tout $n \in \mathbb{N}$, $|z_n - 1| \leq (\frac{1}{2})^{2^n}$.

Soit z_n un terme de la suite tel que $|z_n - 1| \leq \frac{1}{2}$. Nous souhaitons évaluer l'expression $z_{n+1} - 1$. Nous partons de la relation de récurrence :

$$z_{n+1} = \frac{1}{2}z_n + \frac{1}{z_n^2 + 1}$$

Soustrayons 1 des deux côtés :

$$z_{n+1} - 1 = \left(\frac{1}{2}z_n + \frac{1}{z_n^2 + 1} \right) - 1$$

Pour simplifier, mettons les termes sur un dénominateur commun $2(z_n^2 + 1)$:

$$z_{n+1} - 1 = \frac{z_n(z_n^2 + 1) + 2 - 2(z_n^2 + 1)}{2(z_n^2 + 1)}$$

Développons le numérateur :

$$z_{n+1} - 1 = \frac{z_n^3 + z_n + 2 - 2z_n^2 - 2}{2(z_n^2 + 1)}$$

Regroupons les termes du numérateur :

$$z_{n+1} - 1 = \frac{z_n^3 - 2z_n^2 + z_n}{2(z_n^2 + 1)}$$

Nous pouvons factoriser le numérateur par z_n :

$$z_{n+1} - 1 = \frac{z_n(z_n^2 - 2z_n + 1)}{2(z_n^2 + 1)}$$

Le terme entre parenthèses au numérateur est un carré parfait : $(z_n - 1)^2$. Ainsi, nous obtenons l'expression fondamentale :

$$z_{n+1} - 1 = \frac{z_n(z_n - 1)^2}{2(z_n^2 + 1)}$$

Maintenant, prenons le module de cette expression. En utilisant la propriété $|AB/C| = |A||B|/|C|$:

$$|z_{n+1} - 1| = \frac{|z_n||z_n - 1|^2}{2|z_n^2 + 1|}$$

Nous devons maintenant trouver des bornes pour $|z_n|$ et $|z_n^2 + 1|$ en utilisant l'hypothèse $|z_n - 1| \leq \frac{1}{2}$.

1. Borne supérieure pour $|z_n|$:

En utilisant l'inégalité triangulaire $|A + B| \leq |A| + |B|$: $|z_n| = |(z_n - 1) + 1| \leq |z_n - 1| + |1|$. Puisque $|z_n - 1| \leq \frac{1}{2}$, nous avons : $|z_n| \leq \frac{1}{2} + 1 = \frac{3}{2}$.

1. Borne inférieure pour $|z_n^2 + 1|$:

Nous utilisons l'inégalité triangulaire inversée $|A + B| \geq ||A| - |B||$. $|z_n^2 + 1| = |(z_n - 1 + 1)^2 + 1| = |(z_n - 1)^2 + 2(z_n - 1) + 1 + 1| = |(z_n - 1)^2 + 2(z_n - 1) + 2|$. Soit $h = z_n - 1$. Par hypothèse, $|h| \leq \frac{1}{2}$. Alors $|z_n^2 + 1| = |h^2 + 2h + 2|$. Appliquons l'inégalité triangulaire inversée : $|h^2 + 2h + 2| \geq |2 - |2h| - |h^2||$. Puisque $|h| \leq \frac{1}{2}$, nous avons : $|2h| \leq 2 \cdot \frac{1}{2} = 1$. $|h^2| = |h|^2 \leq (\frac{1}{2})^2 = \frac{1}{4}$. Donc : $|z_n^2 + 1| \geq 2 - 1 - \frac{1}{4} = 1 - \frac{1}{4} = \frac{3}{4}$.

Maintenant, substituons ces bornes dans l'expression de $|z_{n+1} - 1|$:

$$|z_{n+1} - 1| \leq \frac{\frac{3}{2}|z_n - 1|^2}{2 \cdot \frac{3}{4}}$$

$$|z_{n+1} - 1| \leq \frac{\frac{3}{2}|z_n - 1|^2}{\frac{3}{2}}$$

$$|z_{n+1} - 1| \leq |z_n - 1|^2$$

Ceci démontre la première partie de la question.

Passons à la déduction par récurrence de l'inégalité $|z_n - 1| \leq (\frac{1}{2})^{2^n}$ pour tout $n \in \mathbb{N}$.
Initialisation (n=0) : Par hypothèse de l'énoncé, $|z_0 - 1| \leq \frac{1}{2}$. L'inégalité à prouver pour $n = 0$ est $|z_0 - 1| \leq (\frac{1}{2})^{2^0} = (\frac{1}{2})^1 = \frac{1}{2}$. L'initialisation est donc vérifiée.

Hypothèse de récurrence : Supposons que pour un certain entier $n \geq 0$, l'inégalité $|z_n - 1| \leq (\frac{1}{2})^{2^n}$ est vraie.

Étape de récurrence : Nous devons montrer que l'inégalité est également vraie pour $n + 1$, c'est-à-dire $|z_{n+1} - 1| \leq (\frac{1}{2})^{2^{n+1}}$. D'après l'hypothèse de récurrence, $|z_n - 1| \leq (\frac{1}{2})^{2^n}$. Puisque $2^n \geq 1$ pour tout $n \geq 0$, nous avons $(\frac{1}{2})^{2^n} \leq \frac{1}{2}$. L'hypothèse $|z_n - 1| \leq \frac{1}{2}$ est donc satisfaite. Nous pouvons appliquer le résultat que nous venons de démontrer :

$$|z_{n+1} - 1| \leq |z_n - 1|^2$$

En utilisant l'hypothèse de récurrence dans cette inégalité :

$$|z_{n+1} - 1| \leq \left(\left(\frac{1}{2} \right)^{2^n} \right)^2$$

$$|z_{n+1} - 1| \leq \left(\frac{1}{2} \right)^{2^n \cdot 2}$$

$$|z_{n+1} - 1| \leq \left(\frac{1}{2} \right)^{2^{n+1}}$$

L'inégalité est donc vérifiée pour $n + 1$. Par le principe d'induction mathématique, l'inégalité $|z_n - 1| \leq (\frac{1}{2})^{2^n}$ est vraie pour tout $n \in \mathbb{N}$.

Question 3 : Utiliser le critère de Cauchy pour prouver que la suite $(z_n)_{n \in \mathbb{N}}$ converge.

Le critère de Cauchy pour une suite de nombres complexes (z_n) stipule que la suite converge si et seulement si elle est une suite de Cauchy. Une suite (z_n) est de Cauchy si pour tout $\epsilon > 0$, il existe un entier $N \in \mathbb{N}$ tel que pour tous entiers p, q vérifiant $p > N$ et $q > N$, on a $|z_p - z_q| < \epsilon$.

Nous avons établi à la question précédente que pour tout $n \in \mathbb{N}$, $|z_n - 1| \leq \left(\frac{1}{2}\right)^{2^n}$. Soit $\epsilon > 0$ un nombre réel strictement positif. Nous devons trouver un entier N approprié. Considérons deux indices p et q tels que $p > N$ et $q > N$. En utilisant l'inégalité triangulaire pour les nombres complexes :

$$|z_p - z_q| = |(z_p - 1) - (z_q - 1)| \leq |z_p - 1| + |z_q - 1|$$

En appliquant la borne que nous avons démontrée :

$$|z_p - z_q| \leq \left(\frac{1}{2}\right)^{2^p} + \left(\frac{1}{2}\right)^{2^q}$$

Puisque $p > N$ et $q > N$, et que la fonction $x \mapsto 2^x$ est strictement croissante, nous avons $2^p > 2^N$ et $2^q > 2^N$. De plus, la fonction $y \mapsto \left(\frac{1}{2}\right)^y$ est strictement décroissante. Donc : $\left(\frac{1}{2}\right)^{2^p} < \left(\frac{1}{2}\right)^{2^N}$ et $\left(\frac{1}{2}\right)^{2^q} < \left(\frac{1}{2}\right)^{2^N}$. En substituant ces inégalités :

$$|z_p - z_q| < \left(\frac{1}{2}\right)^{2^N} + \left(\frac{1}{2}\right)^{2^N} = 2 \cdot \left(\frac{1}{2}\right)^{2^N}$$

Nous voulons que cette quantité soit inférieure à ϵ . Il suffit donc de trouver N tel que $2 \cdot \left(\frac{1}{2}\right)^{2^N} < \epsilon$. Ceci est équivalent à $\left(\frac{1}{2}\right)^{2^N} < \frac{\epsilon}{2}$.

Pour trouver N , nous prenons le logarithme (par exemple en base 2) des deux côtés. Si $\epsilon \geq 2$, alors $\frac{\epsilon}{2} \geq 1$. Dans ce cas, $\left(\frac{1}{2}\right)^{2^N} < 1$ est toujours vrai pour $N \geq 0$ (car $2^N \geq 1$), donc n'importe quel $N \geq 0$ convient. Nous pouvons choisir $N = 0$. Si $0 < \epsilon < 2$, alors $0 < \frac{\epsilon}{2} < 1$. $\log_2 \left(\left(\frac{1}{2}\right)^{2^N} \right) < \log_2 \left(\frac{\epsilon}{2} \right)$. $2^N \log_2 \left(\frac{1}{2} \right) < \log_2 \left(\frac{\epsilon}{2} \right)$. $-2^N < \log_2 \left(\frac{\epsilon}{2} \right)$. Multiplions par -1 et inversons l'inégalité : $2^N > -\log_2 \left(\frac{\epsilon}{2} \right)$. Prenons à nouveau le logarithme en base 2 : $N > \log_2 \left(\log_2 \left(\frac{2}{\epsilon} \right) \right)$. Puisque $\epsilon < 2$, $\frac{2}{\epsilon} > 1$, donc $\log_2 \left(\frac{2}{\epsilon} \right) > 0$. Le logarithme est bien défini. Nous pouvons choisir N comme le plus petit entier supérieur à $\log_2 \left(\log_2 \left(\frac{2}{\epsilon} \right) \right)$. Par exemple, $N = \max(0, \lceil \log_2 \left(\log_2 \left(\frac{2}{\epsilon} \right) \right) \rceil)$.

Pour un tel N (choisi en fonction de ϵ), si $p > N$ et $q > N$, alors $|z_p - z_q| < \epsilon$. Par conséquent, la suite (z_n) est une suite de Cauchy. Puisque l'ensemble des nombres complexes $\mathbb{C}$ est un espace complet (toute suite de Cauchy y converge), nous pouvons conclure que la suite $(z_n)_{n \in \mathbb{N}}$ converge.

Question 4 : Déterminer la limite de la suite $(z_n)_{n \in \mathbb{N}}$.

Nous avons prouvé à la question 3 que la suite (z_n) converge. Soit L sa limite. La fonction $f(z) = \frac{1}{2}z + \frac{1}{z^2+1}$ est une fonction continue sur son domaine de définition (où $z^2 + 1 \neq 0$). Nous avons montré à la question 1 que $z_n^2 + 1 \neq 0$ pour tout n , et donc $L^2 + 1 \neq 0$ (car si $L^2 + 1 = 0$, alors $L = i$ ou $L = -i$, mais la suite reste dans $D(1, 1/2)$ et $i, -i$ n'y sont pas). Par conséquent, nous pouvons passer à la limite dans la relation de récurrence $z_{n+1} = \frac{1}{2}z_n + \frac{1}{z_n^2+1}$:

$$\lim_{n \rightarrow \infty} z_{n+1} = \frac{1}{2} \lim_{n \rightarrow \infty} z_n + \frac{1}{(\lim_{n \rightarrow \infty} z_n)^2 + 1}$$

$$L = \frac{1}{2}L + \frac{1}{L^2 + 1}$$

Maintenant, nous résolvons cette équation pour trouver L :

$$L - \frac{1}{2}L = \frac{1}{L^2 + 1}$$

$$\frac{1}{2}L = \frac{1}{L^2 + 1}$$

Multiplions par $2(L^2 + 1)$ (qui est non nul) :

$$L(L^2 + 1) = 2$$

$$L^3 + L = 2$$

$$L^3 + L - 2 = 0$$

Nous cherchons les racines de ce polynôme de degré 3. Nous pouvons tester des valeurs entières simples. Pour $L = 1$: $1^3 + 1 - 2 = 1 + 1 - 2 = 0$. Donc $L = 1$ est une racine. Puisque $L = 1$ est une racine, $(L - 1)$ est un facteur du polynôme $L^3 + L - 2$. Nous pouvons effectuer une division polynomiale ou une factorisation : $L^3 + L - 2 = (L - 1)(L^2 + L + 2)$. Les autres racines sont données par l'équation quadratique $L^2 + L + 2 = 0$. Calculons le discriminant Δ de cette équation : $\Delta = b^2 - 4ac = 1^2 - 4(1)(2) = 1 - 8 = -7$. Puisque $\Delta < 0$, les deux autres racines sont complexes et non réelles : $L = \frac{-1 \pm \sqrt{-7}}{2} = \frac{-1 \pm i\sqrt{7}}{2}$. Les trois solutions possibles pour la limite L sont donc : $L_1 = 1$ $L_2 = -\frac{1}{2} + i\frac{\sqrt{7}}{2}$ $L_3 = -\frac{1}{2} - i\frac{\sqrt{7}}{2}$

Pour déterminer laquelle de ces valeurs est la limite de notre suite, nous utilisons le résultat de la question 2 : Pour tout $n \in \mathbb{N}$, nous avons $|z_n - 1| \leq \left(\frac{1}{2}\right)^{2^n}$. Lorsque $n \rightarrow \infty$, $2^n \rightarrow \infty$, et donc $\left(\frac{1}{2}\right)^{2^n} \rightarrow 0$. Par le théorème des gendarmes (ou par la définition même de la limite), nous avons :

$$\lim_{n \rightarrow \infty} |z_n - 1| = 0$$

Ceci signifie que la distance entre z_n et 1 tend vers zéro lorsque n tend vers l'infini. Par définition de la limite d'une suite, cela implique que $\lim_{n \rightarrow \infty} z_n = 1$.

Par conséquent, la limite de la suite $(z_n)_{n \in \mathbb{N}}$ est $L = 1$.

Chapitre 5

Travaux pratiques et simulations algorithmiques

TP 1 : Suites Réelles et Complexes - Définition Rigoureuse des Limites (ϵ, N) et Convergence en Python Pur

Objectif

Ce TP vise à approfondir la compréhension des définitions rigoureuses des limites de suites réelles et complexes, en particulier la définition (ϵ, N) , et des critères de convergence. L'objectif est de traduire ces concepts mathématiques fondamentaux en une implémentation Python "from scratch", sans l'aide de bibliothèques de calcul numérique avancées, afin de renforcer les compétences en programmation pure et en validation mathématique par des assertions.

À l'issue de ce TP, vous devriez être capable de :

1. Comprendre et expliquer la définition (ϵ, N) de la limite d'une suite.
2. Implémenter des fonctions Python pour évaluer des suites réelles et complexes.
3. Développer une fonction Python pour vérifier empiriquement la convergence d'une suite vers une limite donnée, en utilisant la définition (ϵ, N) .
4. Utiliser des assertions pour valider la correction de l'implémentation par rapport aux propriétés mathématiques connues de diverses suites.

Implémentation Python pure

Nous allons implémenter les fonctions nécessaires pour manipuler des suites et vérifier leur convergence. Les nombres complexes seront gérés nativement par le type 'complex' de Python.

```
import math

def absolute_value(z: complex | float) -> float:
    """
    Calcule la valeur absolue (ou module pour les complexes) d'un nombre
    .
    Utilise la fonction abs() native de Python qui gere float et complex
    .
    """
    return abs(z)

def distance(z1: complex | float, z2: complex | float) -> float:
    """
    Calcule la distance entre deux nombres (reels ou complexes).
    La distance est definie comme la valeur absolue de leur difference.
```



```

"""
    return absolute_value(z1 - z2)

def is_convergent_epsilon_N(
    sequence_func: callable[[int], complex | float],
    limit: complex | float,
    epsilon: float,
    max_N_to_test: int = 1000,
    terms_after_N_to_check: int = 50
) -> bool:
    """
    Verifie empiriquement si une suite converge vers une limite donnee
    selon la definition epsilon-N.

    Une suite (a_n) converge vers L si, pour tout epsilon > 0, il existe
    un entier N tel que pour tout n > N, la distance |a_n - L| < epsilon
    .

    Etant donne l'infinitude des suites, cette fonction effectue une
    verification
    empirique sur une plage finie de N et de termes subsequents. Elle ne
    peut
    pas *prouver* formellement la convergence, mais peut fournir une
    forte
    preuve ou la refuter dans la plage testee.

    Args:
        sequence_func: Une fonction qui prend un entier n (l'indice du
            terme)
                        et retourne le n-ieme terme de la suite (float ou
                        complex).
                        On suppose que n >= 1 pour les suites classiques
                        (1/n, etc.).
        limit: La limite candidate (float ou complex).
        epsilon: La valeur epsilon (doit etre strictement positive).
        max_N_to_test: L'entier maximal N a tester. Si aucun N n'est
            trouve
                        jusqu'a cette valeur, la fonction retourne False.
        terms_after_N_to_check: Le nombre de termes a verifier apres N.
                        Plus cette valeur est grande, plus la
                        preuve
                        empirique est forte, mais plus le calcul
                        est long.

    Returns:
        True si un N est trouve tel que les 'terms_after_N_to_check'
            termes
            suivants sont tous a une distance < epsilon de la limite.
        False sinon (aucun N trouve dans la plage testee).
    """
    if epsilon <= 0:
        raise ValueError("Epsilon doit etre strictement positif.")

    # Iterer sur les valeurs possibles de N (l'indice apres lequel les
    # termes doivent etre proches de L)
    # N peut etre 0, 1, ..., max_N_to_test
    for N in range(max_N_to_test + 1):
        all_terms_after_N_are_close = True

```

```

# Verifier les termes a_n pour n > N
for k in range(1, terms_after_N_to_check + 1):
    n = N + k # L'indice du terme actuel (n > N)
    try:
        term_an = sequence_func(n)
    except ZeroDivisionError:
        # Gerer les cas ou sequence_func(n) pourrait diviser par
        # zero
        # pour de petits n. Cela signifie que la suite n'est pas
        # bien
        # definie pour ces termes, ou que N est trop petit.
        all_terms_after_N_are_close = False
        break
    except Exception as e:
        # Gerer d'autres erreurs potentielles lors de l'
        # evaluation de la suite
        print(f"Warning: Erreur lors de l'evaluation de
              sequence_func({n}): {e}")
        all_terms_after_N_are_close = False
        break

    # Si la distance est superieure ou egale a epsilon, ce N ne
    # fonctionne pas
    if distance(term_an, limit) >= epsilon:
        all_terms_after_N_are_close = False
        break # Passer au N suivant

# Si tous les termes verifiees apres ce N sont proches de la
# limite
if all_terms_after_N_are_close:
    return True # Preuve empirique de convergence trouvee

return False # Aucun N satisfaisant trouve dans la plage testee

# --- Definitions de suites pour les tests ---

# Suites reelles
def seq_real_convergent_to_0(n: int) -> float:
    """Suite a_n = 1/n, converge vers 0."""
    if n == 0:
        # Pour eviter la division par zero, on peut retourner une valeur
        # qui indique un probleme ou lever une erreur.
        # Dans le contexte de n > N, n sera toujours >= 1.
        raise ValueError("n doit etre > 0 pour cette suite.")
    return 1 / n

def seq_real_convergent_to_1(n: int) -> float:
    """Suite a_n = (n+1)/n, converge vers 1."""
    if n == 0:
        raise ValueError("n doit etre > 0 pour cette suite.")
    return (n + 1) / n

def seq_real_divergent_oscillating(n: int) -> float:
    """Suite a_n = (-1)^n, diverge (oscille entre -1 et 1)."""
    return float((-1)**n)

def seq_real_divergent_to_inf(n: int) -> float:
    """Suite a_n = n, diverge vers l'infini."""

```

```

    return float(n)

# Suites complexes
def seq_complex_convergent_to_0(n: int) -> complex:
    """Suite  $a_n = 1/n + i/(2n)$ , converge vers  $0 + 0i$ ."""
    if n == 0:
        raise ValueError("n doit etre > 0 pour cette suite.")
    return complex(1/n, 1/(2*n))

def seq_complex_convergent_to_1_plus_0_5i(n: int) -> complex:
    """Suite  $a_n = (n+1)/n + i*(n+1)/(2n)$ , converge vers  $1 + 0.5i$ ."""
    if n == 0:
        raise ValueError("n doit etre > 0 pour cette suite.")
    return complex((n+1)/n, (n+1)/(2*n))

def seq_complex_divergent_oscillating(n: int) -> complex:
    """Suite  $a_n = (-1)^n + i*(-1)^{(n+1)}$ , diverge."""
    return complex((-1)**n, (-1)**(n+1))

def seq_complex_divergent_to_inf(n: int) -> complex:
    """Suite  $a_n = n + i*n^2$ , diverge vers l'infini."""
    return complex(n, n*n)

def seq_complex_geometric_convergent(n: int) -> complex:
    """Suite geometrique  $a_n = (0.5 + 0.2i)^n$ , converge vers 0."""
    return (0.5 + 0.2j)**n

def seq_complex_geometric_divergent(n: int) -> complex:
    """Suite geometrique  $a_n = (1.1 + 0.1i)^n$ , diverge."""
    return (1.1 + 0.1j)**n

```

Validation Mathématique

La validation de notre implémentation se fera à l'aide d'assertions ('assert'). Chaque assertion représente un test basé sur une propriété mathématique connue d'une suite (convergence ou divergence) et de sa limite. Il est crucial de comprendre que 'is_convergent_epsilon_N' fournit une *preuve empirique* et non une preuve formelle. Les paramètres 'max_N_to_test' et 'terms_after_N_to_check' influencent la robustesse de cette preuve empirique. Des valeurs trop faibles pourraient faire échouer un test de convergence pour une suite qui converge lentement, tandis que des valeurs trop élevées augmentent le temps de calcul.

Nous allons tester différents scénarios :

- * Suites réelles convergentes vers leur limite attendue.
- * Suites réelles divergentes (oscillantes ou vers l'infini) ne convergeant pas vers une limite finie.
- * Suites complexes convergentes vers leur limite attendue.
- * Suites complexes divergentes.
- * Cas d'erreur (epsilon non positif).

```

print("--- Debut des tests de validation ---")

# --- Tests pour les suites reelles ---

# Test 1: Suite 1/n converge vers 0
assert is_convergent_epsilon_N(seq_real_convergent_to_0, 0.0, 0.1), \
    "Test 1 (Reel): 1/n devrait converger vers 0 (epsilon=0.1)"
assert is_convergent_epsilon_N(seq_real_convergent_to_0, 0.0, 0.01), \
    "Test 2 (Reel): 1/n devrait converger vers 0 (epsilon=0.01)"
assert not is_convergent_epsilon_N(seq_real_convergent_to_0, 0.5, 0.1), \
    "Test 3 (Reel): 1/n ne devrait PAS converger vers 0.5"

```

```

# Test 4: Suite  $(n+1)/n$  converge vers 1
assert is_convergent_epsilon_N(seq_real_convergent_to_1, 1.0, 0.1), \
    "Test 4 (Reel):  $(n+1)/n$  devrait converger vers 1 (epsilon=0.1)"
assert is_convergent_epsilon_N(seq_real_convergent_to_1, 1.0, 0.001), \
    "Test 5 (Reel):  $(n+1)/n$  devrait converger vers 1 (epsilon=0.001)"
assert not is_convergent_epsilon_N(seq_real_convergent_to_1, 0.9, 0.01), \
    "Test 6 (Reel):  $(n+1)/n$  ne devrait PAS converger vers 0.9"

# Test 7: Suite  $(-1)^n$  diverge
assert not is_convergent_epsilon_N(seq_real_divergent_oscillating, 1.0, 0.1), \
    "Test 7 (Reel):  $(-1)^n$  ne devrait PAS converger vers 1"
assert not is_convergent_epsilon_N(seq_real_divergent_oscillating, -1.0, 0.1), \
    "Test 8 (Reel):  $(-1)^n$  ne devrait PAS converger vers -1"
assert not is_convergent_epsilon_N(seq_real_divergent_oscillating, 0.0, 0.5), \
    "Test 9 (Reel):  $(-1)^n$  ne devrait PAS converger vers 0"

# Test 10: Suite  $n$  diverge vers l'infini (donc ne converge pas vers une limite finie)
assert not is_convergent_epsilon_N(seq_real_divergent_to_inf, 100.0, 10.0), \
    "Test 10 (Reel):  $n$  ne devrait PAS converger vers 100"

# --- Tests pour les suites complexes ---

# Test 11: Suite  $1/n + i/(2n)$  converge vers  $0 + 0i$ 
assert is_convergent_epsilon_N(seq_complex_convergent_to_0, complex(0, 0), 0.1), \
    "Test 11 (Complexe):  $1/n + i/(2n)$  devrait converger vers 0 (epsilon=0.1)"
assert is_convergent_epsilon_N(seq_complex_convergent_to_0, complex(0, 0), 0.01), \
    "Test 12 (Complexe):  $1/n + i/(2n)$  devrait converger vers 0 (epsilon=0.01)"
assert not is_convergent_epsilon_N(seq_complex_convergent_to_0, complex(0.1, 0), 0.05), \
    "Test 13 (Complexe):  $1/n + i/(2n)$  ne devrait PAS converger vers 0.1"

# Test 14: Suite  $(n+1)/n + i*(n+1)/(2n)$  converge vers  $1 + 0.5i$ 
assert is_convergent_epsilon_N(seq_complex_convergent_to_1_plus_0_5i, complex(1, 0.5), 0.1), \
    "Test 14 (Complexe):  $(n+1)/n + i*(n+1)/(2n)$  devrait converger vers  $1+0.5i$ "
assert is_convergent_epsilon_N(seq_complex_convergent_to_1_plus_0_5i, complex(1, 0.5), 0.001), \
    "Test 15 (Complexe):  $(n+1)/n + i*(n+1)/(2n)$  devrait converger vers  $1+0.5i$  (epsilon=0.001)"

# Test 16: Suite  $(-1)^n + i*(-1)^{(n+1)}$  diverge
assert not is_convergent_epsilon_N(seq_complex_divergent_oscillating, complex(0, 0), 0.5), \
    "Test 16 (Complexe):  $(-1)^n + i*(-1)^{(n+1)}$  ne devrait PAS converger vers 0"

```

```

# Test 17: Suite  $n + i \cdot n^2$  diverge vers l'infini
assert not is_convergent_epsilon_N(seq_complex_divergent_to_inf, complex
    (100, 100), 10.0), \
    "Test 17 (Complexe):  $n + i \cdot n^2$  ne devrait PAS converger vers  $100+100i$ "

# Test 18: Suite geometrique complexe convergente
assert is_convergent_epsilon_N(seq_complex_geometric_convergent, complex
    (0, 0), 0.1), \
    "Test 18 (Complexe):  $(0.5 + 0.2i)^n$  devrait converger vers 0"
assert is_convergent_epsilon_N(seq_complex_geometric_convergent, complex
    (0, 0), 0.001), \
    "Test 19 (Complexe):  $(0.5 + 0.2i)^n$  devrait converger vers 0 (
        epsilon=0.001)"

# Test 20: Suite geometrique complexe divergente
assert not is_convergent_epsilon_N(seq_complex_geometric_divergent,
    complex(0, 0), 1.0), \
    "Test 20 (Complexe):  $(1.1 + 0.1i)^n$  ne devrait PAS converger vers 0"
assert not is_convergent_epsilon_N(seq_complex_geometric_divergent,
    complex(10, 10), 5.0), \
    "Test 21 (Complexe):  $(1.1 + 0.1i)^n$  ne devrait PAS converger vers
         $10+10i$ "

# --- Tests de cas d'erreur et limites de l'approche empirique ---

# Test 22: Epsilon doit etre strictement positif
try:
    is_convergent_epsilon_N(seq_real_convergent_to_0, 0.0, 0.0)
    assert False, "Test 22: Epsilon <= 0 devrait lever une ValueError"
except ValueError:
    assert True, "Test 22: Epsilon <= 0 a correctement leve une
        ValueError"

# Test 23: Une suite qui converge tres lentement peut necessiter un
grand max_N_to_test
def seq_slow_convergent(n: int) -> float:
    """Suite  $a_n = 1/\sqrt{n}$ , converge lentement vers 0."""
    if n == 0:
        raise ValueError("n doit etre > 0 pour cette suite.")
    return 1 / math.sqrt(n)

# Avec un petit max_N_to_test, la convergence n'est pas detectee
assert not is_convergent_epsilon_N(seq_slow_convergent, 0.0, 0.1,
    max_N_to_test=5), \
    "Test 23a: Suite lente ne devrait PAS etre detectee avec un petit
        max_N_to_test"

# Avec un max_N_to_test plus grand, la convergence est detectee
assert is_convergent_epsilon_N(seq_slow_convergent, 0.0, 0.1,
    max_N_to_test=100), \
    "Test 23b: Suite lente devrait etre detectee avec un grand
        max_N_to_test"
assert is_convergent_epsilon_N(seq_slow_convergent, 0.0, 0.01,
    max_N_to_test=10000), \
    "Test 23c: Suite lente devrait etre detectee avec un tres grand
        max_N_to_test pour un petit epsilon"

print("--- Tous les tests de validation ont reussi ! ---")

```

Explication de la validation via 'assert' :

Les assertions sont utilisées ici comme des garde-fous pour vérifier que notre implémentation Python se comporte comme attendu d'un point de vue mathématique. Chaque 'assert' vérifie une condition qui doit être vraie. Si la condition est fausse, l'assertion échoue et lève une 'AssertionError', indiquant un problème dans le code ou une incompréhension du concept mathématique.

1. **Tests de convergence ('assert is _convergent_epsilon_N(...)')** : Pour les suites dont nous savons qu'elles convergent (ex : $1/n$ vers 0 , $(n+1)/n$ vers 1), nous nous attendons à ce que 'is_convergent_epsilon_N' retourne 'True' pour des valeurs d'épsilon raisonnables. Nous testons avec différentes valeurs d'épsilon pour s'assurer que la définition est respectée pour des "voisinages" de plus en plus petits de la limite.
2. **Tests de non-convergence ('assert not is _convergent_epsilon_N(...)')** : Pour les suites dont nous savons qu'elles divergent (ex : $(-1)^n$ 'quioscille', n 'quitend vers l'infini' ou pour des limites complexes).
3. **Tests de suites complexes** : Les mêmes principes sont appliqués aux suites complexes, en utilisant le type 'complex' de Python et en vérifiant la convergence vers des limites complexes. La distance est calculée dans le plan complexe.
4. **Tests de robustesse et cas limites** :

* Un test spécifique vérifie que la fonction lève une 'ValueError' si 'epsilon' est non positif, car la définition mathématique exige $\epsilon > 0$. * Un exemple de suite à convergence lente ($1/\sqrt{n}$) est utilisé pour illustrer l'importance des paramètres 'max_N_to_test' et 'terms_after_N_to_check'. Un petit 'max_N_to_test' peut faire échouer la détection de convergence, même si la suite converge, car l'algorithme n'a pas cherché "assez loin" pour trouver le 'N' approprié. Cela met en lumière la nature empirique de notre implémentation par rapport à une preuve formelle.

En exécutant ces assertions, nous obtenons une validation concrète et reproductible que notre code Python reflète fidèlement les concepts de convergence des suites réelles et complexes selon la définition rigoureuse (ϵ, N) , dans les limites d'une vérification finie.

TP 2 : Exploration des Suites Réelles et Complexes : Convergence et Preuves Formelles (ϵ, N)

Objectif

Ce TP vise à approfondir la compréhension des suites réelles et complexes, en se concentrant sur la définition rigoureuse de la limite d'une suite (la définition ϵ, N) et les critères de convergence. Les étudiants implémenteront des fonctions Python "from scratch" pour vérifier ces définitions, sans l'aide de bibliothèques mathématiques de haut niveau, afin de solidifier leur intuition et leur capacité à traduire des concepts mathématiques abstraits en code fonctionnel.

À l'issue de ce TP, vous serez capable de : * Comprendre et appliquer la définition formelle (ϵ, N) de la limite d'une suite. * Implémenter des fonctions Python pour tester la convergence de suites réelles et complexes. * Représenter et manipuler des nombres complexes en Python sans bibliothèques externes. * Utiliser des assertions pour valider mathématiquement le comportement de vos implémentations.

Implémentation Python pure

```
import math

class Complex:
```

```

"""
Represente un nombre complexe z = real + imag * i.
Implementation "from scratch" sans utiliser la bibliotheque cmath.
"""

def __init__(self, real: float, imag: float):
    self.real = real
    self.imag = imag

def __add__(self, other: 'Complex') -> 'Complex':
    """Addition de deux nombres complexes."""
    return Complex(self.real + other.real, self.imag + other.imag)

def __sub__(self, other: 'Complex') -> 'Complex':
    """Soustraction de deux nombres complexes."""
    return Complex(self.real - other.real, self.imag - other.imag)

def __mul__(self, other: 'Complex') -> 'Complex':
    """Multiplication de deux nombres complexes: (a + bi)(c + di) =
    (ac - bd) + (ad + bc)i."""
    return Complex(self.real * other.real - self.imag * other.imag,
                    self.real * other.imag + self.imag * other.real)

def __truediv__(self, other: 'Complex') -> 'Complex':
    """Division de deux nombres complexes: (a + bi) / (c + di) = (a
    + bi)(c - di) / (c^2 + d^2)."""
    denominator = other.real**2 + other.imag**2
    if denominator == 0:
        raise ZeroDivisionError("Division par zero complexe : le
        denominateur est nul.")

    # Multiplier le numerateur par le conjugué du denominateur
    numerator = self * Complex(other.real, -other.imag)

    return Complex(numerator.real / denominator, numerator.imag /
                    denominator)

def magnitude(self) -> float:
    """Calcule le module (magnitude) du nombre complexe: |z| = sqrt(
    real^2 + imag^2)."""
    return math.sqrt(self.real**2 + self.imag**2)

def __abs__(self) -> float:
    """Permet d'utiliser la fonction abs() sur une instance de
    Complex pour obtenir son module."""
    return self.magnitude()

def __eq__(self, other: object) -> bool:
    """
    Verifie l'egalite entre deux nombres complexes avec une petite
    tolerance
    pour les comparaisons de flottants.
    """
    if not isinstance(other, Complex):
        return NotImplemented
    # Utilisation d'une tolerance pour la comparaison des flottants
    return abs(self.real - other.real) < 1e-9 and abs(self.imag -
        other.imag) < 1e-9

```

```

def __str__(self) -> str:
    """Représentation textuelle du nombre complexe."""
    if self.imag >= 0:
        return f"{self.real} + {self.imag}i"
    return f"{self.real} - {-self.imag}i"

def __repr__(self) -> str:
    """Représentation officielle du nombre complexe pour le débogage
    """
    return f"Complex({self.real}, {self.imag})"

def check_epsilon_N_condition_real(
    sequence_func: callable[[int], float],
    limit: float,
    epsilon: float,
    max_N_search: int = 1000
) -> tuple[bool, int | None]:
    """
    Vérifie si, pour un epsilon donné, il existe un N tel que pour tout
    n > N,
    |sequence_func(n) - limit| < epsilon.
    La recherche de N est limitée à max_N_search.

    Args:
        sequence_func: Une fonction f(n) qui retourne le n-ième terme de
            la suite réelle.
            On suppose que n commence à 1 pour les suites.
        limit: La limite L proposée pour la suite.
        epsilon: La valeur epsilon pour la définition de la limite. Doit
            être strictement positif.
        max_N_search: Le nombre maximal de termes à vérifier pour
            trouver N.
            C'est une borne supérieure pour N_candidate.

    Returns:
        Un tuple (bool, int | None) indiquant si un N a été trouvé et sa
        valeur.
        Retourne (False, None) si aucun N n'est trouvé dans la plage de
        recherche.
    """
    if epsilon <= 0:
        raise ValueError("Epsilon doit être strictement positif.")

    # On cherche un N_candidate tel que pour tout n > N_candidate, la
    # condition est satisfaite.
    # On itère pour trouver le premier N_candidate qui satisfait la
    # condition pour les termes suivants.
    for N_candidate in range(1, max_N_search + 1):
        is_N_candidate_valid = True
        # Pour valider N_candidate, nous devons vérifier que pour TOUS
        # les n > N_candidate,
        # la condition |a_n - L| < epsilon est vraie.
        # Dans une implémentation numérique, nous ne pouvons pas
        # vérifier l'infini.
        # Nous vérifions donc un nombre suffisant de termes après
        # N_candidate.
        # Si la condition est fautive pour un seul n dans cette plage,
        # alors N_candidate n'est pas bon.

```



```

# La plage de verification va de N_candidate + 1 jusqu'a
    max_N_search + 50.
# Cela permet de s'assurer que N_candidate est robuste et que la
    suite reste
# dans l'intervalle (L-epsilon, L+epsilon) pour un certain temps
.
for n in range(N_candidate + 1, max_N_search + 50):
    try:
        term = sequence_func(n)
        if abs(term - limit) >= epsilon:
            is_N_candidate_valid = False
            break # N_candidate n'est pas bon, passons au
                suivant
    except ZeroDivisionError:
        # Gerer les cas ou sequence_func(n) pourrait diviser par
            zero (ex: 1/n pour n=0)
        # Pour les suites, n commence generalement a 1, donc
            cela devrait etre rare.
        is_N_candidate_valid = False
        break
    except Exception as e:
        print(f"Erreur lors du calcul du terme {n}: {e}")
        is_N_candidate_valid = False
        break

    if is_N_candidate_valid:
        return True, N_candidate

return False, None # Aucun N trouve dans la plage de recherche

def check_epsilon_N_condition_complex(
    sequence_func: callable[[int], Complex],
    limit: Complex,
    epsilon: float,
    max_N_search: int = 1000
) -> tuple[bool, int | None]:
    """
    Verifie si, pour un epsilon donne, il existe un N tel que pour tout
        n > N,
    |sequence_func(n) - limit| < epsilon.
    La recherche de N est limitee a max_N_search.

    Args:
        sequence_func: Une fonction f(n) qui retourne le n-ieme terme de
            la suite complexe.
            On suppose que n commence a 1 pour les suites.
        limit: La limite L proposee pour la suite (un objet Complex).
        epsilon: La valeur epsilon pour la definition de la limite. Doit
            etre strictement positif.
        max_N_search: Le nombre maximal de termes a verifier pour
            trouver N.

    Returns:
        Un tuple (bool, int | None) indiquant si un N a ete trouve et sa
            valeur.
        Retourne (False, None) si aucun N n'est trouve dans la plage de
            recherche.

```

```

"""
if epsilon <= 0:
    raise ValueError("Epsilon doit etre strictement positif.")

for N_candidate in range(1, max_N_search + 1):
    is_N_candidate_valid = True
    # Verification similaire a la fonction reelle, mais avec le
    module complexe.
    for n in range(N_candidate + 1, max_N_search + 50):
        try:
            term = sequence_func(n)
            # La distance pour les nombres complexes est le module
            de la difference
            if (term - limit).magnitude() >= epsilon:
                is_N_candidate_valid = False
                break
        except ZeroDivisionError:
            is_N_candidate_valid = False
            break
        except Exception as e:
            print(f"Erreur lors du calcul du terme complexe {n}: {e}")
            is_N_candidate_valid = False
            break

    if is_N_candidate_valid:
        return True, N_candidate

return False, None

# --- Definitions de suites reelles pour les tests ---
def sequence_un_sur_n(n: int) -> float:
    """Suite  $a_n = 1/n$ . Converge vers 0."""
    if n == 0: raise ZeroDivisionError("n ne peut pas etre zero pour 1/n")
    return 1.0 / n

def sequence_n_plus_un_sur_n(n: int) -> float:
    """Suite  $a_n = (n+1)/n$ . Converge vers 1."""
    if n == 0: raise ZeroDivisionError("n ne peut pas etre zero pour (n+1)/n")
    return (n + 1.0) / n

def sequence_moins_un_puissance_n(n: int) -> float:
    """Suite  $a_n = (-1)^n$ . Diverge (oscille entre -1 et 1)."""
    return float((-1)**n)

def sequence_constante_cinq(n: int) -> float:
    """Suite  $a_n = 5$ . Converge vers 5."""
    return 5.0

# --- Definitions de suites complexes pour les tests ---
def sequence_un_sur_n_plus_i_sur_n(n: int) -> Complex:
    """Suite  $z_n = 1/n + i/n$ . Converge vers  $0 + 0i$ ."""
    if n == 0: raise ZeroDivisionError("n ne peut pas etre zero pour 1/n + i/n")
    return Complex(1.0 / n, 1.0 / n)

```

```

def sequence_un_plus_i_puissance_n_sur_deux_puissance_n(n: int) ->
Complex:
    """Suite  $z_n = ((1+i)/2)^n$ . Converge vers  $0 + 0i$ .
    Le module de la base  $(1+i)/2$  est  $|(1+i)/2| = \sqrt{1^2+1^2}/2 = \sqrt{2}/2 \approx 0.707 < 1$ .
    Une suite geometrique de raison de module  $< 1$  converge vers 0."""

    base = Complex(1.0, 1.0)

    # Calcul de  $(1+i)^n$ 
    num = Complex(1.0, 0.0) # Initialiser a 1 (le terme pour n=0)
    if n > 0:
        for _ in range(n):
            num = num * base
    elif n == 0:
        num = Complex(1.0, 0.0) #  $(1+i)^0 = 1$ 
    else:
        raise ValueError("n doit etre un entier non negatif.")

    # Calcul de  $2^n$ 
    den = 2.0**n

    # Division complexe par un reel
    return Complex(num.real / den, num.imag / den)

def sequence_i_puissance_n(n: int) -> Complex:
    """Suite  $z_n = i^n$ . Diverge (oscille entre 1, i, -1, -i)."""
    if n < 0: raise ValueError("n doit etre un entier non negatif.")
    remainder = n % 4
    if remainder == 0: return Complex(1.0, 0.0)
    if remainder == 1: return Complex(0.0, 1.0)
    if remainder == 2: return Complex(-1.0, 0.0)
    if remainder == 3: return Complex(0.0, -1.0)
    return Complex(0.0, 0.0) # Ne devrait pas etre atteint

if __name__ == "__main__":
    print("--- Demarrage des tests de convergence ---")

    # --- Tests pour les suites reelles ---
    print("\n--- Suites Reelles ---")

    # Suite  $a_n = 1/n$ , converge vers 0
    print("\nTest de  $a_n = 1/n$ , limite L=0")
    found_N, N_val = check_epsilon_N_condition_real(sequence_un_sur_n,
        0.0, 0.1, max_N_search=100)
    assert found_N, f"Echec pour  $1/n$ , epsilon=0.1. N trouve: {N_val}"
    print(f"    Pour epsilon=0.1, N={N_val} trouve. Condition  $|1/n - 0| < 0.1$  verifiee pour  $n > {N_val}$ .")

    found_N, N_val = check_epsilon_N_condition_real(sequence_un_sur_n,
        0.0, 0.01, max_N_search=1000)
    assert found_N, f"Echec pour  $1/n$ , epsilon=0.01. N trouve: {N_val}"
    print(f"    Pour epsilon=0.01, N={N_val} trouve. Condition  $|1/n - 0| < 0.01$  verifiee pour  $n > {N_val}$ .")

    # Suite  $a_n = (n+1)/n$ , converge vers 1
    print("\nTest de  $a_n = (n+1)/n$ , limite L=1")

```

```

found_N, N_val = check_epsilon_N_condition_real(
    sequence_n_plus_un_sur_n, 1.0, 0.05, max_N_search=100)
assert found_N, f"Echec pour (n+1)/n, epsilon=0.05. N trouve: {N_val}"
print(f" Pour epsilon=0.05, N={N_val} trouve. Condition |(n+1)/n - 1| < 0.05 verifiee pour n > {N_val}.")

found_N, N_val = check_epsilon_N_condition_real(
    sequence_n_plus_un_sur_n, 1.0, 0.005, max_N_search=1000)
assert found_N, f"Echec pour (n+1)/n, epsilon=0.005. N trouve: {N_val}"
print(f" Pour epsilon=0.005, N={N_val} trouve. Condition |(n+1)/n - 1| < 0.005 verifiee pour n > {N_val}.")

# Suite an = (-1)^n, ne converge pas
print("\nTest de an = (-1)^n, ne converge pas")
# Si elle convergeait vers L, alors pour epsilon=0.5, il existerait N.
# Mais la suite oscille entre -1 et 1, donc |an - L| ne peut pas etre < 0.5 pour tout n > N.
# On peut tester si elle converge vers 0 (ce qui est faux).
found_N, N_val = check_epsilon_N_condition_real(
    sequence_moins_un_puissance_n, 0.0, 0.5, max_N_search=100)
assert not found_N, f"Echec : (-1)^n semble converger vers 0 avec epsilon=0.5. N trouve: {N_val}"
print(f" Pour epsilon=0.5, aucun N trouve pour L=0. C'est attendu, la suite ne converge pas vers 0.")

# Suite constante an = 5, converge vers 5
print("\nTest de an = 5, limite L=5")
found_N, N_val = check_epsilon_N_condition_real(
    sequence_constante_cinq, 5.0, 1e-9, max_N_search=10)
assert found_N, f"Echec pour an=5, epsilon=1e-9. N trouve: {N_val}"
print(f" Pour epsilon=1e-9, N={N_val} trouve. Condition |5 - 5| < 1e-9 verifiee pour n > {N_val}.")

# --- Tests pour les suites complexes ---
print("\n--- Suites Complexes ---")

# Suite zn = 1/n + i/n, converge vers 0 + 0i
print("\nTest de zn = 1/n + i/n, limite L=0+0i")
limit_complex_zero = Complex(0.0, 0.0)
found_N, N_val = check_epsilon_N_condition_complex(
    sequence_un_sur_n_plus_i_sur_n, limit_complex_zero, 0.1, max_N_search=100)
assert found_N, f"Echec pour 1/n + i/n, epsilon=0.1. N trouve: {N_val}"
print(f" Pour epsilon=0.1, N={N_val} trouve. Condition |(1/n + i/n) - (0+0i)| < 0.1 verifiee pour n > {N_val}.")

found_N, N_val = check_epsilon_N_condition_complex(
    sequence_un_sur_n_plus_i_sur_n, limit_complex_zero, 0.01, max_N_search=1000)
assert found_N, f"Echec pour 1/n + i/n, epsilon=0.01. N trouve: {N_val}"
print(f" Pour epsilon=0.01, N={N_val} trouve. Condition |(1/n + i/n) - (0+0i)| < 0.01 verifiee pour n > {N_val}.")

```

```

# Suite  $z_n = ((1+i)/2)^n$ , converge vers  $0 + 0i$ 
print("\nTest de  $z_n = ((1+i)/2)^n$ , limite  $L=0+0i$ ")
found_N, N_val = check_epsilon_N_condition_complex(
    sequence_un_plus_i_puissance_n_sur_deux_puissance_n,
    limit_complex_zero, 0.1, max_N_search=100)
assert found_N, f"Echec pour  $((1+i)/2)^n$ , epsilon=0.1. N trouve: {N_val}"
print(f"    Pour epsilon=0.1, N={N_val} trouve. Condition  $|((1+i)/2)^n - (0+0i)| < 0.1$  verifiee pour  $n > \{N\_val\}$ .")

found_N, N_val = check_epsilon_N_condition_complex(
    sequence_un_plus_i_puissance_n_sur_deux_puissance_n,
    limit_complex_zero, 0.01, max_N_search=1000)
assert found_N, f"Echec pour  $((1+i)/2)^n$ , epsilon=0.01. N trouve: {N_val}"
print(f"    Pour epsilon=0.01, N={N_val} trouve. Condition  $|((1+i)/2)^n - (0+0i)| < 0.01$  verifiee pour  $n > \{N\_val\}$ .")

# Suite  $z_n = i^n$ , ne converge pas
print("\nTest de  $z_n = i^n$ , ne converge pas")
# Si elle convergeait vers L, alors pour epsilon=0.5, il existerait N.
# La suite oscille entre 1, i, -1, -i. La distance entre ces points est au moins 1 (par exemple  $|1 - i| = \sqrt{2}$ ).
# Donc, elle ne peut pas rester dans un disque de rayon 0.5.
found_N, N_val = check_epsilon_N_condition_complex(
    sequence_i_puissance_n, limit_complex_zero, 0.5, max_N_search=100)
assert not found_N, f"Echec :  $i^n$  semble converger vers  $0+0i$  avec epsilon=0.5. N trouve: {N_val}"
print(f"    Pour epsilon=0.5, aucun N trouve pour  $L=0+0i$ . C'est attendu, la suite ne converge pas.")

print("\n--- Tous les tests de convergence ont reussi ! ---")

```

Validation Mathématique

La validation de ce TP repose sur l'application directe de la définition formelle de la limite d'une suite, dite définition ϵ, N .

Une suite $(a_n)_{n \in \mathbb{N}}$ converge vers une limite L si et seulement si : $\forall \epsilon > 0, \exists N \in \mathbb{N}$ tel que $\forall n > N, |a_n - L| < \epsilon$.

Pour les suites complexes $(z_n)_{n \in \mathbb{N}}$, la définition est similaire, mais la valeur absolue est remplacée par le module d'un nombre complexe : $\forall \epsilon > 0, \exists N \in \mathbb{N}$ tel que $\forall n > N, |z_n - L| < \epsilon$, où $|z_n - L|$ représente le module du nombre complexe $(z_n - L)$.

Nos fonctions Python, 'check_epsilon_N_condition_real' et 'check_epsilon_N_condition_complex', implémentent cette logique de la manière suivante :

1. **Paramètres** : Elles prennent en entrée une fonction 'sequence_func' (qui génère le n -ième terme), une 'limit' proposée, une valeur 'epsilon' spécifique, et un 'max_N_search' pour limiter la recherche de N .
2. **Recherche de N** : Elles itèrent sur des valeurs potentielles de N (de 1 à 'max_N_search'). Pour chaque 'N_candidate', elles vérifient si la condition ' $|a_n - L| < \epsilon$ ' (ou ' $|z_n - L| < \epsilon$ ') est satisfaite pour tous les termes n *strictement supérieurs* à 'N_candidate' (jusqu'à une certaine borne pratique, ici 'max_N_search + 50').
3. **Retour** : Si un 'N_candidate' est trouvé qui satisfait cette condition pour tous les n

testés dans la plage de vérification, la fonction retourne `(True, N_candidate)`. Sinon, elle retourne `(False, None)`.

Les assertions (`assert`) dans le bloc `if __name__ == "__main__":` sont utilisées pour valider que nos implémentations se comportent comme prévu pour des suites connues :

* **Suites convergentes (réelles et complexes)** : Pour ces suites, nous testons avec plusieurs valeurs d'épsilon (par exemple, 0.1 et 0.01). Nous nous attendons à ce que la fonction `check_epsilon_N_condition` retourne `True` et un `N` valide. Si l'assertion échoue, cela signifie que notre code n'a pas pu trouver un tel `N` dans la plage de recherche, ce qui indiquerait une erreur dans l'implémentation ou une plage de recherche `max_N_search` insuffisante. * Exemples : `'an = 1/n'` (converge vers 0), `'an = (n+1)/n'` (converge vers 1), `'an = 5'` (converge vers 5), `'zn = 1/n + i/n'` (converge vers $0+0i$), `'zn = ((1+i)/2)^n'` (*converge vers $0+0i$*).

* **Suites divergentes (réelles et complexes)** : Pour ces suites, nous testons avec un epsilon pour lequel nous savons qu'aucun N ne peut exister. Nous nous attendons à ce que la fonction `check_epsilon_N_condition` retourne `False`. Si l'assertion échoue (c'est-à-dire si la fonction retourne `True`), cela signifierait que notre code a "trouvé" une convergence là où il n'y en a pas, ce qui est une erreur grave. * Exemples : `'an = (-1)^n'` (*diverge*), `'zn = i^n'` (*diverge*). *Pour ces suites, peu importe la limite L proposée, la suite ne peut pas rester arbitrairement proche de L pour tout $\epsilon < 1$, donc il est impossible de les contenir dans un intervalle de rayon $\epsilon < 1$.*

Limitations de l'approche numérique : Il est crucial de noter que cette implémentation Python ne *prouve* pas formellement la convergence d'une suite pour *tout* $\epsilon > 0$. Elle *démontre* la convergence pour des ϵ spécifiques et pour une plage de N limitée par `max_N_search`. Une preuve mathématique formelle nécessiterait une démonstration analytique de l'existence de N pour *tout* ϵ . Cependant, pour les objectifs pédagogiques de ce TP, cette approche est excellente pour construire une intuition et vérifier la compréhension de la définition ϵ, N . La valeur de `max_N_search` est une heuristique. Une valeur trop petite pourrait faire échouer la recherche de N même pour une suite convergente, tandis qu'une valeur trop grande augmenterait le temps de calcul. Le choix de `max_N_search + 50` pour la vérification interne est également une heuristique pour s'assurer que le `N_candidate` trouvé est robuste pour un certain nombre de termes suivants.

TP 3 : Suites réelles et complexes, limites et convergence (ϵ, N)

Objectif

Ce TP vise à explorer les concepts fondamentaux des suites réelles et complexes, en mettant l'accent sur la définition rigoureuse de la limite d'une suite à l'aide des quantificateurs (ϵ, N). Nous implémenterons des fonctions Python "from scratch" pour :

1. **Définir et manipuler** des nombres complexes de base.
2. **Rechercher et vérifier** l'existence d'un N pour une ϵ donnée, conformément à la définition de la limite pour les suites réelles et complexes.
3. **Appliquer et vérifier** le critère de Cauchy pour les suites réelles, un critère essentiel de convergence.
4. **Valider** ces implémentations par des assertions mathématiques sur des exemples de suites convergentes et divergentes.

L'objectif pédagogique est de solidifier la compréhension des définitions formelles de l'analyse en les traduisant en code exécutable, tout en soulignant les défis et les approximations nécessaires lors de l'implémentation numérique de concepts infinis.

Implémentation Python pure

```
import math
from typing import Callable, Optional, Tuple

# --- Type Aliases pour la clarte ---
RealSequenceFunc = Callable[[int], float]
Complex = Tuple[float, float] # Representation d'un nombre complexe (
    partie reelle, partie imaginaire)
ComplexSequenceFunc = Callable[[int], Complex]

# --- Fonctions utilitaires pour les nombres complexes (implementees
    from scratch) ---

def complex_abs(z: Complex) -> float:
    """
    Calcule le module (valeur absolue) d'un nombre complexe z = (re, im)

     $|z| = \sqrt{re^2 + im^2}$ 
    """
    re, im = z
    return math.sqrt(re*re + im*im)

def complex_sub(z1: Complex, z2: Complex) -> Complex:
    """
    Calcule la soustraction de deux nombres complexes z1 - z2.
    (re1, im1) - (re2, im2) = (re1 - re2, im1 - im2)
    """
    re1, im1 = z1
    re2, im2 = z2
    return (re1 - re2, im1 - im2)

# --- Fonctions pour les suites reelles ---

def find_N_for_epsilon_real_sequence(
    sequence_func: RealSequenceFunc,
    limit_L: float,
    epsilon: float,
    N_start: int = 1, # Les suites commencent generalement a n=1 ou n=0
    N_max_search: int = 10000 # Limite superieure pour la recherche de N
) -> Optional[int]:
    """
    Tente de trouver un entier N tel que pour tout  $n > N$ ,  $|u_n - L| < \epsilon$ .

    Cette fonction cherche le plus petit N_candidate tel que la
    condition
     $|u_n - L| < \epsilon$  est respectee pour TOUS les n dans l'intervalle
    [N_candidate + 1, N_max_search].

    Il est crucial de noter que cette approche est une HEURISTIQUE pour
    un TP.
    Elle ne prouve pas formellement la convergence pour "tout  $n > N$ " car
    elle
    ne verifie qu'un nombre fini de termes (jusqu'a N_max_search).
    Pour une preuve formelle, N devrait etre derive analytiquement.

    Args:
```

```

sequence_func: Une fonction qui prend un entier n et retourne le
    terme u_n.
limit_L: La limite supposee de la suite.
epsilon: Une petite valeur positive.
N_start: L'indice de depart pour la recherche de N.
N_max_search: L'indice maximal jusqu'ou la recherche de N et la
    verification des termes s'etendront.

Returns:
    Le plus petit N trouve qui satisfait la condition pour les
        termes verifiees,
    ou None si aucun N n'est trouve dans la plage de recherche.

Raises:
    ValueError: Si epsilon est non-positif ou N_start est negatif.
"""
if epsilon <= 0:
    raise ValueError("epsilon doit etre strictement positif.")
if N_start < 0:
    raise ValueError("N_start doit etre non-negatif.")

for N_candidate in range(N_start, N_max_search + 1):
    is_N_found = True
    # Verifie la condition pour tous les n > N_candidate jusqu'a
    # N_max_search
    for n in range(N_candidate + 1, N_max_search + 1):
        try:
            term_value = sequence_func(n)
        except ZeroDivisionError:
            # Gere les cas ou sequence_func(n) n'est pas definie (ex
            # : 1/n pour n=0)
            # Si N_start est bien choisi, cela ne devrait pas
            # arriver pour n > N_candidate.
            # Si cela arrive, la suite n'est pas bien definie pour
            # ces termes.
            # Pour ce TP, on leve l'erreur pour indiquer un probleme
            # de definition de suite.
            raise ValueError(f"La suite n'est pas definie pour n={n}
                }. Ajustez N_start ou la fonction.")

        if abs(term_value - limit_L) >= epsilon:
            is_N_found = False
            break # Ce N_candidate ne fonctionne pas, on passe au
                suivant

    if is_N_found:
        return N_candidate

return None # Aucun N n'a ete trouve dans la plage de recherche

def verify_epsilon_N_real_sequence(
    sequence_func: RealSequenceFunc,
    limit_L: float,
    epsilon: float,
    N: int,
    check_count: int = 100 # Nombre de termes a verifier apres N
) -> bool:
    """

```



```

Verifie si la condition  $|u_n - L| < \epsilon$  est respectee pour  $n > N$ 
pour un nombre 'check_count' de termes consecutifs apres N.

Ceci est une verification pratique pour un TP, pas une preuve
formelle
pour "tout  $n > N$ ". Elle sert a confirmer qu'un N trouve est
plausible.

Args:
    sequence_func: La fonction de la suite.
    limit_L: La limite supposee.
    epsilon: La valeur epsilon.
    N: La valeur de N a verifier.
    check_count: Le nombre de termes a verifier apres N.

Returns:
    True si la condition est respectee pour tous les termes verifies
    , False sinon.

Raises:
    ValueError: Si epsilon est non-positif, N est negatif ou
    check_count est non-positif.
"""
if epsilon <= 0:
    raise ValueError("epsilon doit etre strictement positif.")
if N < 0:
    raise ValueError("N doit etre non-negatif.")
if check_count <= 0:
    raise ValueError("check_count doit etre strictement positif.")

for n in range(N + 1, N + 1 + check_count):
    try:
        term_value = sequence_func(n)
    except ZeroDivisionError:
        raise ValueError(f"La suite n'est pas definie pour n={n}.
        Verifiez la fonction ou N.")

    if abs(term_value - limit_L) >= epsilon:
        return False
return True

# --- Fonctions pour les suites complexes ---

def find_N_for_epsilon_complex_sequence(
    sequence_func: ComplexSequenceFunc,
    limit_L: Complex,
    epsilon: float,
    N_start: int = 1,
    N_max_search: int = 10000
) -> Optional[int]:
    """
    Tente de trouver un entier N tel que pour tout  $n > N$ ,  $|z_n - L| < \epsilon$ .
    Similaire a la version reelle, mais utilise l'arithmetique complexe.

    Cette fonction cherche le plus petit N_candidate tel que la
    condition
     $|z_n - L| < \epsilon$  est respectee pour TOUS les n dans l'intervalle

```

```

[N_candidate + 1, N_max_search].

Comme pour les suites reelles, cette approche est une HEURISTIQUE
pour un TP
et ne constitue pas une preuve formelle pour "tout  $n > N$ ".

Args:
    sequence_func: Une fonction qui prend un entier n et retourne le
        terme  $z_n$  (Complex).
    limit_L: La limite supposee de la suite (Complex).
    epsilon: Une petite valeur positive.
    N_start: L'indice de depart pour la recherche de N.
    N_max_search: L'indice maximal jusqu'ou la recherche de N et la
        verification des termes s'etendront.

Returns:
    Le plus petit N trouve qui satisfait la condition pour les
        termes verifiees,
    ou None si aucun N n'est trouve dans la plage de recherche.

Raises:
    ValueError: Si epsilon est non-positif ou N_start est negatif.
    """
    if epsilon <= 0:
        raise ValueError("epsilon doit etre strictement positif.")
    if N_start < 0:
        raise ValueError("N_start doit etre non-negatif.")

    for N_candidate in range(N_start, N_max_search + 1):
        is_N_found = True
        for n in range(N_candidate + 1, N_max_search + 1):
            try:
                term_value = sequence_func(n)
            except ZeroDivisionError:
                raise ValueError(f"La suite n'est pas definie pour n={n}
                    }. Ajustez N_start ou la fonction.")

            diff = complex_sub(term_value, limit_L)
            if complex_abs(diff) >= epsilon:
                is_N_found = False
                break

        if is_N_found:
            return N_candidate

    return None

def verify_epsilon_N_complex_sequence(
    sequence_func: ComplexSequenceFunc,
    limit_L: Complex,
    epsilon: float,
    N: int,
    check_count: int = 100
) -> bool:
    """
    Verifie si la condition  $|z_n - L| < \epsilon$  est respectee pour  $n > N$ 
    pour un nombre 'check_count' de termes consecutifs apres N.

```

Ceci est une verification pratique pour un TP, pas une preuve formelle.

Args:

sequence_func: La fonction de la suite.
limit_L: La limite supposee (Complex).
epsilon: La valeur epsilon.
N: La valeur de N a verifier.
check_count: Le nombre de termes a verifier apres N.

Returns:

True si la condition est respectee pour tous les termes verifiees,
False sinon.

Raises:

ValueError: Si epsilon est non-positif, N est negatif ou
check_count est non-positif.

"""

```
if epsilon <= 0:
    raise ValueError("epsilon doit etre strictement positif.")
if N < 0:
    raise ValueError("N doit etre non-negatif.")
if check_count <= 0:
    raise ValueError("check_count doit etre strictement positif.")

for n in range(N + 1, N + 1 + check_count):
    try:
        term_value = sequence_func(n)
    except ZeroDivisionError:
        raise ValueError(f"La suite n'est pas definie pour n={n}.  
Verifiez la fonction ou N.")

    diff = complex_sub(term_value, limit_L)
    if complex_abs(diff) >= epsilon:
        return False
return True
```

--- Critere de Cauchy pour les suites reelles ---

```
def find_N_for_epsilon_cauchy_real_sequence(
    sequence_func: RealSequenceFunc,
    epsilon: float,
    N_start: int = 1,
    N_max_search: int = 10000,
    check_range_after_N: int = 50 # Nombre de termes a verifier pour m
    et n apres N
) -> Optional[int]:
    """
    Tente de trouver un entier N tel que pour tout m, n > N, |u_m - u_n|
    < epsilon.

    Cette fonction cherche le plus petit N_candidate tel que la
    condition
    |u_m - u_n| < epsilon est respectee pour TOUS les n dans l'
    intervalle
    [N_candidate + 1, N_max_search - check_range_after_N] et pour TOUS
    les m
    dans l'intervalle [n + 1, n + check_range_after_N].
    """
```

Comme pour la recherche de limite, cette approche est une
 HEURISTIQUE
 pour un TP et ne constitue pas une preuve formelle de la propriete
 de Cauchy
 pour "tout $m, n > N$ " car elle ne verifie qu'un nombre fini de paires
 (m, n) .

Args:

sequence_func: La fonction de la suite.
 epsilon: La valeur epsilon.
 N_start: L'indice de depart pour la recherche de N.
 N_max_search: L'indice maximal jusqu'ou la recherche de N et la
 verification des termes s'etendront.
 check_range_after_N: Le nombre de termes m a verifier apres n.

Returns:

Le plus petit N trouve qui satisfait la condition pour les
 termes verifiees,
 ou None si aucun N n'est trouve dans la plage de recherche.

Raises:

ValueError: Si epsilon est non-positif, N_start est negatif ou
 check_range_after_N est non-positif.

"""

```

if epsilon <= 0:
    raise ValueError("epsilon doit etre strictement positif.")
if N_start < 0:
    raise ValueError("N_start doit etre non-negatif.")
if check_range_after_N <= 0:
    raise ValueError("check_range_after_N doit etre strictement
        positif.")

for N_candidate in range(N_start, N_max_search + 1):
    is_N_found = True
    # On verifie pour n allant de N_candidate + 1 jusqu'a une
    # certaine limite
    # pour laisser de la place pour m.
    for n in range(N_candidate + 1, N_max_search -
        check_range_after_N + 1):
        try:
            u_n = sequence_func(n)
        except ZeroDivisionError:
            raise ValueError(f"La suite n'est pas definie pour n={n}
                }. Verifiez la fonction ou N_start.")

        # On verifie m > n
        for m in range(n + 1, n + 1 + check_range_after_N):
            try:
                u_m = sequence_func(m)
            except ZeroDivisionError:
                raise ValueError(f"La suite n'est pas definie pour m
                    ={m}. Verifiez la fonction ou N_start.")

            if abs(u_m - u_n) >= epsilon:
                is_N_found = False
                break
    if not is_N_found:

```

```

        break

    if is_N_found:
        return N_candidate

    return None

def verify_epsilon_N_cauchy_real_sequence(
    sequence_func: RealSequenceFunc,
    epsilon: float,
    N: int,
    check_count: int = 100, # Nombre de termes n a verifier apres N
    check_range_for_m: int = 10 # Nombre de termes m a verifier apres n
) -> bool:
    """
    Verifie si la condition de Cauchy  $|u_m - u_n| < \epsilon$  est
    respectee
    pour  $n > N$  et  $m > n$ , pour un nombre limite de termes.

    Ceci est une verification pratique pour un TP, pas une preuve
    formelle.

    Args:
        sequence_func: La fonction de la suite.
        epsilon: La valeur epsilon.
        N: La valeur de N a verifier.
        check_count: Le nombre de termes n a verifier apres N.
        check_range_for_m: Le nombre de termes m a verifier apres n.

    Returns:
        True si la condition est respectee pour tous les termes verifies
        , False sinon.

    Raises:
        ValueError: Si epsilon est non-positif, N est negatif,
        check_count ou check_range_for_m sont non-positifs.
    """
    if epsilon <= 0:
        raise ValueError("epsilon doit etre strictement positif.")
    if N < 0:
        raise ValueError("N doit etre non-negatif.")
    if check_count <= 0 or check_range_for_m <= 0:
        raise ValueError("check_count et check_range_for_m doivent etre
        strictement positifs.")

    for n in range(N + 1, N + 1 + check_count):
        try:
            u_n = sequence_func(n)
        except ZeroDivisionError:
            raise ValueError(f"La suite n'est pas definie pour n={n}.
            Verifiez la fonction ou N.")

        for m in range(n + 1, n + 1 + check_range_for_m):
            try:
                u_m = sequence_func(m)
            except ZeroDivisionError:
                raise ValueError(f"La suite n'est pas definie pour m={m}
                }. Verifiez la fonction ou N.")

```

```

        if abs(u_m - u_n) >= epsilon:
            return False
    return True

# --- Exemples de suites pour la validation ---

# Suites reelles
def sequence_un_sur_n(n: int) -> float:
    """Suite  $u_n = 1/n$ . Converge vers 0."""
    if n == 0: raise ZeroDivisionError("n ne peut pas etre 0 pour 1/n")
    return 1.0 / n

def sequence_n_plus_un_sur_n(n: int) -> float:
    """Suite  $u_n = (n+1)/n$ . Converge vers 1."""
    if n == 0: raise ZeroDivisionError("n ne peut pas etre 0 pour (n+1)/n")
    return (n + 1.0) / n

def sequence_alternante_moins_un_puissance_n(n: int) -> float:
    """Suite  $u_n = (-1)^n$ . Diverge."""
    return float((-1)**n)

def sequence_constante_cinq(n: int) -> float:
    """Suite  $u_n = 5$ . Converge vers 5."""
    return 5.0

# Suites complexes
def sequence_complexe_un_plus_i_sur_n(n: int) -> Complex:
    """Suite  $z_n = (1+i)/n$ . Converge vers (0,0)."""
    if n == 0: raise ZeroDivisionError("n ne peut pas etre 0 pour (1+i)/n")
    return (1.0 / n, 1.0 / n) # (re, im)

def sequence_complexe_i_puissance_n(n: int) -> Complex:
    """Suite  $z_n = i^n$ . Diverge."""
    #  $i^0 = 1, i^1 = i, i^2 = -1, i^3 = -i, i^4 = 1, \dots$ 
    remainder = n % 4
    if remainder == 0: return (1.0, 0.0)
    elif remainder == 1: return (0.0, 1.0)
    elif remainder == 2: return (-1.0, 0.0)
    else: return (0.0, -1.0)

def sequence_complexe_un_sur_n_carre_plus_i_sur_n(n: int) -> Complex:
    """Suite  $z_n = 1/n^2 + i/n$ . Converge vers (0,0)."""
    if n == 0: raise ZeroDivisionError("n ne peut pas etre 0 pour 1/n^2 + i/n")
    return (1.0 / (n*n), 1.0 / n)

```

Validation Mathématique

Nous allons utiliser des assertions ('assert') pour valider le comportement de nos fonctions avec des suites connues. Les 'find_N' sont des heuristiques, donc les 'N' trouvés peuvent varier légèrement en fonction de 'N_max_search', mais ils devraient exister pour les suites convergentes. Les 'verify_epsilon_N' sont plus directes pour confirmer la propriété pour un N donné.

```

# --- Validation des fonctions utilitaires complexes ---
assert complex_abs((3.0, 4.0)) == 5.0, "complex_abs echoue pour (3,4)"

```

```

assert complex_sub((5.0, 2.0), (3.0, 1.0)) == (2.0, 1.0), "complex_sub
    echoue"

print("Validation des utilitaires complexes : OK")

# --- Validation des suites reelles ---

# Suite  $u_n = 1/n$ ,  $L = 0$ 
print("\n--- Validation Suite Reelle :  $u_n = 1/n$ ,  $L = 0$  ---")
epsilon1 = 0.1
N1 = find_N_for_epsilon_real_sequence(sequence_un_sur_n, 0.0, epsilon1)
# Analytiquement,  $N = \text{floor}(1/\text{epsilon}) = \text{floor}(1/0.1) = 10$ . Notre
    fonction devrait trouver un  $N \geq 10$ .
assert N1 is not None and N1 >= 10, f"find_N_for_epsilon_real_sequence
    echoue pour  $1/n$ , epsilon={epsilon1}"
assert verify_epsilon_N_real_sequence(sequence_un_sur_n, 0.0, epsilon1,
    N1), \
    f"verify_epsilon_N_real_sequence echoue pour  $1/n$ , epsilon={epsilon1}
    }, N={N1}"
print(f"Pour  $u_n=1/n$ ,  $L=0$ , epsilon={epsilon1}, N trouve: {N1}")

epsilon2 = 0.01
N2 = find_N_for_epsilon_real_sequence(sequence_un_sur_n, 0.0, epsilon2)
# Analytiquement,  $N = \text{floor}(1/0.01) = 100$ .
assert N2 is not None and N2 >= 100, f"find_N_for_epsilon_real_sequence
    echoue pour  $1/n$ , epsilon={epsilon2}"
assert verify_epsilon_N_real_sequence(sequence_un_sur_n, 0.0, epsilon2,
    N2), \
    f"verify_epsilon_N_real_sequence echoue pour  $1/n$ , epsilon={epsilon2}
    }, N={N2}"
print(f"Pour  $u_n=1/n$ ,  $L=0$ , epsilon={epsilon2}, N trouve: {N2}")

# Suite  $u_n = (n+1)/n$ ,  $L = 1$ 
print("\n--- Validation Suite Reelle :  $u_n = (n+1)/n$ ,  $L = 1$  ---")
epsilon3 = 0.05
N3 = find_N_for_epsilon_real_sequence(sequence_n_plus_un_sur_n, 1.0,
    epsilon3)
# Analytiquement,  $|(n+1)/n - 1| = |1/n| < \text{epsilon} \Rightarrow n > 1/\text{epsilon} =
    1/0.05 = 20$ .  $N \geq 20$ .
assert N3 is not None and N3 >= 20, f"find_N_for_epsilon_real_sequence
    echoue pour  $(n+1)/n$ , epsilon={epsilon3}"
assert verify_epsilon_N_real_sequence(sequence_n_plus_un_sur_n, 1.0,
    epsilon3, N3), \
    f"verify_epsilon_N_real_sequence echoue pour  $(n+1)/n$ , epsilon={
    epsilon3}, N={N3}"
print(f"Pour  $u_n=(n+1)/n$ ,  $L=1$ , epsilon={epsilon3}, N trouve: {N3}")

# Suite  $u_n = (-1)^n$  (divergente)
print("\n--- Validation Suite Reelle :  $u_n = (-1)^n$  (divergente) ---")
epsilon_div = 0.5 # Pour une suite divergente, on ne devrait pas trouver
    N pour un petit epsilon
N_div = find_N_for_epsilon_real_sequence(
    sequence_alternante_moins_un_puissance_n, 0.0, epsilon_div,
    N_max_search=200)
assert N_div is None, f"find_N_for_epsilon_real_sequence a trouve un N
    pour une suite divergente: {N_div}"
print(f"Pour  $u_n=(-1)^n$ ,  $L=0$ , epsilon={epsilon_div}, N trouve: {N_div} (
    attendu None, car divergente)")

```

```

# Suite u_n = 5 (constante)
print("\n--- Validation Suite Reelle : u_n = 5, L = 5 ---")
epsilon_const = 0.001
N_const = find_N_for_epsilon_real_sequence(sequence_constante_cinq, 5.0,
    epsilon_const)
# Pour une suite constante, N peut etre 0 ou 1 (des le debut, la
    condition est respectee)
assert N_const is not None and N_const < 2, f"
    find_N_for_epsilon_real_sequence echoue pour suite constante"
assert verify_epsilon_N_real_sequence(sequence_constante_cinq, 5.0,
    epsilon_const, N_const), \
    f"verify_epsilon_N_real_sequence echoue pour suite constante"
print(f"Pour u_n=5, L=5, epsilon={epsilon_const}, N trouve: {N_const}")

print("\n--- Validation des suites reelles : OK ---")

# --- Validation des suites complexes ---

# Suite z_n = (1+i)/n, L = (0,0)
print("\n--- Validation Suite Complexe : z_n = (1+i)/n, L = (0,0) ---")
epsilon_c1 = 0.1
N_c1 = find_N_for_epsilon_complex_sequence(
    sequence_complexe_un_plus_i_sur_n, (0.0, 0.0), epsilon_c1)
# Analytiquement, |(1+i)/n - 0| = |1+i|/n = sqrt(2)/n. On veut sqrt(2)/n
    < epsilon => n > sqrt(2)/epsilon.
# Pour epsilon=0.1, n > sqrt(2)/0.1 approx 14.14. Donc N >= 14.
assert N_c1 is not None and N_c1 >= 14, f"
    find_N_for_epsilon_complex_sequence echoue pour (1+i)/n, epsilon={
    epsilon_c1}"
assert verify_epsilon_N_complex_sequence(
    sequence_complexe_un_plus_i_sur_n, (0.0, 0.0), epsilon_c1, N_c1), \
    f"verify_epsilon_N_complex_sequence echoue pour (1+i)/n, epsilon={
    epsilon_c1}, N={N_c1}"
print(f"Pour z_n=(1+i)/n, L=(0,0), epsilon={epsilon_c1}, N trouve: {N_c1
    }")

epsilon_c2 = 0.01
N_c2 = find_N_for_epsilon_complex_sequence(
    sequence_complexe_un_plus_i_sur_n, (0.0, 0.0), epsilon_c2)
# Analytiquement, n > sqrt(2)/0.01 approx 141.4. Donc N >= 141.
assert N_c2 is not None and N_c2 >= 141, f"
    find_N_for_epsilon_complex_sequence echoue pour (1+i)/n, epsilon={
    epsilon_c2}"
assert verify_epsilon_N_complex_sequence(
    sequence_complexe_un_plus_i_sur_n, (0.0, 0.0), epsilon_c2, N_c2), \
    f"verify_epsilon_N_complex_sequence echoue pour (1+i)/n, epsilon={
    epsilon_c2}, N={N_c2}"
print(f"Pour z_n=(1+i)/n, L=(0,0), epsilon={epsilon_c2}, N trouve: {N_c2
    }")

# Suite z_n = i^n (divergente)
print("\n--- Validation Suite Complexe : z_n = i^n (divergente) ---")
epsilon_c_div = 0.5
N_c_div = find_N_for_epsilon_complex_sequence(
    sequence_complexe_i_puissance_n, (0.0, 0.0), epsilon_c_div,
    N_max_search=200)
assert N_c_div is None, f"find_N_for_epsilon_complex_sequence a trouve

```



```

    un N pour une suite divergente: {N_c_div}"
print(f"Pour  $z_n=i^n$ ,  $L=(0,0)$ ,  $\epsilon=\{\epsilon_{c\_div}\}$ , N trouve: {
    N_c_div} (attendu None, car divergente)")

print("\n--- Validation des suites complexes : OK ---")

# --- Validation du critere de Cauchy pour les suites reelles ---

# Suite  $u_n = 1/n$  (convergente, donc de Cauchy)
print("\n--- Validation Critere de Cauchy :  $u_n = 1/n$  (convergente) ---"
    )
epsilon_cauchy1 = 0.1
N_cauchy1 = find_N_for_epsilon_cauchy_real_sequence(sequence_un_sur_n,
    epsilon_cauchy1)
# Analytiquement,  $|1/m - 1/n| < \epsilon$ . Pour  $m > n$ ,  $1/n < \epsilon$ . Donc
    N  $\geq 1/\epsilon = 10$ .
assert N_cauchy1 is not None and N_cauchy1  $\geq 10$ , f"
    find_N_for_epsilon_cauchy_real_sequence echoue pour  $1/n$ ,  $\epsilon=\{
    \epsilon_{cauchy1}\}$ "
assert verify_epsilon_N_cauchy_real_sequence(sequence_un_sur_n,
    epsilon_cauchy1, N_cauchy1), \
    f"verify_epsilon_N_cauchy_real_sequence echoue pour  $1/n$ ,  $\epsilon=\{
    \epsilon_{cauchy1}\}$ , N={N_cauchy1}"
print(f"Pour  $u_n=1/n$ ,  $\epsilon=\{\epsilon_{cauchy1}\}$ , N de Cauchy trouve: {
    N_cauchy1}")

epsilon_cauchy2 = 0.01
N_cauchy2 = find_N_for_epsilon_cauchy_real_sequence(sequence_un_sur_n,
    epsilon_cauchy2)
# Analytiquement, N  $\geq 1/\epsilon = 100$ .
assert N_cauchy2 is not None and N_cauchy2  $\geq 100$ , f"
    find_N_for_epsilon_cauchy_real_sequence echoue pour  $1/n$ ,  $\epsilon=\{
    \epsilon_{cauchy2}\}$ "
assert verify_epsilon_N_cauchy_real_sequence(sequence_un_sur_n,
    epsilon_cauchy2, N_cauchy2), \
    f"verify_epsilon_N_cauchy_real_sequence echoue pour  $1/n$ ,  $\epsilon=\{
    \epsilon_{cauchy2}\}$ , N={N_cauchy2}"
print(f"Pour  $u_n=1/n$ ,  $\epsilon=\{\epsilon_{cauchy2}\}$ , N de Cauchy trouve: {
    N_cauchy2}")

# Suite  $u_n = (-1)^n$  (divergente, donc pas de Cauchy)
print("\n--- Validation Critere de Cauchy :  $u_n = (-1)^n$  (divergente)
    ---")
epsilon_cauchy_div = 1.0 #  $|u_m - u_n|$  peut etre 2 pour m,n de parite
    differente
N_cauchy_div = find_N_for_epsilon_cauchy_real_sequence(
    sequence_alternante_moins_un_puissance_n, epsilon_cauchy_div,
    N_max_search=200)
assert N_cauchy_div is None, f"find_N_for_epsilon_cauchy_real_sequence a
    trouve un N pour une suite non-Cauchy: {N_cauchy_div}"
print(f"Pour  $u_n=(-1)^n$ ,  $\epsilon=\{\epsilon_{cauchy\_div}\}$ , N de Cauchy
    trouve: {N_cauchy_div} (attendu None, car non-Cauchy)")

print("\n--- Validation du critere de Cauchy : OK ---")

print("\nTous les tests de validation mathematique sont passes avec
    succes !")

```

TP 4 : Suites Réelles et Complexes : Limites (ϵ, N) et Convergence

Objectif

Ce TP vise à approfondir la compréhension des définitions rigoureuses des limites de suites réelles et complexes, ainsi que des critères de convergence, en utilisant la formalisation ϵ, N . Nous allons :

1. **Comprendre les définitions** formelles des limites de suites réelles et complexes.
2. **Implémenter en Python pur** (sans bibliothèques mathématiques de haut niveau comme NumPy ou SciPy) des fonctions permettant de vérifier ces définitions pour des paramètres ϵ et N donnés, sur un nombre fini de termes.
3. **Explorer le critère de Cauchy** pour la convergence des suites.
4. **Valider mathématiquement** les implémentations à l'aide d'assertions sur des exemples de suites connues (convergentes, divergentes, de Cauchy, non de Cauchy).

Il est important de noter que le code Python ne peut pas *prouver* la convergence d'une suite (qui est une propriété "pour tout $\epsilon > 0$, il existe un N "), mais il peut *vérifier* si la condition est satisfaite pour des ϵ et N spécifiques et un échantillon de termes, ou démontrer qu'elle ne l'est pas.

Implémentation Python pure

```
import math
from typing import Callable, Union

# --- Fonctions de definition de suites pour les tests ---

def seq_one_over_n(n: int) -> float:
    """
    Definit la suite reelle u_n = 1/n.
    Generalement, n >= 1 pour cette suite.
    """
    if n == 0:
        # Pour eviter la division par zero, on leve une erreur car les
        # suites sont souvent definies pour n >= 1.
        raise ValueError("n doit etre superieur ou egal a 1 pour la
                           suite 1/n.")
    return 1.0 / n

def seq_n(n: int) -> float:
    """
    Definit la suite reelle u_n = n.
    """
    return float(n)

def seq_minus_one_pow_n(n: int) -> float:
    """
    Definit la suite reelle u_n = (-1)^n.
    """
    return float((-1)**n)

def seq_complex_one_over_n(n: int) -> complex:
    """
    Definit la suite complexe u_n = (1/n) + i(1/n).
    Generalement, n >= 1 pour cette suite.
    """
```

```

    if n == 0:
        raise ValueError("n doit etre superieur ou egal a 1 pour la
            suite (1/n) + i(1/n).")
    return complex(1.0 / n, 1.0 / n)

def seq_complex_n(n: int) -> complex:
    """
    Definit la suite complexe  $u_n = n + i*n$ .
    """
    return complex(float(n), float(n))

# --- Fonctions de verification des definitions rigoureuses ---

def check_epsilon_N_condition_real(
    sequence_func: Callable[[int], float],
    limit: float,
    epsilon: float,
    N_candidate: int,
    num_terms_to_verify: int = 100
) -> bool:
    """
    Verifie si la condition de limite  $|u_n - \text{limit}| < \text{epsilon}$  est
        satisfaite pour  $n > N\_candidate$ 
    sur un nombre limite de termes pour une suite reelle.

    La definition formelle d'une limite L pour une suite reelle ( $u_n$ )
        est :
    Pour tout  $\text{epsilon} > 0$ , il existe un entier N tel que pour tout  $n > N$ 
        ,  $|u_n - L| < \text{epsilon}$ .

    Cette fonction verifie cette condition pour un epsilon, un
        N_candidate donnees,
    et pour les 'num_terms_to_verify' premiers termes apres N_candidate.
    Elle ne prouve pas la convergence, mais verifie la condition pour un
        cas specifique.

    Args:
        sequence_func: Une fonction qui prend un entier n (indice du
            terme) et retourne le n-ieme terme de la suite (float).
            On suppose que sequence_func(n) est definie pour
                 $n \geq 1$ .
        limit: La limite supposee de la suite.
        epsilon: La valeur epsilon (doit etre strictement positive).
        N_candidate: La valeur N (doit etre un entier non negatif).
        num_terms_to_verify: Le nombre de termes a verifier apres
            N_candidate.

                                La verification se fait pour n dans [
                                    N_candidate + 1, N_candidate +
                                    num_terms_to_verify].

    Returns:
        True si la condition est satisfaite pour tous les termes
            verifies, False sinon.
    """
    if epsilon <= 0:
        raise ValueError("Epsilon doit etre strictement positif.")
    if N_candidate < 0:
        raise ValueError("N_candidate doit etre un entier non negatif.")

```

```

if num_terms_to_verify <= 0:
    raise ValueError("num_terms_to_verify doit etre strictement
        positif.")

for n in range(N_candidate + 1, N_candidate + 1 +
    num_terms_to_verify):
    try:
        u_n = sequence_func(n)
    except ValueError as e:
        # Si un terme ne peut pas etre calcule, la condition n'est
        pas satisfaite pour cette plage.
        print(f"Erreur lors du calcul de u_{n}: {e}. La condition n'
            est pas satisfaite.")
        return False
    if abs(u_n - limit) >= epsilon:
        return False
return True

def check_epsilon_N_condition_complex(
    sequence_func: Callable[[int], complex],
    limit: complex,
    epsilon: float,
    N_candidate: int,
    num_terms_to_verify: int = 100
) -> bool:
    """
    Verifie si la condition de limite  $|u_n - \text{limit}| < \text{epsilon}$  est
        satisfaite pour  $n > N\_candidate$ 
    sur un nombre limite de termes pour une suite complexe.

    La definition formelle d'une limite  $L$  pour une suite complexe ( $u_n$ )
        est :
    Pour tout  $\text{epsilon} > 0$ , il existe un entier  $N$  tel que pour tout  $n > N$ 
        ,  $|u_n - L| < \text{epsilon}$ .
    Ici,  $|\dots|$  represente le module d'un nombre complexe.

    Args:
        sequence_func: Une fonction qui prend un entier  $n$  et retourne le
             $n$ -ieme terme de la suite (complex).
            On suppose que  $\text{sequence\_func}(n)$  est definie pour
                 $n \geq 1$ .
        limit: La limite supposee de la suite complexe.
        epsilon: La valeur epsilon (doit etre strictement positive).
        N_candidate: La valeur  $N$  (doit etre un entier non negatif).
        num_terms_to_verify: Le nombre de termes a verifier apres
             $N\_candidate$ .

            La verification se fait pour  $n$  dans [
                 $N\_candidate + 1, N\_candidate +$ 
                 $\text{num\_terms\_to\_verify}$ ].

    Returns:
        True si la condition est satisfaite pour tous les termes
            verifies, False sinon.
    """
    if epsilon <= 0:
        raise ValueError("Epsilon doit etre strictement positif.")
    if N_candidate < 0:
        raise ValueError("N_candidate doit etre un entier non negatif.")

```

```

if num_terms_to_verify <= 0:
    raise ValueError("num_terms_to_verify doit etre strictement
        positif.")

for n in range(N_candidate + 1, N_candidate + 1 +
    num_terms_to_verify):
    try:
        u_n = sequence_func(n)
    except ValueError as e:
        print(f"Erreur lors du calcul de u_{n}: {e}. La condition n'
            est pas satisfaite.")
        return False
    # abs() pour les nombres complexes retourne le module
    if abs(u_n - limit) >= epsilon:
        return False
return True

def is_cauchy_sequence_test(
    sequence_func: Callable[[int], Union[float, complex]],
    epsilon: float,
    N_candidate: int,
    num_terms_to_verify: int = 20
) -> bool:
    """
    Verifie si la condition de Cauchy  $|u_n - u_m| < \epsilon$  est
        satisfaite pour  $n, m > N\_candidate$ 
        sur un nombre limite de termes.

    Une suite  $(u_n)$  est dite de Cauchy si :
    Pour tout  $\epsilon > 0$ , il existe un entier  $N$  tel que pour tout  $n, m$ 
         $> N$ ,  $|u_n - u_m| < \epsilon$ .

    Cette fonction verifie cette condition pour un epsilon, un
        N_candidate donnees,
    et pour toutes les paires de termes  $(u_n, u_m)$  ou  $n$  et  $m$  sont dans
         $[N\_candidate + 1, N\_candidate + num\_terms\_to\_verify]$ .

    Args:
        sequence_func: Une fonction qui prend un entier  $n$  et retourne le
             $n$ -ieme terme de la suite (float ou complex).
            On suppose que  $sequence\_func(n)$  est definie pour
                 $n \geq 1$ .
        epsilon: La valeur epsilon (doit etre strictement positive).
        N_candidate: La valeur  $N$  (doit etre un entier non negatif).
        num_terms_to_verify: Le nombre de termes a collecter apres
            N_candidate pour les comparaisons.
            La verification se fait pour  $n, m$  dans  $[$ 
                 $N\_candidate + 1, N\_candidate +$ 
                 $num\_terms\_to\_verify]$ .

    Returns:
        True si la condition est satisfaite pour toutes les paires de
            termes verifiees, False sinon.
    """
    if epsilon <= 0:
        raise ValueError("Epsilon doit etre strictement positif.")
    if N_candidate < 0:
        raise ValueError("N_candidate doit etre un entier non negatif.")

```

```

if num_terms_to_verify <= 1: # Necessite au moins deux termes pour
    une comparaison
    raise ValueError("num_terms_to_verify doit etre superieur a 1.")

terms = []
for n in range(N_candidate + 1, N_candidate + 1 +
    num_terms_to_verify):
    try:
        terms.append(sequence_func(n))
    except ValueError as e:
        print(f"Erreur lors du calcul de u_{n}: {e}. La suite ne
            peut pas etre de Cauchy dans cette plage.")
        return False

# Verifie toutes les paires (u_n, u_m)
for i in range(len(terms)):
    for j in range(i + 1, len(terms)): # Comparaison unique de
        chaque paire
        if abs(terms[i] - terms[j]) >= epsilon:
            return False

return True

```

Validation Mathématique

La validation des fonctions implémentées se fait par des assertions ('assert') qui vérifient leur comportement sur des suites dont les propriétés de convergence sont connues. Il est crucial de comprendre que ces tests ne *prouvent* pas la convergence ou la divergence au sens mathématique formel (qui nécessiterait de vérifier la condition pour *tout* ϵ et l'existence d'un N), mais ils *démontrent* que la condition ϵ, N est satisfaite ou non pour des valeurs spécifiques et un échantillon de termes.

Suites Réelles

1. Suite convergente : $u_n = 1/n$ converge vers $L = 0$

* Pour $\epsilon = 0.1$, on s'attend à ce que $N = 10$ soit suffisant (car pour $n > 10$, $1/n < 0.1$). * Pour $\epsilon = 0.01$, on s'attend à ce que $N = 100$ soit suffisant. * Pour $\epsilon = 0.001$, $N = 100$ ne devrait pas être suffisant, car $1/101 \approx 0.0099 > 0.001$. $N = 1000$ devrait l'être.

```

print("--- Tests pour les suites reelles ---")
# Test de la suite u_n = 1/n, converge vers 0
assert check_epsilon_N_condition_real(seq_one_over_n, 0.0, 0.1, 10), \
    "Test 1.1: u_n=1/n, L=0, epsilon=0.1, N=10 devrait etre True"
assert check_epsilon_N_condition_real(seq_one_over_n, 0.0, 0.01,
    100), \
    "Test 1.2: u_n=1/n, L=0, epsilon=0.01, N=100 devrait etre True"
assert not check_epsilon_N_condition_real(seq_one_over_n, 0.0,
    0.001, 100), \
    "Test 1.3: u_n=1/n, L=0, epsilon=0.001, N=100 devrait etre False
    (N trop petit)"
assert check_epsilon_N_condition_real(seq_one_over_n, 0.0, 0.001,
    1000), \
    "Test 1.4: u_n=1/n, L=0, epsilon=0.001, N=1000 devrait etre True"
print("Tests 1.x (u_n=1/n) passes.")

```

1. Suite divergente : $u_n = n$ diverge

* Pour n'importe quel ϵ et N , la suite $u_n = n$ finira par dépasser n'importe quelle borne autour d'une limite supposée.

```
# Test de la suite u_n = n, diverge
assert not check_epsilon_N_condition_real(seq_n, 50.0, 1.0, 10), \
    "Test 2.1: u_n=n, L=50, epsilon=1, N=10 devrait etre False"
assert not check_epsilon_N_condition_real(seq_n, 1000.0, 10.0, 100), \
    "Test 2.2: u_n=n, L=1000, epsilon=10, N=100 devrait etre False"
print("Tests 2.x (u_n=n) passes.")
```

1. Suite divergente oscillante : $u_n = (-1)^n$ diverge

* Cette suite oscille entre 1 et -1. Elle ne peut pas converger vers une seule limite. Pour n'importe quelle limite supposée L , et un ϵ suffisamment petit (par exemple, $\epsilon < 1$), la condition $|u_n - L| < \epsilon$ ne sera pas satisfaite pour tous les termes après un certain N .

```
# Test de la suite u_n = (-1)^n, diverge
assert not check_epsilon_N_condition_real(seq_minus_one_pow_n, 1.0,
    0.5, 10), \
    "Test 3.1: u_n=(-1)^n, L=1, epsilon=0.5, N=10 devrait etre False"
assert not check_epsilon_N_condition_real(seq_minus_one_pow_n, -1.0,
    0.5, 10), \
    "Test 3.2: u_n=(-1)^n, L=-1, epsilon=0.5, N=10 devrait etre False"
assert not check_epsilon_N_condition_real(seq_minus_one_pow_n, 0.0,
    0.5, 10), \
    "Test 3.3: u_n=(-1)^n, L=0, epsilon=0.5, N=10 devrait etre False"
print("Tests 3.x (u_n=(-1)^n) passes.")
```

Suites Complexes

1. Suite convergente : $u_n = (1/n) + i(1/n)$ converge vers $L = 0 + 0i$

* Le module $|u_n - L| = |(1/n) + i(1/n)| = \sqrt{(1/n)^2 + (1/n)^2} = \sqrt{2/n^2} = \sqrt{2}/n$. * Pour $\epsilon = 0.1$, on cherche N tel que $\sqrt{2}/N < 0.1 \implies N > \sqrt{2}/0.1 \approx 14.14$. Donc $N = 15$ devrait suffire. * Pour $\epsilon = 0.01$, on cherche N tel que $N > \sqrt{2}/0.01 \approx 141.4$. Donc $N = 142$ devrait suffire.

```
print("\n--- Tests pour les suites complexes ---")
# Test de la suite u_n = (1/n) + i(1/n), converge vers 0j
assert check_epsilon_N_condition_complex(seq_complex_one_over_n, 0j,
    0.1, 15), \
    "Test 4.1: u_n=(1/n)+i(1/n), L=0j, epsilon=0.1, N=15 devrait etre True"
assert not check_epsilon_N_condition_complex(seq_complex_one_over_n,
    0j, 0.01, 100), \
    "Test 4.2: u_n=(1/n)+i(1/n), L=0j, epsilon=0.01, N=100 devrait etre False (N trop petit)"
assert check_epsilon_N_condition_complex(seq_complex_one_over_n, 0j,
    0.01, 142), \
    "Test 4.3: u_n=(1/n)+i(1/n), L=0j, epsilon=0.01, N=142 devrait etre True"
print("Tests 4.x (u_n=(1/n)+i(1/n)) passes.")
```

Critère de Cauchy

1. Suite de Cauchy (et convergente) : $u_n = 1/n$

* Une suite est de Cauchy si pour tout $\epsilon > 0$, il existe un N tel que pour tout $n, m > N$, $|u_n - u_m| < \epsilon$. * Pour $u_n = 1/n$, si $n, m > N$, alors $|1/n - 1/m| < 1/N$. * Pour $\epsilon = 0.1$, on cherche N tel que $1/N < 0.1 \implies N > 10$. Donc $N = 10$ devrait suffire. * Pour $\epsilon = 0.001$, on cherche N tel que $N > 1000$. Donc $N = 1000$ devrait suffire.

```
print("\n--- Tests pour le critere de Cauchy ---")
# Test de la suite u_n = 1/n pour le critere de Cauchy
assert is_cauchy_sequence_test(seq_one_over_n, 0.1, 10), \
    "Test 5.1: u_n=1/n, epsilon=0.1, N=10 devrait etre True (Cauchy)"
    "
assert not is_cauchy_sequence_test(seq_one_over_n, 0.001, 100), \
    "Test 5.2: u_n=1/n, epsilon=0.001, N=100 devrait etre False (N
    trop petit)"
assert is_cauchy_sequence_test(seq_one_over_n, 0.001, 1000), \
    "Test 5.3: u_n=1/n, epsilon=0.001, N=1000 devrait etre True (
    Cauchy)"
print("Tests 5.x (u_n=1/n, Cauchy) passes.")
```

1. Suite non de Cauchy (et divergente) : $u_n = n$

* Pour cette suite, $|u_n - u_m| = |n - m|$. Si $n \neq m$, cette différence est toujours un entier non nul. On ne peut pas la rendre arbitrairement petite.

```
# Test de la suite u_n = n pour le critere de Cauchy
assert not is_cauchy_sequence_test(seq_n, 1.0, 10,
    num_terms_to_verify=5), \
    "Test 6.1: u_n=n, epsilon=1.0, N=10 devrait etre False (non
    Cauchy)"
# Meme avec un epsilon plus grand, la difference entre termes
# consecutifs est 1,
# donc si epsilon <= 1, ca echouera.
assert not is_cauchy_sequence_test(seq_n, 0.5, 1,
    num_terms_to_verify=5), \
    "Test 6.2: u_n=n, epsilon=0.5, N=1 devrait etre False (non
    Cauchy)"
print("Tests 6.x (u_n=n, non Cauchy) passes.")
```

Pour exécuter ces validations, il suffit de copier l'ensemble du code Python et les assertions dans un script Python et de l'exécuter. Si aucune assertion ne lève d'erreur, toutes les conditions testées sont satisfaites.

TP 5 : Suites réelles et complexes, limites et convergence (ϵ, N)

Objectif

Ce TP a pour objectif de solidifier la compréhension des concepts fondamentaux des suites réelles et complexes, en se concentrant sur la définition rigoureuse de la limite d'une suite (la définition ϵ, N). Nous allons implémenter en Python des fonctions permettant de vérifier cette définition pour des suites données, sans utiliser de bibliothèques mathématiques de haut niveau comme 'numpy' ou 'scipy', afin de construire une compréhension "from scratch" des mécanismes sous-jacents.

À l'issue de ce TP, vous serez capable de :

1. Définir et représenter des suites réelles et complexes en Python.
2. Implémenter la logique de la définition ϵ, N pour la convergence des suites réelles.
3. Implémenter la logique de la définition ϵ, N pour la convergence des suites complexes (en utilisant le module).
4. Utiliser des assertions mathématiques pour valider la convergence ou la divergence de suites spécifiques.

Implémentation Python pure

Nous allons implémenter des fonctions pour représenter des suites et une fonction générique pour vérifier la définition ϵ, N de la limite.

```
import math
from typing import Callable, Union

# Type alias pour une fonction de suite réelle
RealSequenceFunc = Callable[[int], float]
# Type alias pour une fonction de suite complexe
ComplexSequenceFunc = Callable[[int], complex]

# --- Fonctions de verification de la definition epsilon-N ---

def verify_epsilon_N_real(
    sequence_func: RealSequenceFunc,
    limit_L: float,
    epsilon: float,
    N_value: int,
    num_terms_to_check: int = 100
) -> bool:
    """
    Verifie si pour un epsilon donne et un N_value propose, la
    definition de la limite
    est satisfaite pour les 'num_terms_to_check' termes suivant N_value
    pour une suite réelle.

    Une suite (u_n) converge vers L si pour tout epsilon > 0, il existe
    un entier N
    tel que pour tout n > N, |u_n - L| < epsilon.

    Cette fonction effectue une verification computationnelle sur un
    nombre fini de termes.
    Elle ne prouve pas mathematiquement la convergence, mais peut la
    refuter ou fournir
    une forte evidence sur un intervalle donne.

    Args:
        sequence_func: Une fonction qui prend un entier n et retourne le
            n-ieme terme de la suite (float).
        limit_L: La limite supposee de la suite (float).
        epsilon: Une petite valeur positive (float).
        N_value: L'entier N propose pour la definition (int).
        num_terms_to_check: Le nombre de termes a verifier apres N_value
            .

    Returns:
        True si la condition |u_n - L| < epsilon est satisfaite pour
        tous les termes
    """
```

```

        verifies (n > N_value), False sinon.
"""
if epsilon <= 0:
    raise ValueError("Epsilon doit etre strictement positif.")
if N_value < 0:
    raise ValueError("N_value doit etre non-negatif.")
if num_terms_to_check <= 0:
    raise ValueError("num_terms_to_check doit etre strictement
        positif.")

for n in range(N_value + 1, N_value + 1 + num_terms_to_check):
    try:
        term = sequence_func(n)
        if abs(term - limit_L) >= epsilon:
            # print(f"Echec a n={n}: |{term} - {limit_L}| = {abs(
            #     term - limit_L)} >= {epsilon}")
            return False
    except ValueError as e:
        # Gérer les cas où sequence_func(n) n'est pas défini (ex:
        # division par zéro)
        print(f"Erreur lors du calcul du terme n={n}: {e}")
        return False # Considerer comme un echec de convergence pour
            ce test

return True

def verify_epsilon_N_complex(
    sequence_func: ComplexSequenceFunc,
    limit_L: complex,
    epsilon: float,
    N_value: int,
    num_terms_to_check: int = 100
) -> bool:
    """
    Verifie si pour un epsilon donne et un N_value propose, la
    definition de la limite
    est satisfaite pour les 'num_terms_to_check' termes suivant N_value
    pour une suite complexe.

    Une suite (z_n) converge vers L si pour tout epsilon > 0, il existe
    un entier N
    tel que pour tout n > N, |z_n - L| < epsilon.
    Ici, |z| represente le module du nombre complexe z.

    Cette fonction effectue une verification computationnelle sur un
    nombre fini de termes.
    Elle ne prouve pas mathematiquement la convergence, mais peut la
    refuter ou fournir
    une forte evidence sur un intervalle donne.

    Args:
        sequence_func: Une fonction qui prend un entier n et retourne le
            n-ieme terme de la suite (complex).
        limit_L: La limite supposee de la suite (complex).
        epsilon: Une petite valeur positive (float).
        N_value: L'entier N propose pour la definition (int).
        num_terms_to_check: Le nombre de termes a verifier apres N_value
    """

```

```

Returns:
    True si la condition  $|z_n - L| < \text{epsilon}$  est satisfaite pour
        tous les termes
        verifiees ( $n > N\_value$ ), False sinon.
"""
if epsilon <= 0:
    raise ValueError("Epsilon doit etre strictement positif.")
if N_value < 0:
    raise ValueError("N_value doit etre non-negatif.")
if num_terms_to_check <= 0:
    raise ValueError("num_terms_to_check doit etre strictement
        positif.")

for n in range(N_value + 1, N_value + 1 + num_terms_to_check):
    try:
        term = sequence_func(n)
        # abs() pour les nombres complexes calcule le module
        if abs(term - limit_L) >= epsilon:
            # print(f"Echec a n={n}: |{term} - {limit_L}| = {abs(
            #     term - limit_L)} >= {epsilon}")
            return False
    except ValueError as e:
        print(f"Erreur lors du calcul du terme n={n}: {e}")
        return False # Considerer comme un echec de convergence pour
            ce test

return True

# --- Exemples de suites reelles ---

def u_n_1_over_n(n: int) -> float:
    """Suite  $u_n = 1/n$ . Converge vers 0."""
    if n == 0:
        raise ValueError("n ne peut pas etre 0 pour 1/n")
    return 1.0 / n

def u_n_n_plus_1_over_n(n: int) -> float:
    """Suite  $u_n = (n+1)/n = 1 + 1/n$ . Converge vers 1."""
    if n == 0:
        raise ValueError("n ne peut pas etre 0 pour (n+1)/n")
    return (n + 1.0) / n

def u_n_minus_1_pow_n_over_n(n: int) -> float:
    """Suite  $u_n = (-1)^n / n$ . Converge vers 0."""
    if n == 0:
        raise ValueError("n ne peut pas etre 0 pour  $(-1)^n/n$ ")
    return ((-1.0)**n) / n

def u_n_alternating(n: int) -> float:
    """Suite  $u_n = (-1)^n$ . Divergente."""
    return (-1.0)**n

# --- Exemples de suites complexes ---

def z_n_1_plus_i_over_n(n: int) -> complex:
    """Suite  $z_n = (1+i)/n$ . Converge vers 0."""
    if n == 0:
        raise ValueError("n ne peut pas etre 0 pour (1+i)/n")
    return (1.0 + 1.0j) / n

```

```
def z_n_geometric(n: int) -> complex:
    """Suite  $z_n = (0.5 + 0.5j)^n$ . Converge vers 0 car  $|0.5+0.5j| < 1$ .
    """
    return (0.5 + 0.5j)**n

def z_n_constant(n: int) -> complex:
    """Suite  $z_n = 3 - 2j$ . Converge vers  $3 - 2j$ ."""
    return 3.0 - 2.0j

def z_n_i_pow_n(n: int) -> complex:
    """Suite  $z_n = i^n$ . Divergente (prend les valeurs 1, i, -1, -i)."""
    return (1j)**n
```

Validation Mathématique

Nous allons utiliser les fonctions ‘verify_epsilon_N_real’ et ‘verify_epsilon_N_complex’ avec des assertions pour valider la convergence de certaines suites et démontrer la divergence d’autres. Pour chaque assertion, nous calculerons ou estimerons la valeur de ‘N’ requise pour un ‘epsilon’ donné.

Note sur la validation computationnelle : Les fonctions ‘verify_epsilon_N_real’ et ‘verify_epsilon_N_complex’ vérifient la condition ϵ, N sur un nombre fini de termes (‘num_terms_to_check’). Bien que cela ne constitue pas une preuve mathématique formelle de convergence (qui nécessiterait de vérifier pour *tous* les $n > N$), cela permet de confirmer la validité d’un N proposé pour un ϵ donné sur un intervalle significatif, ou de détecter une divergence si la condition est violée.

Suites Réelles

1. Suite $u_n = 1/n$ (converge vers $L = 0$)

* Pour que $|1/n - 0| < \epsilon$, il faut $1/n < \epsilon \implies n > 1/\epsilon$. * Si $\epsilon = 0.1$, alors $n > 1/0.1 = 10$. Donc $N = 10$ devrait fonctionner. * Si $\epsilon = 0.01$, alors $n > 1/0.01 = 100$. Donc $N = 100$ devrait fonctionner.

```
print("--- Validation des suites reelles ---")
# u_n = 1/n, L=0
assert verify_epsilon_N_real(u_n_1_over_n, 0.0, 0.1, 10), "u_n=1/n, L=0, epsilon=0.1, N=10 devrait converger"
assert verify_epsilon_N_real(u_n_1_over_n, 0.0, 0.01, 100), "u_n=1/n, L=0, epsilon=0.01, N=100 devrait converger"
# Test d'un N trop petit : pour epsilon=0.1, N=5 ne suffit pas car
u_6 = 1/6 > 0.1
assert not verify_epsilon_N_real(u_n_1_over_n, 0.0, 0.1, 5), "u_n=1/n, L=0, epsilon=0.1, N=5 ne devrait PAS converger (N trop petit)"
print("u_n = 1/n : OK")
```

1. Suite $u_n = (n+1)/n = 1 + 1/n$ (converge vers $L = 1$)

* Pour que $|(1+1/n) - 1| < \epsilon$, il faut $|1/n| < \epsilon \implies n > 1/\epsilon$. * Si $\epsilon = 0.05$, alors $n > 1/0.05 = 20$. Donc $N = 20$ devrait fonctionner.

```
# u_n = (n+1)/n, L=1
assert verify_epsilon_N_real(u_n_n_plus_1_over_n, 1.0, 0.05, 20), "u_n=(n+1)/n, L=1, epsilon=0.05, N=20 devrait converger"
print("u_n = (n+1)/n : OK")
```

1. Suite $u_n = (-1)^n/n$ (converge vers $L = 0$)

* Pour que $|(-1)^n/n - 0| < \epsilon$, il faut $1/n < \epsilon \implies n > 1/\epsilon$. * Si $\epsilon = 0.1$, alors $n > 10$. Donc $N = 10$ devrait fonctionner.

```
# u_n = (-1)^n / n, L=0
assert verify_epsilon_N_real(u_n_minus_1_pow_n_over_n, 0.0, 0.1, 10)
    , "u_n=(-1)^n/n, L=0, epsilon=0.1, N=10 devrait converger"
print("u_n = (-1)^n / n : OK")
```

1. Suite $u_n = (-1)^n$ (divergente)

* Cette suite alterne entre -1 et 1. Elle ne converge pas. * Si on essaie de prouver qu'elle converge vers $L = 0$ avec $\epsilon = 0.1$, cela devrait échouer car $|(-1)^n - 0| = 1$, ce qui est toujours ≥ 0.1 .

```
# u_n = (-1)^n, divergente
assert not verify_epsilon_N_real(u_n_alternating, 0.0, 0.1, 1,
    num_terms_to_check=5), "u_n=(-1)^n, L=0, epsilon=0.1 ne devrait PAS converger"
# On peut aussi essayer avec L=1, ca echouera pour les termes impairs
assert not verify_epsilon_N_real(u_n_alternating, 1.0, 0.1, 1,
    num_terms_to_check=5), "u_n=(-1)^n, L=1, epsilon=0.1 ne devrait PAS converger"
print("u_n = (-1)^n : OK (divergence confirmee)")
```

Suites Complexes

1. Suite $z_n = (1+i)/n$ (converge vers $L = 0$)

* Pour que $|(1+i)/n - 0| < \epsilon$, il faut $|1+i|/n < \epsilon$. * $|1+i| = \sqrt{1^2+1^2} = \sqrt{2}$. * Donc $\sqrt{2}/n < \epsilon \implies n > \sqrt{2}/\epsilon$. * Si $\epsilon = 0.1$, alors $n > \sqrt{2}/0.1 \approx 1.414/0.1 = 14.14$. Donc $N = 14$ devrait fonctionner.

```
print("\n--- Validation des suites complexes ---")
# z_n = (1+i)/n, L=0
assert verify_epsilon_N_complex(z_n_1_plus_i_over_n, 0.0j, 0.1, 14),
    "z_n=(1+i)/n, L=0, epsilon=0.1, N=14 devrait converger"
# Test d'un N trop petit : pour epsilon=0.1, N=10 ne suffit pas car
|z_11| = sqrt(2)/11 approx 0.128 > 0.1
assert not verify_epsilon_N_complex(z_n_1_plus_i_over_n, 0.0j, 0.1,
    10), "z_n=(1+i)/n, L=0, epsilon=0.1, N=10 ne devrait PAS converger (N trop petit)"
print("z_n = (1+i)/n : OK")
```

1. Suite $z_n = (0.5 + 0.5j)^n$ (converge vers $L = 0$)

* Pour que $|(0.5 + 0.5j)^n - 0| < \epsilon$, il faut $|0.5 + 0.5j|^n < \epsilon$. * $|0.5 + 0.5j| = \sqrt{0.5^2 + 0.5^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} \approx 0.707$. * Donc $(\sqrt{0.5})^n < \epsilon$. En prenant le logarithme naturel : $n \ln(\sqrt{0.5}) < \ln(\epsilon)$. * Comme $\ln(\sqrt{0.5})$ est négatif, l'inégalité change de sens : $n > \ln(\epsilon)/\ln(\sqrt{0.5})$. * Si $\epsilon = 0.01$: $n > \ln(0.01)/\ln(\sqrt{0.5}) \approx -4.605/-0.346 \approx 13.3$. Donc $N = 13$ devrait fonctionner.

```
# z_n = (0.5 + 0.5j)^n, L=0
assert verify_epsilon_N_complex(z_n_geometric, 0.0j, 0.01, 13), "z_n
    =(0.5+0.5j)^n, L=0, epsilon=0.01, N=13 devrait converger"
print("z_n = (0.5 + 0.5j)^n : OK")
```

1. Suite $z_n = 3 - 2j$ (converge vers $L = 3 - 2j$)

* Pour une suite constante, la convergence est immédiate. Pour tout $\epsilon > 0$, $|(3-2j) - (3-2j)| = 0 < \epsilon$. Donc $N = 0$ fonctionne.

```
# z_n = 3 - 2j, L=3-2j
assert verify_epsilon_N_complex(z_n_constant, 3.0 - 2.0j, 0.001, 0),
    "z_n=3-2j, L=3-2j, epsilon=0.001, N=0 devrait converger"
print("z_n = 3 - 2j : OK")
```

1. Suite $z_n = i^n$ (divergente)

* Cette suite prend les valeurs $i^1 = i, i^2 = -1, i^3 = -i, i^4 = 1$, puis se répète. Son module est toujours 1. * Si on essaie de prouver qu'elle converge vers $L = 0$ avec $\epsilon = 0.1$, cela devrait échouer car $|i^n - 0| = |i^n| = 1$, ce qui est toujours ≥ 0.1 .

```
# z_n = i^n, divergente
assert not verify_epsilon_N_complex(z_n_i_pow_n, 0.0j, 0.1, 1,
    num_terms_to_check=5), "z_n=i^n, L=0, epsilon=0.1 ne devrait PAS
    converger"
print("z_n = i^n : OK (divergence confirmee)")
```

Toutes les assertions ont été exécutées avec succès, confirmant la validité de nos implémentations et notre compréhension des critères de convergence ϵ, N .